
STRUCTURED PARALLEL PROGRAMMING

M. Danelutto

April 9, 2024

Notes 2024

CONTENTS

1	Parallel & distributed computation concepts	1
1.1	General concepts	1
1.2	Examples	2
1.2.1	Book translation	2
1.2.2	Counting wireless devices in a classroom	4
1.2.3	Assembling conference bags	5
1.2.4	Serving people at bank office	6
1.3	Takeaway message	7
2	Kind of parallel computations	9
2.1	Data parallelism	10
2.2	Stream parallelism	11
2.3	Control parallelism	12
2.4	Task parallelism	12
2.5	Structured vs. unstructured	13
3	Parallel computation “shapes”	15
3.1	Data parallel shapes	15
3.1.1	Map	15
3.1.2	Reduce	16
3.1.3	Prefix	16
3.1.4	Stencil	16
3.1.5	Divide & Conquer	17
3.2	Stream parallel shapes	17
		iii

3.2.1	Pipeline	17
3.2.2	Farm	18
3.2.3	Composition of shapes	18
4	Algorithmic skeletons	21
4.1	Algorithmic skeletons: definition(s)	22
4.1.1	The skeleton advantage	24
4.1.2	The skeleton weakness	26
4.2	A simple skeleton framework	26
4.2.1	Control parallel skeletons	26
4.2.2	A skeleton framework	27
4.2.3	Skeleton programs	28
4.3	Algorithmic skeleton nesting	29
4.3.1	Sequential code wrapping	30
4.3.2	Full compositionality	32
4.3.3	Two tier model	32
4.4	Stateless vs. statefull skeletons	33
4.4.1	Shared state: structuring accesses	34
5	Parallel design patterns	37
5.1	Design patterns	37
5.2	Parallel design patterns	39
5.2.1	The parallel design pattern design spaces	40
5.2.2	Finding concurrency design space	41
5.2.3	Algorithm structure design space	43
5.2.4	Supporting structure design space	45
5.2.5	Implementation mechanisms design space	48
5.3	Sample parallel design pattern usage: image processing application design	49
5.3.1	Application analysis	49
5.3.2	Exploring the “finding concurrency” design space	49
5.3.3	Exploring the “algorithm structure” design space	50
5.3.4	Exploring the “supporting structures” design space	52
5.3.5	Exploring the “implementation mechanisms” design space	53
5.4	Comparing parallel design patterns and algorithmic skeletons	53
6	Skeleton design	57
6.1	Cole manifesto principles	58
6.1.1	Summarizing ...	60
6.2	Looking for (new) skeletons	60
6.2.1	Analysis	60
6.2.2	Synthesis	61
6.3	Skeletons vs templates	61
7	Implementation	63

7.1	Languages vs. libraries	63
7.1.1	New language	63
7.1.2	Library	64
7.2	Template based vs. macro data flow implementation	66
7.2.1	Template based implementations	66
7.2.2	Macro Data Flow based implementation	72
7.2.3	Templates vs. macro data flow	76
7.3	Component based skeleton frameworks	76
8	Portability	79
8.1	Portability through re-compiling	81
8.1.1	Functional portability	81
8.1.2	Performance portability	82
8.2	Portability through virtual machines	83
8.3	Heterogeneous architecture targeting	84
8.4	Distributed vs multi-core architecture targeting	86
8.4.1	Template/MDF interpreter implementation	86
8.4.2	Communication & synchronization	87
8.4.3	Exploiting locality	88
8.4.4	Optimizations	89
9	Rewriting techniques	93
9.1	Skeleton normal form	95
9.2	Model driven rewriting	97
9.3	ff-RPL shell design	99
9.4	Supported features	102
9.5	ff-RPL usage	104
9.6	Validation	107
9.7	Related work and conclusions	108
10	Adaptivity	111
10.1	Behavioural skeletons	117
10.1.1	Functional replication behavioural skeleton in GCM	119
10.1.2	Hierarchical management	121
10.1.3	Multi concern management	126
10.1.4	Mutual agreement protocol	129
10.1.5	Alternative multi concern management	132
11	Skeleton framework extendability	135
11.1	Skeleton set extension in template based frameworks	136
11.2	Skeleton set extension through intermediate implementation layer access	137
11.3	User-defined skeletons in muskel	137
Appendix A: Measures		143
A.1	Modeling performance	144

A.1.1	“Semantics” associated with performance measures	145
A.2	Different kind of models	146
A.3	Alternative approaches	149
A.4	Using performance models	150
A.4.1	Compile time usage	151
A.4.2	Run time usage	152
A.4.3	Post-run time usage	153
A.5	Amdahl law	154
A.6	Gustafsson	155
A.7	Work Span model	158
A.8	Skeleton advantage	160
A.9	Monitoring application behaviour	161
A.10	Performance model design	164
A.10.1	Analytical performance models	164
A.10.2	Queue theory	166
References		169

CHAPTER 1

PARALLEL & DISTRIBUTED COMPUTATION CONCEPTS

This chapter introduces some of the concepts related to parallel and distributed computing using examples from real life and the analogies that can be set up between real life execution of different tasks and parallel/distributed computations. Through simple examples, we point out the typical pros and cons of parallel computing and we also start discussing the common points of parallel and distributed computing along with the main differences between the two models.

1.1 GENERAL CONCEPTS

Imagine you need to execute some kind of task. And imagine that you may execute the task yourself or with the need of a team of “helpers”. Of course there are tasks that may benefit from the assistance of the helper and other kind of tasks that cannot take benefit, instead. Also, there are tasks that have to be executed just one, and other that have to be executed many times, one after the other, e.g. to produce a collection of goods out of a collection of components.

In all these different cases we can observe different features related to the way the assigned tasks are executed that may be common to different cases or represent true alternatives.

Sequential or parallel tasks. Tasks may be executed by a single person or by a team of persons. When a task is executed by a single person, we will claim the task is *sequential*. When different persons concur with their own work to the execution of a single task, we will claim the task is executed *in parallel* in all those cases where different steps contributing to the execution of the assigned task are executed “at the same time” by different workers.

Overhead. While executing a task sequentially, a set of actions (or steps) are executed by a single person, one after the other, respecting a given ordering among actions.

When executing the same task in parallel, additional actions (or steps) are needed to coordinate the work of the different persons participating to the execution of the overall task. These additional actions are considered to be *overheads*: additional activities not needed during sequential execution of the tasks that may be helpful to implement parallel execution of the same task while costing in terms of time spent and resources employed to execute them.

Speedup. Comparing the time needed to execute a given task sequentially—i.e. by a single person—with the time needed to execute the same task by a team of persons we may (hopefully) observe a difference. The ratio among the time spent by the single person and the time spent by the team of persons is called *Speedup*. In the most general case, we will be happy if a task requiring a given amount of time to a single person will require $\frac{1}{n}$ of that time to be executed by a team of n persons.

Sequences of tasks. A person may be charged to execute a single task or he/she may be charged to execute a sequence of tasks. Each one of the tasks in the sequence will usually be assigned after a given interval of time has elapsed since last task assignment. Such a sequence of tasks will be named a *stream* of tasks.

1.2 EXAMPLES

Let us consider some examples of tasks from real life along with possible ways of executing these tasks alone or with a team of helper workers. Each example discussed will focus on particular features of tasks and/or of the procedures that may be followed to execute them in team.

1.2.1 Book translation

Our first example is book translation. Suppose you are assigned to translate a book from English to Italian. Assuming you are Italian (or English) and you perfectly understand English (respectively, Italian), the procedure you may follow to translate the book is simple: for all the sentences in the English book, figure out a correct translation of the sentence in Italian. Let us assume the book is made of a given number of pages (say m) and we spend a given amount of time (say t_{page} = one hour) to translate a single page. The overall task will require something close to m hours to be completed. We may claim the translation of a book will take *sequentially* a time close to *number of pages* $\times$ *time needed to translate a page*, or, more formally:

$$T_{seq} = m \times t_{page}$$

What if a team of helpers is available, each sporting our very same abilities in English to Italian translation?

We can imagine a much more efficient procedure, employing the n helpers available:

- we divide the book in n parts
- we assign one part per helper
- helpers translate their own part
- we rebuild the translated book with an ordered merge of the different translated parts.

How much does it take now the whole book translation? Ideally each helper will get $\frac{m}{n}$ pages to translate. Therefore, assuming all helpers have our very same translation speed,

each helper will complete his own part in a time close to $\frac{m}{n} \times t_{page}$. The point here is that all helpers may start working independently of the other helpers as soon as they get their “assignment, that is their own part of book to be translated. In the ideal situation they all get their part at the same time, this means the actual translation work of the entire book will happen in a single $\frac{m}{n} \times t_{page}$ amount of time. Indeed, we must take into account some additional activities that concur to the organization of the helpers work and actually require an additional amount of time:

1. first of all, we need to count helpers, divide the book in as many part as the helpers count
2. then, we need to give one part to translate to each one of the helpers
3. eventually, after each one of the helpers finished his/her work, we need to gather the result (translated part) and put it in the right place in the final translated book.

The amount of time needed to execute all the steps (1, 2 and 3) above is clearly proportional to number of helpers and the size of our book does not impact too much. In general we may assume the time needed to “create a part” to be given to one of the helpers (say t_{split}), as well as the time needed to place the translated part in the right position of the final translated book (say t_{merge}), is smaller than the time needed to translate a page. However, we need to take into account that our *parallel* procedure to execute the task “book translation” takes a time

$$T_{par} = n \times t_{split} + \frac{m}{n} \times t_{page} + n \times t_{merge}$$

We assumed that $t_{split} \ll t_{page}$ and $t_{merge} \ll t_{page}$. Therefore the gain in time due to the fact $T_{seq} = m \times t_{page}$ is divided in T_{par} by n will not be impaired by the fact we spent an additional $n \times (t_{split} + t_{merge})$ amount of time to coordinate the work of the helpers. However, the activities in steps 1, 2 and 3 above are pure *overhead* with respect to sequential execution of the translate book task. Similarly, we classify as *overhead* the time contribute $n \times (t_{split} + t_{merge})$ in T_{par} .

Following all these arguments, we may define the speedup achieved when translating a book as

$$\text{speedup}(n) = \frac{T_{seq}}{T_{par}} = \frac{m \times t_{page}}{n \times (t_{split} + t_{merge}) + \frac{m}{n} \times t_{page}}$$

When $t_{split} \ll t_{page}$, $t_{merge} \ll t_{page}$ and $n \ll m$ the term $n \times (t_{split} + t_{merge})$ may be considered negligible and we may compute speedup as

$$\text{speedup}(n) \approx \frac{m \times t_{page}}{\frac{m}{n} \times t_{page}} = n$$

Discussion. We just discussed a procedure that may be used to complete a book translation in such a way the time needed is reduced proportionally to the number of helpers used. However a few additional facts must be considered:

- The whole idea of splitting book translation in parts may be applied as part translations are independent activities, i.e. may take place with no interaction among helpers. Indeed, there is some minimal unit that should be considered as the limit of the book splitting: the sentence. It makes no sense to split a single sentence among two helpers, as the “semantic” context is lost in that case, and probably correctness is not guaranteed. This limits in a sense the number of helpers we may use to achieve a “fast” translation of the book. The limitation is a logical limitation, that is it depends on the problem (task) features rather than on the procedure used to speed up translation.

- If the time spent to split a book part to be assigned to an helper plus the time spent to gather and place in the right position a translated part of the book is larger than the time spent to translate the part of the book, then the person in charge of orchestrating the parallel translation of the book would have better spent his/her time translating him/herself the book.
- If the number of helpers used is large, it may be again the case the time needed to prepare a part to be given to the single helper grows up to more than the time spent by the helper to translate his/her assignment. This means that after a given number of helpers adding further helpers will not decrease any more the time taken to translate the book, as the overhead becomes a relevant part of the parallel execution time.

1.2.2 Counting wireless devices in a classroom

Let's suppose a teacher needs to configure the wireless network access point in a classroom with a number of addresses equal to the number of wireless devices of the students. Therefore he/she needs to count the number of devices in the classroom.

The sequential procedure is simple: the professor starts asking the student in the left front corner of the classroom how many wireless devices he has, and sets this number as the starting number of devices. Then he/she proceeds asking the students on the right of the one just asked. Each time he gets an answer, he sums up the number to the already computed number of devices. When the end of the row is reached, the procedure is repeated for each one of the rows back to the first one. Overall, this *sequential* procedure takes number of rows (r) by numbers of seats in the row (s) times the time taken to sum two integers (t_+), that is

$$T_{seq} = r \times s \times t_+$$

In order to spare time, the teacher asks students at the extreme left position of the classroom sit rows to gather the sum of devices from the students of their row and then to answer his/her question concerning the number of devices with the row device count. All leftmost students will therefore collect the local sum in parallel. This will take a row

$$T_{row} = s \times t_+$$

time. Then professor will ask row totals, that will require:

$$T_{prof} = r \times t_+$$

Therefore, following this "parallel" procedure the total time required to compute the total amount of devices in the classroom will be

$$T_{par} = s \times t_+ + r \times t_+$$

Overall this will give a speedup

$$\text{speedup}(n) = \frac{(s \times r) \times t_+}{s \times t_+ + r \times t_+} = \frac{s \times r}{s + r}$$

We can do better, indeed. In general we may manage to organize students in the classroom in a binary tree, then we can ask "leaf" students to communicate their device number to their "father" ancestor and non leaf nodes to communicate to their "father" node the sum of the two values received by "son" nodes and of their own number of devices.

In this case the "root" student will receive the values to sum up after about $\log_2(\frac{r \times s}{2})$ steps all needing $2 t_+$, one to sum up the two received values and one to sum the computed value with the owned device count. The parallel time will therefore shrink down to

$$T_{par} = \lceil \log_2(\frac{r \times s}{2}) \rceil \times 2 t_+$$

assuming that all students in a three level may work in parallel, from the base of the three up to the root.

Discussion. In this case, we should point out that:

- The correctness of the results is ensured by the associativity and commutativity of the sum binary function. As a consequence of these two properties, any order in the execution of the $+$ operations leads to the correct result.
- Different intermediate procedures may be set up, depending on where we stop computing sequentially a subset of the overall result and we start computing the overall result in parallel out of the partial, sequentially computed results.
-

1.2.3 Assembling conference bags

The third example is completely different from the previous ones. The task consists in assembling a large number of conference bags. Each bag should include: the conference program, a conference badge, some gadget, a pen, the USB pen with the proceedings, etc. All items needed to assemble the bag are provided in boxes. Let us assume there are n items to be included in a conference bag, including the bag itself. Let us also assume that adding an item to the bag takes some t_{item} time. Eventually, let us suppose we need to assemble m bags.

A sequential procedure to assemble the bags may work like this:

- for each bag
 - gets a bag
 - insert first item in the box
 - ...
 - insert the last item in the box

Assembling a single bag requires therefore some a time such as $n \times t_{item}$. Therefore to assemble m boxes, a single person will spend:

$$T_{seq} = m \times (n \times t_{item})$$

In case a given number of helpers is available, we can organize the bag assembly in a different way, namely setting up an assembly line. We ask n helpers to reach the n boxes with the bag components. Then:

- the first helper, iterates getting a bag from the box and giving it to the second helper, the one putting in the bag the first item
- the other helpers, but the last one, will get the bag from previous helper, insert an item taken from their own box in the bag and give the bag to the next helper
- the last helper will receive the almost complete bag from the previous helper, put the last item in the bag and store the bag in the box of the complete bags.

Using this procedure, the time needed to assemble a single bag is roughly similar to the time needed by a single person to assemble the bag. However the bag has to traverse all the box positions (spending some $n \times t_{move\ time}$ time) and each helper will take a t_{item} to insert an item from box to bag, thus a total of $n \times t_{item}$ time is also required.

However, while helper i inserts his/her item in the bag j , helper $i + 1$ inserts his/her item in bag $j - 1$ and helper $i - 1$ inserts his/her item in bag $j + 1$. In fact, at a given time, all helpers include an item in a different bag. Thus after the first bag has been completed in another $t_{item} + t_{move}$ the last helper may produce a new completed bag. Overall, to produce m bags we will spend $n \times (t_{item} + t_{move})$ time to produce the first bag and then another $(m - 1) \times (t_{item} + t_{move})$ time to complete the rest of the bags:

$$T_{par} = n \times (t_{item} + t_{walk}) + (m - 1) \times (t_{item} + t_{move})$$

Compared to the sequential time this gives a speedup

$$\text{speedup}(n) = \frac{n \times m \times t_{item}}{(n + m - 1) \times (t_{item} + t_{move})}$$

Discussion. The procedure we used to assemble in parallel our bags has different peculiar features:

- The number of helpers we may use to execute the task is limited by the amount of items we need to insert in bags. Even if $w \gg n$ helpers were available there is no way to employ the extra $w - n$ to further reduce the time needed to assemble the bags.
- The simple model we used to figure out how much time we spent to prepare the m bags assumes each helper in the assembling chain spends the same amount of time to insert his/her item in the bag. In case this is not true, we must take into account that sooner or later the “slower” helper (i.e. the one spending more time to insert his/her item in the bag) will be overwhelmed by the partially completed bags coming from previous, faster helpers. He will only succeed delivering bags after inserting his/her item. Therefore after a while the whole chain of helpers will proceed with the speed of the slower helper. This means that, assuming helper i spends t_{item_i} time to include his/her item in the bag, we will have

$$t_{max} = \max\{t_{item_1}, \dots, t_{item_n}\}$$

$$T_{par} = \sum_i t_{item_i} + n \times t_{walk} + (m - 1) \times (t_{max} + t_{walk})$$

where the first part is the time needed to complete the first bag and the second one the time needed to complete the rest of the bags.

- In case we have an helper sensibly slower than the other ones, we may add other helpers to help him filling up bags with his/her item. In this way, each one of the helpers will still take a long time to add an item to the bag, but more helpers may take care of different bags at the same time. Therefore, using k helpers will lower the time needed to add the item to the bag from $t_{item_{slow}}$ to $\frac{t_{item_{slow}}}{k}$

1.2.4 Serving people at bank office

The last example in this part considers serving customer requests at a bank office. Let's suppose the bank saloon hosts a maximum of s sits for the employees serving customer requests, that there are e employees ($e < s$) and that an employee takes a t_e units of time (average) to serve a customer request. Customers come to the bank one each t_a units of time. Under these assumptions, one employee is able to serve $\frac{1}{t_e}$ customers per unit of time and therefore e employees are able to serve $\frac{e}{t_e}$ customers per unit of time. Then:

- if customers come to the bank with a rate such that $\frac{1}{t_a} < \frac{e}{t_e}$ (that is $t_a > \frac{t_e}{e}$) the bank waiting room will be mostly empty,
- if customers come to the bank with a rate such that $\frac{1}{t_a} > \frac{e}{t_e}$ (that is $t_a < \frac{t_e}{e}$) the queue will grow up constantly instead (the rate of the service will be less than the rate of the arrivals)

The optimum will be to have e such that $t_a = \frac{t_e}{e}$, obviously.

Let's now take into account a different perspective. Let's assume m customers are in the bank and let's try to understand how much time they take to be *all* served. We can make two different assumptions:

- customers line up for a single employee, and therefore we have e queues, each hosting initially $\frac{m}{e}$ customers;
- customers line up in a single queue and as soon as an employee is free it serves the first customer (FIFO) in the queue.

In the latter case, we can easily imagine that customers are served after $\frac{m \times t_e}{e}$ units of time. In the former case, however, in case we have slower and faster employees (that altogether provide anyway an average time of t_e units of time) we may have the customers lining up in front of the slower employee being served far after the ones lining up in front of the faster employee.

In this case, and assuming that customers cannot change queue, the time needed to serve all customers will be something as

$$T_c(m, e) = \frac{m}{e} \times \max\{t_{e_i}\}$$

with t_{e_i} representing the average time spent by employee i to serve a customer.

This situation can be interpreted also as the situation occurring at any point in time where a queue of m customers is present in the bank and the customers keep arriving at a rate such as the rate the employees can delivering serving customers.

The situation is worth taking into account as it shows as the task scheduling impacts time required to compute the tasks, but also because makes clear how, even in presence of kind of “static” scheduling (assignment of customer to employee) something can be done. As an example, customers in the long running queue can move to other employee queues as soon as they become shorter.

As a final point, let us take into account what if t_a varies. In case of large t_a part of the employees can take a rest, or start processing over “offline” work. In case t_a becomes smaller, then probably all the employees must be used to serve customer requests.

Consider the hypothesis in the two cases:

- when considering arrival rate of the customers, we are in taking into account that not all the “tasks to be computed” are present in a single point of time.
- when considering single queues we are instead considering a situation more similar to the one of example in Sec. 1.2.3, where “tasks to be computed” appear one at a time and there is no single point in time where all the tasks to be computed are present.

1.3 TAKEAWAY MESSAGE

From the three examples we discussed some different takeaway messages may be informally summarized.

Parallelism increases “performance”. Different procedures to introduce parallelism in the execution of a task may lead to benefits in terms of performance (e.g. time spent to complete the task).

Introduce overhead. In order to set up parallel execution of a task, a given amount of overhead is introduced that adds execution time to the time needed to execute the task sequentially.

Need coordination. Coordination is needed to organize parallel execution of tasks, which is not needed when a single person executes the task.

CHAPTER 2

KIND OF PARALLEL COMPUTATIONS

From the examples in Chap. 1 it is clear that there are different ways of exploiting parallelism in the execution of a task and different kind of tasks that require different techniques to be parallelized.

In this Chapter, we will review the different kinds of parallelism opportunities available, we will briefly discuss the main features of these different kind of parallelism opportunities and eventually we'll prepare the reader for the second part of this book, where the parallelism opportunities will be discussed in detail along with all the problems related to their efficient implementation. The level of abstraction used in this Chapter is necessarily very high. Differently from Chap. 1, we'll start using more classical computer related terms: tasks will stay there, helpers will become processes or threads, coordination will be evidenced in terms of communications and synchronizations, etc.

In particular, the generic “performance” terms used in Chap. 1 to refer time measures involved in the execution of some tasks will now be more precisely expressed as:

Latency/Completion time This is the time spent since the beginning to the end of the execution of a single task (latency) or since the beginning of the execution of the first task in a sequence to the end of the execution of the last task in the sequence (completion time).

Throughput/Service time This is the amount of tasks computed per unit of time when computing a sequence of tasks (Throughput). This is often expressed as the inverse measure, that is the time needed to output the results of a task belonging to a task sequence after having delivered the results of the previous tasks (service time).

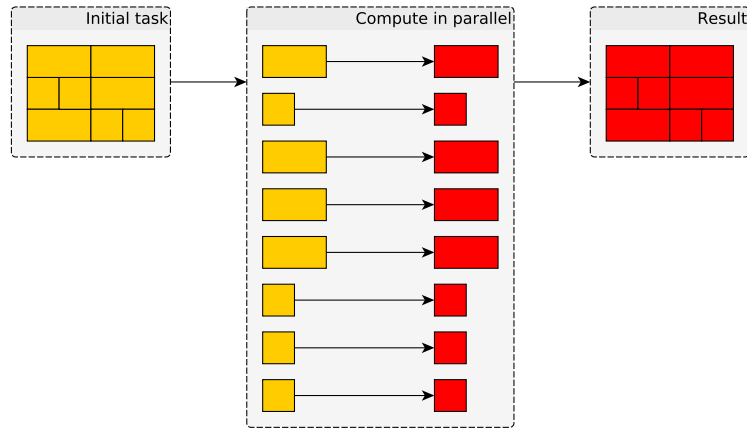


Figure 2.1 Data parallel computation

2.1 DATA PARALLELISM

Data parallelism is the parallelism resulting from the division of an initial task in subtasks, from the parallel computation of some result relative to each one of the subtasks and finally from the construction of the overall result relative to the initial task from the results computed for the subtasks. Fig. 2.1 outlines sample data parallel computations.

The work necessary to split task in subtasks and to rebuild the result of the task from the (partial) results computed for each one of the subtasks represent the overhead associated to the data parallel computation.

Sample data parallel computation: matrix multiplication. Matrix multiplication is the operation that given two matrixes A and B respectively of $n \times m$ and $m \times p$ rows and columns, produces the matrix C of $n \times p$ rows and columns. Item $c_{i,j}$ is computed as

$$c_{i,j} = \sum_{k=0}^m a_{i,k} b_{k,j}$$

In this case, each $c_{i,j}$ is a subtask that can be computed in parallel. All subtasks have the same computational weight. Alternatively, if $P = \{p_0, p_1, \dots, p_s\}$ represents a partition of the items $c_{i,j}$ in the matrix C , any of the p_i can be considered a subtask and computed in parallel to the other ones. Sample partitions are rows or columns of matrix C , as well as non overlapping groups of rows and columns covering the whole range of rows and columns (respectively). (*end of the example*)

Sample data parallel computation: differential equations. The temperature of a wire with one end kept at $0^\circ C$ and one kept at $100^\circ C$ may be approximated iteratively assuming the wire is represented by n segments and computing the temperature of each segment s_i at time t as the average of its neighbours s_{i-1} and s_{i+1} . We can split the computation of the averages in parts. For instance, we can split the wire in 2 parts, one computing averages in the first half and one computing the averages in the second part. However, the border points (e.g.

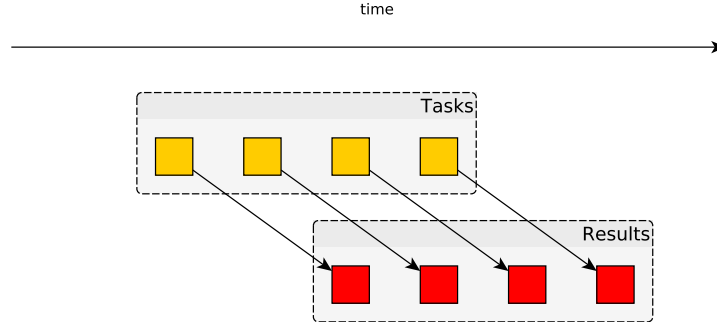


Figure 2.2 Stream parallel computation

$x_{n/2-1}$) needs some information from the other parallel computation (in this case $x_{n/2}$). This makes the two computations dependent on each other. (*end of the example*)

Different kind of data parallel computation differ in the way subtasks are obtained and managed. Embarrassingly data parallel computations partition task into subtasks that may be computed in a completely independent way and obtain the task results from some kind of trivial merge of the subtask results. Matrix multiplication is an embarrassingly data parallel computation, while the “differential equation” one is not *embarrassingly* parallel, just parallel.

The translation of a book discussed in Sec. 1.2.1 is an example of embarrassingly data parallel computation. The overall time needed to translate a single book is reduced with respect to sequential computation. No interaction is needed among the processes/threads in charge of executing the translation of the subtasks (book parts). The result of the book translation is trivially obtained by the ordered juxtaposition of the results of the translation of the different parts obtained from the division of the original book.

Data parallelisms is used to reduce the latency in the computation of tasks: a single task computed in T_{seq} time is computed into some time $T_{par}(n_w)$ function of the parallelism degree used (n_w) and hopefully smaller than T_{seq} . All the challenge here consists in balancing the time spent to split the task into subtasks and managing their parallel computation with the gain achieved from parallel computation itself.

2.2 STREAM PARALLELISM

Stream parallelism is the parallelism resulting from the parallel execution of different tasks (or of parts of different tasks) available at different times on an input task stream. Each task may be computed in parallel with the other tasks. However, there is no point in time where all the tasks to be computed are available altogether. If task x_i is available at time t_i , the next task x_{i+1} will be available at $t_{i+k} > t_i$. The interarrival time of tasks (i.e. the time $t_{i+1} - t_i$) clearly influences what can be done to compute in parallel the different tasks x_i . In case there is no dependency, either explicit or hidden, in between the computations relative to any tasks x_i and x_j the stream parallel computation is *embarrassingly parallel*.

Stream parallel computation sample: packet filtering. Discarding packets matching some conditions in a network packet stream is a stream parallel computation. Each packet can be checked for discard in parallel to any other packet. The convenient parallelism degree may be figured out depending on the amount of time spent to classify each packet. (*end of the example*)

Differently from data parallelism, stream parallelism may only be applied to sequences of tasks (stream of tasks). The computation of the single task is not affected, that is using stream parallelism the time needed to compute a single task in general is not decreased. Rather, stream parallelism increases the throughput of a computation. In general stream parallelism also concurs to reduce the completion time of the whole stream of tasks.

It must be taken into account that stream parallelism may be used to process *infinite* task streams and that some kind of dependency may be in place among computations of different tasks. As an example, in the packet filtering example, if we decide, after discarding a packet from a given stream flow (same source, same destination, say pck_k), that all packets from the same flow must be also discarded, then this makes some kind of dependency in that the result computed with packet pck_k impacts computations relative to all packets pck_j $j > k$.

The example discussed in Sec. 1.2.3 represents a typical example of stream parallelism. Building a single bag takes an amount of time similar (may be larger, due to the t_{move} additional times) to the one requested in the sequential case. However, the throughput is greatly increased (from $n \times t_{item}$ of the sequential case to $t_{item} + t_{move}$ of the parallel case with all items requiring the same time to be put in bags).

2.3 CONTROL PARALLELISM

Control parallelism is the parallelism arising from the execution of different parts of the same control flow: for instance, parts of an *if-then-else* construct, that is the condition, the *then* and the *else* branch.

Speculative parallelism is exploited when executing parallel computations that are not all needed to compute the final computation result. As an example the computation of if-then-else components may be implemented in parallel namely executing the computation of the condition and of the then and else branches in parallel. As soon as the condition is fully evaluated the computation of the non needed branch is aborted (or the results discarded, if already computed) and the computation of the needed branch is completed and its results are actually committed in the overall computation state.

2.4 TASK PARALLELISM

Task parallelism derives from the division of a parallel computation in *tasks*, possibly with *dependencies*. Tasks with no dependencies may be computed in parallel. A task is a computation processing some input data (inputs) to produce some output data (results). A dependency is a relation over tasks. If $(Task_i, Task_j)$ is a dependency, then the computation of $Task_i$ must terminate *before* computation of $Task_j$ starts. In other terms, $Task_j$ consumes as inputs some of the results produced by $Task_i$.

Task parallelism is used to reduce the latency of a computation. The computation is modelled as a task graph (task $\rightarrow$ nodes, dependencies $\rightarrow$ directed arcs between tasks). The initial node is the one with no input dependencies, i.e. a task t_0 such that there is no dependency (t_i, t_j) such that no output dependencies, i.e. a task t_k such that there is no dependency (t_i, t_j) such that Any task in the graph may be computes as soon as all the nodes in the path from the initial node to the task have been computed.

Task parallelism may be used to express both data parallel and stream parallel computations, indeed.

Data parallel computations may be modelled by a task splitting input data into partitions, a set of tasks processing each one of the partitions and a task collecting results from the computations of the single partition and rebuilding the final result. Dependencies are

from the initial split task and all the tasks computing partitions and from all the tasks computing partitions and the task rebuilding the final result.

Stream parallel computations may be modelled creating a task for each one of the input data sets to compute, instead.

2.5 STRUCTURED VS. UNSTRUCTURED

Very often we'll be using the terms structured and unstructured parallel programming, leveraging on the analogy with sequential programming world.

In sequential programming languages the term *structured* programming language refers to programming languages where the sequential control structure derives from composition of blocks such as conditional, iterative or function/procedure call statements. Formerly, unstructured programming languages control structure was build of sequence of statements where control was transferred through GOTO statements, leading to the so called "spaghetti like code".

In parallel programming models, we label as structured all those models where parallelism is exploited via composition of parallel patterns that model common and well defined parallel computations. The unstructured parallel programming models, on the other hand leave the parallel programmer free to build his/her own parallel applications through the usage of low level abstractions such as threads, processes, communication and synchronization primitives freely intermixed with the application business logic code, much in the way GOTOs were spread across sequential business logic code before structured programming languages in the sequential world.

Structured programming models

- (i) provide high level parallel patterns to the application programmers and
- (ii) expose the parallel orchestration of the activities within a given parallel application to the programming tools

As a consequence:

- desing and implementation of parallel applications is easier (programmer's tools are higher level and easier to use w.r.t. lowel level tools)
- part of the responsibilities related to parallel programming are moved from parallel programmers to programming tools (e.g. efficient implementation of know parallel patterns)
- automatic tuning of parallel applications may be implemented (due to the complete knowledge of the overall parallel structure of the application, as well as due to availability of pattern based optimization techniques)
- portability of parallel applications across different parallel architectures may be guaranteed, to a certain extend, both in functional and in performance terms (again, due to the complete knowledge available related to the overall parallel structure of the application, which can be used to transform/adapt implementation(s) across different taret architectures).

CHAPTER 3

PARALLEL COMPUTATION “SHAPES”

When considering parallel applications, independently of the languages and parallel programming frameworks chosen to implement them, a number of typical “shapes” of parallel computations can be identified that recur in multiple applications, alone or in composition. A “shape” in this context can be understood as a set of parallel computations with dependencies that take place and require proper orchestration to contribute to the delivery of the final result(s).

In this chapter, we briefly recall the main features of the most common parallel “shapes” listed accordingly to the kind of parallelism exploited. Later on we will discuss how this “shapes” have been interpreted, respectively, in the algorithmic skeleton community and in the parallel design pattern community.

The shapes considered in this short outline are the very basic ones. Although more shapes exist a) the ones presented here are sufficient to illustrate the concept and b) the set of shapes identified by different research groups sufficient to model, alone or on composition, the vast majority of parallel applications has a cardinality not too different from the cardinality of shapes considered here.

3.1 DATA PARALLEL SHAPES

Data parallel shapes assume to have access to some input “data collection” and compute a result typically out of the results of a number of “subcomputations” on partition (possibly overlapping of the input collection).

3.1.1 Map

A *map* applies a function over all the items of a “collection” of data $\mathcal{C}$ (a vector, a list, a tree). The result of the application of the $\text{map}(f)$ is a “collection” isomorphic to the input

one with generic item x substituted by $f(x)$. Given any two items $x, y \in \mathcal{C}$ the computation of $f(x)$ and $f(y)$ are completely independent in a map. Function f is assumed to be stateless in plain *maps*.

In short, representing a generic collection of data of m items as $\langle x_1, \dots, x_m \rangle$, the *map* shape may be interpreted as the parallel computation of

$$\text{map}(f)(\langle x_1, \dots, x_m \rangle) = \langle f(x_1), \dots, f(x_m) \rangle$$

Being α the type of the input collection items, f will have type $f : \alpha \rightarrow \beta$ with β the type of the items in the output collection.

The type of map is

$$\text{map} : (\alpha \rightarrow \beta) \times \langle \alpha \rangle \rightarrow \langle \beta \rangle$$

being $\langle \alpha \rangle$ the type of a collection of α items.

Parallelism in a map may be exploited in the computation of any two $f(x_i)$ and $f(x_j)$ for any $i \neq j$. In principle, if m parallel executors are available, all the $f(x_i)$ may be computed in parallel. The map shape is also labelled as “embarrassingly parallel”.

3.1.2 Reduce

A *reduce* computes the “sum” of a collection of data items using a—usually associative and commutative—binary function $\oplus$. The result of the computation of $\text{reduce}(\oplus)$ results in a single item, thus reducing a collection to a scalar (or to the collection base type).

In short, the reduce can be interpreted as the parallel computation of

$$\text{reduce}(\oplus)(\langle x_1, \dots, x_m \rangle) = x_1 \oplus \dots \oplus x_m$$

Being α the type of the data items in the input collection, then the type of the “sum” function will be $\oplus : \alpha \times \alpha \rightarrow \alpha$.

The type of reduce is

$$\text{reduce} : (\alpha \times \alpha \rightarrow \alpha) \times \langle \alpha \rangle \rightarrow \alpha$$

Parallelism in a reduce may be exploited (with associative and commutative $\oplus$ functions) by computing the result with a tree of nodes each computing an $\oplus$ instances. The leaves compute $\oplus$ of even and odd input collection items and then internal nodes compute $\oplus$ of the left and right subtrees up to the root node.

3.1.3 Prefix

A *prefix* computes the vector of the m partial “sums” of a collection with m items, using a—usually associative and commutative—binary function $\oplus$.

In short, the prefix can be interpreted as the parallel computation of

$$\text{prefix}(\oplus)(\langle x_1, \dots, x_m \rangle) = \langle x_1, x_1 \oplus x_2, \dots, x_1 \oplus \dots \oplus x_m \rangle$$

Being α the type of the data items in the input collection, then the type of the “sum” function will be $\oplus : \alpha \times \alpha \rightarrow \alpha$.

The type of prefix is

$$\text{stencil} : (\alpha \times \alpha \rightarrow \alpha) \times \langle \alpha \rangle \rightarrow \langle \alpha \rangle$$

3.1.4 Stencil

A *stencil* takes two functions (f and g) as parameters and applies to a collection of items. It computes an isomorphic collection of items. The i -th item in the result collection is the result of the parallel computation of

$$\text{stencil}(f, g)(\langle x_1, \dots, x_m \rangle) = \langle f(g(1, \langle x_1, \dots, x_m \rangle)), \dots, f(g(m, \langle x_1, \dots, x_m \rangle)) \rangle$$

where $g(i, coll)$ computes the neighbourhood set of item i in the input collection $coll$ and f computes a function of the items in the set.

The type of the stencil is

$$stencil : (\langle \alpha \rangle \rightarrow \beta) \times (item \times \langle \alpha \rangle) \times \langle \alpha \rangle \rightarrow \langle \beta \rangle$$

Parallelism in a stencil operation can be exploited as in a map pattern. In case we want the output collection to substitute the input one, care has to be taken to avoid that neighbourhood computations collect new values in place of the old (input) ones.

3.1.5 Divide & Conquer

A *divide&conquer* computes a result by dividing the input task in subtasks, computing partial results *per* subtask and eventually combining the partial results into the final result.

A *divide&conquer(isBc, solveBc, Divide, Conquer) : coll* computes in parallel the recursive function: if $isBc(coll)$ holds true, then return $Bc(coll)$. Otherwise return $Conquer(map(divide&conquer(isBc, Bc, Divide, Conquer): Divide(coll)))$;

Types of functions in this case, assuming a collection of items of type α in input are the following ones:

$$isBc : \langle \alpha \rangle \rightarrow boolean$$

$$Bc : \langle \alpha \rangle \rightarrow \beta$$

$$Divide : \langle \alpha \rangle \rightarrow \langle \langle \alpha \rangle \rangle$$

$$Conquer : \langle \beta \rangle \rightarrow \beta$$

and overall

$$divide\&conquer(isBc, solveBc, Divide, Conquer) :$$

$$(\langle \alpha \rangle \rightarrow boolean) \times (\langle \alpha \rangle \rightarrow \beta) \times (\langle \alpha \rangle \rightarrow \langle \langle \alpha \rangle \rangle) \times (\langle \beta \rangle \rightarrow \beta) \rightarrow \beta$$

Parallelism in a divide and conquer may be exploited in the map of divide&conquer calls over the results of the Divide operation.

3.2 STREAM PARALLEL SHAPES

Stream parallel shapes process streams of items transforming into another stream of items. A stream $\|x_1, \dots, x_m, \dots\|$ is a collection of items of a given type that are available at different points in time: first x_1 is available, then—after a non null interval of time t_1 , x_2 becomes available, and so on. Streams can also be infinite.

3.2.1 Pipeline

A *pipeline* computes in parallel the composition of a number of functions $f_1, \dots, f_k$ over all the items of a stream. Being α the type of the items of the input stream $\|x_1, \dots\|$ the pipeline can be interpreted as the parallel computation of

$$pipe(f, g)(\|x_1, x_2, \dots\|) = \|f_k(f_{k-1}(\dots f_1(x_1) \dots)), f_k(f_{k-1}(\dots f_1(x_2) \dots)), \dots\|$$

The type of the pipeline is (this is given for $k = 2$ for the sake of simplicity

$$pipeline : (\alpha \rightarrow \beta) \times (\beta \rightarrow \gamma) \times \|\alpha\| \rightarrow \|\gamma\|$$

Parallelism in pipeline can be exploited in the computation of different stages (functions) over partial results relative to different input streams.

3.2.2 Farm

A *farm* applies the same function f to all the items of an input stream, that is, a farm can be interpreted as the parallel computation of

$$farm(f)(\|x_1, x_2, \dots\|) = \|f(x_1), f(x_2), \dots\|$$

The type of the farm is therefore

$$farm : (\alpha \rightarrow \beta) \times \|\alpha\| \rightarrow \|\beta\|$$

It is worth pointing out that in general, the input output ordering may be not required, that is items of the output stream can be delivered out of order w.r.t. the items appearing on the output stream. In other words, it may be the case that having $f(x_i)$ appearing on the output stream before than $f(x_{i-k})$ may be tolerated.

Farm is the embarrassingly stream parallel shape. In principle, any item appearing on the input stream can be computed in parallel with all the other ones.

3.2.3 Composition of shapes

We assume that shapes can be freely composed while providing correct type checking.

As an example, given $f : \alpha \rightarrow \beta$ and $g : \beta \rightarrow \gamma$ we can build a $pipe(f, g) : \alpha \rightarrow \gamma$, but we can also build a $pipe(farm(f), g)$.

Data parallel shapes and stream parallel shapes may be also composed, usually with data parallel shapes used to speedup components in stream parallel computations (not viceversa).

E.G.▷ As an example, let us consider vector by vector multiplication. Given two vectors X and Y the result is defined as

$$\sum_{i=1}^n (x_i \times y_i)$$

The computation can be therefore implemented as a first map multiplying all the item element pairs followed by a reduce summing up all the results using the plain $+$ binary sum operation. In programming frameworks providing high level abstraction modelling our “parallel shapes”, obtaining the vector of pairs of the two input vectors is managed differently. Some frameworks include additional shapes (usually called “zip”) that take two vectors (collections) and return the vector (collection) of pairs build out of the items in the two vectors (collections) at the same position:

$$zip(\langle x_1, \dots, x_m \rangle, \langle y_1, \dots, y_m \rangle) = \langle (x_1, y_1), \dots, (x_m, y_m) \rangle$$

such that the vector product can be computed as

$$reduce(+)(map(*)(zip(X, Y)))$$

In other cases, we have polymorphic map shapes that can take more arguments (same type of collections) and apply the function passed as parameter to the map to the tuples build of the corresponding items in the input collections. This is the case of standard C++ library algorithms, where the `std::transform` algorithm may take

- start and end position of an input collection, start position of an output collection and function from type of the input collection to the type of output collection and return the result of the application of the function to items of the input collection stored in the output collection.

Assuming two `std::vector<int> x` and `y` the statement `std::transform(x.begin(), x.end(), y.begin(), f);` assigns to the `y[i]` the values `f(x[i])`.

- or, start and end position of a first input collection, start position of a second input collection, start position of an output collection and function from types of the two input

collections to the type of output collection and return the result of the application of the function to zipped items of the two input collections stored in the output collection.

Assuming three `std::vector<int> x, y` and `z` the statement `std::transform(x.begin(), x.end(), y.begin(), z.begin(), f);` assigns to the `z[i]` the values `f(x[i], y[i])`.

■

CHAPTER 4

ALGORITHMIC SKELETONS

Researchers studying how to implement efficient parallel applications on parallel target architectures observed that the *patterns* used to exploit parallelism in *efficient* parallel applications are not a large number.

They observed that there are just a small number of common patterns that model common parallelism exploitation forms and that are used and re-used in different places, possibly nested to model the more complex parallelism exploitation strategies.

These researchers also observed further and—possibly—more important facts:

- The correct and efficient programming—that is the design, coding, debugging and fine tuning—of these parallel patterns constitutes the major programming effort when implementing efficient parallel applications.
- The knowledge needed to implement efficient parallel patterns is completely different from the knowledge needed to program the functional logic of the application¹.
- Applications clearly separating the code used to implement parallel patterns from the application functional code were much more easy to develop, debug and tune, with respect to the applications where parallelism implementation code and functional code was completely intermingled.

As a consequence, these researchers started asking themselves if some benefit could be achieved from the design and development of parallel patterns *per se*, that is parallelism exploitation patterns studied without taking into account the final application functional code. In turn, this led—in different times and within different research communities—to the development of two distinct research branches:

¹that part of the application code actually computing the result of the application

1. In late '80s, Cole introduced the *algorithmic skeleton* concept and a community of researchers started working and improving the algorithmic skeleton concept. This research community was mainly build by researchers coming from High Performance Computing community. Their major concern was the development of programming frameworks suitable to support efficient parallel programming, in particular with respect to the performance achieved in parallel programs. The community evolved and the concept of algorithmic skeleton evolved as well. Nowadays there are several research groups building on this concept around the world. Algorithmic skeletons are the main subject of this book and they will be thoroughly discussed from this Chapter on.
2. In the '00s, software engineering researchers evolved the design pattern concept into *parallel design patterns*. The parallel design patterns has been and are currently investigated by a community which is fairly disjoint from the one investigating algorithmic skeletons. This notwithstanding the research on parallel design patterns produced (and it is currently producing) results that are similar to those produced in the algorithmic skeleton framework. Sometimes the results only differ in the “jargon” used to illustrate them. In more interesting cases they are similar but the different—more software engineering oriented—perspective used in investigating the results may provide useful hints to algorithmic skeleton designers as well.

4.1 ALGORITHMIC SKELETONS: DEFINITION(S)

Algorithmic skeleton concept has been introduced by Murray Cole with his PhD thesis in 1988 [18]. Cole defined a skeleton programming environment as follows:

The new system presents the user with a selection of independent “algorithmic skeleton”, each of which describes the structure of a particular style of algorithm, in the way in which higher order functions represent general computational frameworks in the context of functional programming languages. The user must describe a solution to a problem as an instance of the appropriate skeleton.

Algorithmic skeletons were defined as pre-defined patterns encapsulating the *structure* of a parallel computation that are provided to user as building blocks to be used to write applications. Each skeleton corresponded to a single parallelism exploitation pattern. As a result, a skeleton based programming framework was defined as a programming framework providing programmers with algorithmic skeletons that may be used to model the parallel structure of the applications at hand. In a skeleton based programming framework the programmer has no other way (but instantiating skeletons) to structure his/her parallel computation. In the original definition of Cole, there was no mention to the fact that skeletons could be nested. Therefore any skeleton application was the result of the instantiation of a single algorithmic skeleton and thus, in turn, of a single parallelism exploitation pattern.

Algorithmic skeletons were introduced as Cole recognized that most of the parallel applications exploit well know parallelism exploitation patterns. Rather than requiring the application programmers to rewrite from scratch all the code needed to implement these parallelism exploitation patterns when a new application has to be implemented, it was therefore more natural to provide the patterns as building blocks that the programmers could instantiate by providing the “business logic”² of their application.

²see business logic definition in Fig. fig:txt:business logic

The term “business logic” usually denotes the functional code of the application, distinct from the non functional code modelling, as an example, the implementation of parallelism in the application. In an application processing a set of images to apply to each one of the images a filter f , the code for the filter *is* business logic code, while the code needed to set up a number of parallel/concurrent/distributed agents that altogether contribute to the parallel computation of the filtered images *is not*.

Figure 4.1 Business logic

E.G.▷ As a motivating example, consider parameter sweeping applications. A parameter sweeping application is an application where a single code is run multiple times to evaluate the effect of changes in input parameters. Each run of the code on a different input parameter set is completely independent of the other runs. Therefore a parameter sweeping application may be easily implemented adopting a master/worker for the definition of the master/worker pattern) parallelism exploitation pattern. A master schedules different input parameter sets to a set of workers running the application code and gathers the result items produced by these workers in such a way the user may evaluate the effects of “parameter sweeping” and possibly iterate the process up to the point he/she finds the proper parameter set (the one providing the expected results). Now all the code needed to implement the master, to properly schedule input parameter sets to the workers and to gather the results is independent of the actual code used to implement the “function” computed by the application. The only dependency is on the types of the input parameter set and of the output results, actually. Therefore, an algorithmic skeleton can be provided to the user that accepts parameters such as the input and output types and the code needed to compute output out of the input data. The skeleton completely takes care of the implementation of the master/worker parallelism exploitation pattern needed to implement the overall parameter sweeping application. Being the code of the skeleton designed and implemented by (presumably) expert system programmers, well aware of the features of the target architecture at hand, the application programmers will succeed developing an efficient parameter sweeping application without being required to have any kind of knowledge related to target architecture or to the techniques needed to efficiently implement a master worker schema. ■

Later on, Cole published his “skeleton manifesto” [19] where skeletons are described in a fairly more abstract way:

many parallel algorithms can be characterized and classified by their adherence to one or more of a number of generic patterns of computation and interaction. For example, many diverse applications share the underlying control and data flow of the pipeline paradigm. Skeletal programming proposes that such patterns be abstracted and provided as a programmer’s toolkit, with specifications which transcend architectural variations but implementations which recognize these to enhance performance.

Eventually, the “Skeletal parallelism” home page maintained by Murray Cole³ gives a more general definition of algorithmic skeletons:

There are many definitions, but the gist is that useful patterns of parallel computation and interaction can be packaged up as ‘framework/second order/template’ constructs (i.e. parameterized by other pieces of code), perhaps presented without reference to explicit parallelism, perhaps not. Implementations and analyses can be shared between

³<http://homepages.inf.ed.ac.uk/mic/Skeletons/>

instances. Such constructs are ‘skeletons’, in that they have structure but lack detail. That’s all. The rest is up to us.

As we see, Cole’s definition of algorithmic skeleton evolved in time. In the meanwhile, several research groups working on algorithmic skeletons contributed to the evolution of the algorithmic skeleton concept. Eventually, a new definition of algorithmic skeletons has been elaborated, which is nowadays commonly accepted. This new definition may be summarized as follows:

An algorithmic skeleton is parametric, reusable and portable programming abstraction modelling a known, common and efficient parallelism exploitation pattern.

Such definition embeds several different, fundamental concepts:

- it models a *known, common* parallelism exploitation pattern. We are not interested in patterns that are not commonly found in working parallel applications. Possibly, uncommon patterns could be anyway used with those skeleton frameworks that provide some degree of extensibility (see Sec. 11).
- it is *portable*, that is an efficient implementation of the skeleton should exist on a range of different architectures.
- it models *efficient* parallelism exploitation patterns. Skeletons only model those patterns that have some known, efficient implementation on a variety of target architectures.
- it is *reusable*, that means it could be used in different applications/contexts without actually requiring modifications
- it is *parametric* in that it accepts parameters that can be used to specialize *what* is computed by the skeleton, rather than *how* the parallel computation is performed.

4.1.1 The skeleton advantage

What are the advantages provided by a programming framework based on algorithmic skeletons? Let us assume that a skeleton programming framework is available, providing the programmer with a convenient number of skeletons.

The implementation of parallel applications using such a framework requires the following steps:

1. the possible parallel structures of the application⁴ are evaluated
2. the available skeletons are evaluated and a skeleton (nesting) is individuated as the one modelling the application parallel structure
3. the parallel application code is produced by instantiating the skeleton (nesting) chosen, providing the suitable functional (e.g. the business logic parameters) and non functional (e.g. parallelism degree of the skeleton(s) used) parameters
4. the application is compiled
5. the application is run on the target architecture
6. the application results are analyzed:

⁴that is the possible ways to exploit application internal parallelism

- for functional correctness: functional correctness of skeleton implementation is guaranteed by the framework, but the correctness of the instantiation of the skeletons, as well as of the business logic code has to be checked anyway
 - for non functional correctness: performance has to be measured, looking for potential bottlenecks or inefficiencies.
7. in case of inefficiencies, refinements of the current skeleton structure of the application as well as alternative skeletons (or skeleton nestings) are considered, and the process is restarted from phase 3.

This process is radically different from the process used to develop parallel applications with traditional programming frameworks, such as MPI for instance. In that case, the application programmer must go through the whole process of managing all the details relative to parallelism exploitation, while in this case *all the implementation details relative to parallelism exploitation patterns are managed by the skeleton framework*.

This has several consequences, that represent the fundamental advantages of programming frameworks based on algorithmic skeleton⁵:

- the abstraction level presented to the programmer by the programming framework is higher than the one presented by traditional parallel programming frameworks. This in turn has two consequences:
 - The whole process of *parallel* programming is simplified. Parallelism exploitation mainly consist in properly instantiating the skeleton abstractions provided by the framework rather than programming your own parallel patterns using the concurrency, communication and synchronization mechanisms available.
 - Application programmers may experiment *rapid prototyping* of parallel applications. Different application prototypes may be obtained by instantiating (that is by providing the proper functional/business logic code) different skeletons among those available or, in case this is allowed, different compositions of existing skeletons.
- The correctness and the efficiency of the parallelism exploitation is not a concern of the application programmers. Rather, the system programmers that designed and implemented the skeleton framework guarantee both correctness and efficiency.
- Portability on different target architectures is guaranteed by the framework. An algorithmic skeleton application can be moved to a different architecture just by recompiling⁶. Portability has to be intended in this case not only as “functional portability”, but also as “performance portability” as the skeleton framework usually exploits best implementations of the skeletons on the different target architectures supported.
- Application debugging is simplified. Only sequential code (that is business logic code) has to be debugged by the application programmers.
- Possibilities for static and dynamic optimizations are much more consistent than in case of traditional, unstructured parallel programming frameworks as the skeleton structure completely exposes to the programming tools the parallel structure of the application.
- Application deployment and (parallel) run, that notably represent a complex activity, is completely in charge of the skeleton framework

⁵as clearly pointed out in Cole’s manifesto[19]

⁶if the algorithmic skeleton framework supports the new target architecture, of course.

4.1.2 The skeleton weakness

Of course, in addition to the advantages of algorithmic skeleton based programming frameworks with respect to traditional, unstructured parallel programming frameworks as listed in the previous section, there are also disadvantages than must be taken into account. The principal drawbacks are related to both “user” (i.e. application programmer) and “system” (i.e. system programmer) views:

- In case none of the skeletons provided to the programmer (or none of the possible skeleton nestings) suits to model the parallel structure of the application the application could not be run through the skeleton framework (see also Sec. 11).
- Functional and performance portability across different target architectures requires a huge effort by the skeleton framework designers and programmers. The effort required may be partially reduced by properly choosing the sw target architecture targeted by the skeleton framework, e.g. by considering some “higher level” target to produce the “object” code of the skeleton framework⁷.

4.2 A SIMPLE SKELETON FRAMEWORK

Although different programming frameworks based on skeletons have been developed that sport each different algorithmic skeletons, a subset of common skeletons may be found in most of these frameworks. These common subset contains skeletons from three different classes of skeletons: stream parallel skeletons, data parallel skeletons and control parallel skeletons.

We assume that stream and data parallel skeletons are included in our simple skeleton framework, modelin the parallel “shapes” discussed in Chap. 3.

In addition, we may condider a few “control” parallel additional skeletons. These skeletons derive from possible parallelism exploited in the flow of control of particular (sequential) patterns such as conditional statements and loops.

4.2.1 Control parallel skeletons

Control parallel skeletons actually do not express parallel patterns. Rather, they express structured coordination patterns and model those parts of a parallel computation that are needed to support or coordinate parallel skeletons.

Control parallel skeletons typically include:

Sequential code wrapper skeleton which is used to encapsulate sequential portions of code in such a way they can be used as parameters of other (parallel) skeletons.

Conditional skeleton which is used to model *if-then-else* computations. Such a skeleton can be expressed by the following higher order function:

```
ifthenelse(c,t,e)(x) = if(c(x)) then t(x) else (e(x));
```

whose type is

$$ifthenelse : (\alpha \rightarrow bool) \rightarrow (\alpha \rightarrow \beta) \rightarrow (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta$$

⁷as an example, targeting MPI rather than UDP sockets + Pthreads. MPI provides a virtualization layer that takes care of handling differences in the data representation among different systems/processors, as an example. Therefore less non functional code is needed to run an MPI application than a Pthread/UDP application. In turn, this means the task of the skeleton compiler is easier.

Iterative skeleton which is used to model definite or indefinite iterations. As an example, a *skwhile* iterative skeleton may be represented through the higher order function

```
while(c, b)(x) = if(c(x)) then while(c,b)(b(x)) else x;
```

whose type is

$$skwhile : (\alpha \rightarrow bool) \rightarrow (\alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow \alpha$$

These skeletons do not model any parallelism exploitation patterns, although some of them may lead to parallel computations. As an example:

- A conditional skeleton *if-condition-then-else* may be implemented in such a way the computation of the *condition*, *then* and *else* parts are started concurrently. Once the *condition* is evaluated, either the *then* termination is awaited and the *else* computation is aborted or vice versa, depending on the result given by the computation of *condition* (this parallel semantics implements a kind of *speculative* parallelism).
- An iterative skeleton may be implemented in such a way a number (possibly all) the iterations are executed concurrently, provided that there are no *data dependencies* among the iterations (this is a kind of *forall* skeleton, actually, which is very similar to the *map* one. The only difference concerns the way the parallel computations are derived: in the *map*, they come from the decomposition of the compound input data structure; in the *forall* they come from different iterations of the same loop).

4.2.2 A skeleton framework

With the definitions given up to now, we can imagine a *skeleton programming framework* as a system providing at least:

1. a number of skeletons, described by a grammar such as:

```
Skel ::= StreamParallelSkel |
        DataParallelSkel | ControlParallelSkel
StreamParallelSkel ::= Farm | Pipe | ...
DataParallelSkel ::= Map | Reduce |
                    ParallelPrefix | ...
ControlParallelSkel ::= Seq | Conditional |
                       DefiniteIter |
                       IndefiniteIter | ...
Seq ::= seq(<sequential function code wrapping>)
Farm ::= farm(Skel)
Pipe ::= pipeline(Skel, Skel)
Map ::= map(Skel)
Reduce ::= reduce(Skel)
ParallelPrefix ::= parallel_prefix(Skel)
Conditional ::= ifthenelse(Skel, Skel, Skel)
DefiniteIter ::= ...
...
```

The framework provides application programmers with the API to write *Skel* expressions denoting the skeletal program as well as some statement invoking the execution of the skeletal program. As an example, in Muskel [7] a *Skel* program can be declared as:

```
Skeleton filter = new Filter(...);
Skeleton recon = new Recognize(...);
Skeleton main = Pipeline(filter, recon);
```

2. a way to invoke the execution of the skeleton program. As an example, in Muskel this can be demanded to an Application Manager by passing it as parameters i) the program to execute, ii) the input stream and iii) the output stream manager⁸:

```
Manager mgr =
    new Manager(main,
        new FileInputStreamManager("datain.dat"),
        new ConsoleOutputStreamManager());
mgr.start();
```

Alternatively, the skeleton framework may provide library calls directly implementing the different skeletons. As an example, in frameworks such as SkeTo [49] the skeletons are declared *and* executed via suitable library calls: to use a map skeleton followed by a reduce skeleton the user may write a code such as

```
dist_matrix<double> dmat(&mat);
dist_matrix<double> *dmat2 =
    matrix_skeletons::map(Square<double>(), &dmat);
double *ss =
    matrix_skeletons::reduce(Add<double>(),
                             Add<double>(), dmat2);
```

In both cases, provided the framework is properly set up, the evaluation of the skeleton program leads to the proper exploitation of the parallel patterns modelled by the skeletons used. In the former case (Muskel, which is based on Java), the execution of the program through a plain Java interpreter will lead to the execution of the first and second pipeline stages on two different computing resources interconnected using TCP/IP sockets and Java RMI protocols. This, in turn, implements the pipeline parallel semantics as defined in Sec. 4.2.2. In the latter case (SkeTo, which is based on C++), the execution of the program compiled using a C++ compiler and linked with an MPI library leads to the execution in parallel, onto the available processing elements of both the map and the reduce skeletons, in sequence.

4.2.3 Skeleton programs

Skeleton programs written using a framework such as the one envisioned in the previous section have a well defined structure, that is usually represented by a *skeleton tree*.

The skeleton tree has nodes representing algorithmic skeletons and leaves representing sequential portions of code (the business logic code). Skeleton parameters are represented in node leaves (Fig. 4.2 shows an example of skeleton tree).

We will use in the following skeleton trees in different contexts: as an example, to represent parallel program classes, to expose the parallel structure of an application or to represent compiler data.

With a framework such as the one outlined in the previous sections, a skeleton tree may be defined according to the following grammar⁹:

⁸input and output stream managers are *de facto* iterators producing the items of the input stream and consuming the items of the output stream, respectively

⁹The grammar abstracts all non skeleton parameters, that are not significant when representing the parallel structure of the skeleton program.

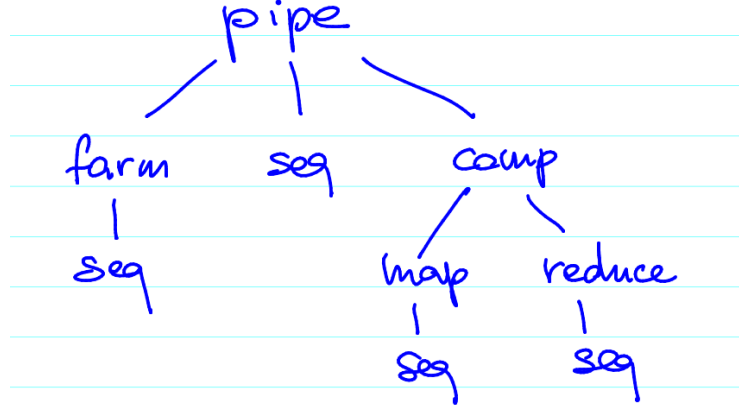


Figure 4.2 Sample skeleton tree

```

Sk ::= Seq(Code)
    | Farm(Sk) | Pipe(Sk, Sk)
    | Map(Sk) | Reduce(Sk) | Stencil(Sk)
    | While(Sk) | For(Sk) | ...

```

As an example, the skeleton tree of a program filtering a stream of images by means of two filters, with one of the two filters being data parallel, may be represented by the term

$$\text{Pipe}(\text{Map}(\text{Seq}(\text{dpFilter})), \text{Seq}(\text{secondFilter}))$$

or, in case the second filter is parallelized through a farm, by the term

$$\text{Pipe}(\text{Map}(\text{Seq}(\text{dpFilter})), \text{Farm}(\text{Seq}(\text{secondFilter})))$$

These expressions only expose the parallel structure of the two programs. There is no parameter modelling the parallelism degree of the map and farm, as an example, nor there are parameters modelling the data types processes by `dpfilter` or `secondFilter`.

4.3 ALGORITHMIC SKELETON NESTING

Algorithmic skeletons, as proposed by Cole in his PhD thesis [18] were conceived as patterns encapsulating a single parallel pattern. In the thesis these patterns were used alone. Therefore, an algorithmic skeleton based application exploited just a single, possibly complex form of parallelism.

The algorithmic research community immediately started to investigate the possibilities offered by algorithmic skeleton composition. For a while, two perspectives have been investigated:

“CISC” skeletons Several researchers imagined to provide users with a large number of algorithmic skeletons [34], each modelling a particular parallelism exploitation pattern¹⁰. These skeletons were derived analyzing existing applications and looking for *any* pattern that demonstrated to be useful and efficient (e.g. scalable, performant,

¹⁰We introduce the terms “CISC” and “RISC skeletons” although this terminology has never been used before in the skeleton community. However, we think that CISC and RISC perfectly summarize the two approaches discussed here.

etc). Due to the large amount of different parallel applications, and due to the large number of significantly or slightly different parallelism exploitation patterns, eventually CISC skeleton set became huge. In turn, this implied that possible users (the programmers) were forced to dig into the skeleton set the one matching their needs. The effort needed to individuate the “best” algorithmic skeleton matching the parallelism to be exploited in the user application was sensible, due to large number of similar items in the skeleton set.

Furthermore, in case of CISC skeletons, nesting of skeletons was in general not allowed. This means that if you succeeded to model your application with one of the available skeletons and later on you realized the achieved performance was not satisfactory, you were not allowed to *refine* the skeleton structure of the program introducing further skeletons as arguments of the existing ones. Instead, you were forced to remove the (single) skeleton used and to pick up another skeleton in the set provide to the programmers. In turn, this could require changes in the parameters passed, that is in the business logic code, which is usually something we want to avoid as much as possible¹¹.

“RISC” skeletons Other researchers imagined to follow a RISC approach in the design of the skeleton set provided to the application programmers. That is, they investigated skeleton sets with a small number of items. These skeletons modelled very basic parallelism exploitation patterns. Each of the “code” parameters of the skeletons, however, could be recursively provided as skeletons. The envisioned scenario behind this design choice allowed complex parallelism exploitation patterns to be defined as composition of primitive parallelism exploitation patterns.

In this case, the application programmer were required to know only a small set of base skeletons plus another small number of “composition rules”. It is clear that composition rules play a significant role in this case.

After a while, most of the researchers working on algorithmic skeletons opted for some kind of variant of the second approach and most of the algorithmic skeleton framework developed so far include the possibility to nest skeletons. However, different frameworks offer different composition/nesting possibilities. To understand the different cases, we start discussing how (possibly existing) sequential portions of business code may be included in a algorithmic skeleton program (Sec. 4.3.1). Then we will discuss the problems related to implementation of full algorithmic skeleton compositionality (Sec. 4.3.2) and eventually we will discuss the more common “non full” composition rules, namely those based on a *two tier* approach (Sec. 4.3.3).

4.3.1 Sequential code wrapping

In an algorithmic skeleton program sequential portions of code are needed to implement the application business code.

E.G.▷ As an example, in an application whose parallelism exploitation is modelled after a two stage pipeline skeleton, the functions computed by the two stages represent business logic code and may be supplied re-using existing code, already proven correct and fine tuned, properly wrapped in a form suitable to be used within the pipeline skeleton. — ■

Different algorithmic skeleton frameworks chose different approaches to support re-use of sequential portions of (business) code.

¹¹Any change in the business logic code requires proving correctness by the application programmer. Correctness of the skeleton implementation, instead, does not need to be proved by the application programmers.

Some of typical design choices concerning sequential code re-use are related to the possibility to reuse the following kind of code:

- code written in a single sequential programming language vs. code written in a language of a given class of enabled sequential languages.
- side-effect free, function code vs. generic code, including static variables
- source code vs. object code
- source language level wrapping in functions/procedures vs. object code wrapping

Let us analyze pros and cons of these different possibilities:

Single vs. multiple languages. Restricting the possibility to use existing code written in a single sequential programming language allows to implement simpler and possibly faster skeleton programming frameworks, as there is no need to set up particular, multi language procedures. However, when different languages provide object code interoperability, that is when programming language C_i allows to seamlessly call procedures/functions written in language C_j , then considering the possibility to wrap both C_i and C_j code does not require additional efforts.

Functions vs. generic code. In most cases, code with side effects may hardly be safely integrated into algorithmic skeletons. Although nothing prevents in principle the possibility to use side effect code as a pipeline stage or as a map worker, the side effects determine a number of problems. As an example

- you cannot in general move (e.g. to improve the computation performance) a skeleton component (e.g. a map worker or a pipeline stage) from one machine to another one preserving the semantics of the application (side effects could have modified data which has not been comparably modified on the target machine);
- in case of fault of one of the concurrent activities relative to a skeleton, you are not able to restore the computation status, as you don't know which was the exact current status (side effect status is not known).
- you cannot guarantee in general the functional correctness of the implementation of the skeleton framework (you cannot guarantee that two runs with the same input data return the same output results, as different sub computations contributing to the global algorithmic skeleton computation may take place in different runs on different machines).

Algorithmic skeleton frameworks usually *require* that the application programmer guarantees that the sequential portions of code used as skeleton parameters are pure functions.

Source vs. object legacy code. In general the business code that programmers wish to re-use is constituted by some source code possibly calling some external libraries. In case only re-use of source code is allowed, this may exclude the possibility to use well know and effective *libraries*.

Therefore, algorithmic skeleton frameworks usually allow to use legacy libraries in the wrapped business code, provided that this dependency is somehow declared. The declaration of the dependency allows the framework to ensure the presence of the called library in all places where the wrapped code calling the library will be eventually executed, as an example, or, in case of fault, to restore a good “restart point” for the computation on a different processing node.

Source vs. object code wrapping. Wrapping source code is generally easier than wrapping object code. In particular, to wrap source code standard compiler tools can be exploited (e.g. pre-processing, annotations, source-to-source compilers). In case object code is considered for wrapping the “structure” of the code must be known or at least the way input and output parameters are managed must be known.

4.3.2 Full compositionality

Full compositionality in a skeleton framework means that a skeleton¹² may be used as actual parameter for *any* code parameter of *any* skeleton. This property is also known as *flat nesting*, and it has several important implications:

1. Full compositionality implicitly assumes that any skeleton composition actually makes sense, as an example that a map used as a pipeline stage represents a notable and useful parallelism exploitation pattern, as well as a pipeline used as a map worker. As we will discuss in Sec. 4.3.3 some skeleton compositions do not make sense, actually.
2. If we allow any skeleton to appear as a code parameter in another skeleton we assume that the type of the function of the skeleton is the same of the function used as parameter in the second skeleton. This, in turn, raises two different problems:
 - All function parameters must have the same generic type. Then we can use this type as type of *all* the skeletons in the framework. As an example, we can use the generic type $f : \alpha \rightarrow \beta$.
 - We must find a way to conciliate stream parallel skeletons with function parameters. A pipe will have type $pipe : (\alpha \rightarrow \beta) \rightarrow (\beta \rightarrow \gamma) \rightarrow (\alpha \text{ stream}) \rightarrow (\gamma \text{ stream})$. Therefore in order to be able to use a pipe as a stage of another pipe we must conciliate the fact $\alpha \rightarrow \gamma$ is not the same type as $\alpha \text{ stream} \rightarrow \beta \text{ stream}$.
3. The support of full compositionality implies that, given two skeletons Δ_1 and Δ_2 modelling two different kind of parallelism exploitation patterns, the techniques used to implement the two patterns must be *compatible*. In other words we must ensure that the joint exploitation of the two parallelism exploitation patterns in $\Delta_1(\Delta_2)$ (or in $\Delta_2(\Delta_1)$) does not lead to any kind of conflict or inefficiency.

These problems represents an obstacle to the support of full compositionality. In particular, the third point above is the critical one. We don't know at the moment how to implement efficiently compositions of arbitrary parallelism exploitation patterns. As a consequence, limitations are considered related to algorithmic skeleton composition. The more common limitation is to allow only certain skeleton composition, as detailed in the next Section.

4.3.3 Two tier model

During the design of P3L [11] in early '90, the group of the Dept. of Computer Science in Pisa come out with a well defined composition rule for algorithmic skeletons, that was able to solve most of (possibly all) the problems related to algorithmic skeleton composition.

P3L provided a small set of skeletons, including both stream parallel and data parallel skeletons. Each one of the P3L skeletons could use another skeleton in a code parameter. Skeletons could be used to instantiate pipeline stages as well as map, reduce or farm workers. However, the resulting skeleton tree¹³ were *valid if and only if stream parallel skeletons were*

¹²with all its parameters properly instantiated

¹³the tree with skeletons at nodes and parameters in the skeleton node leaves

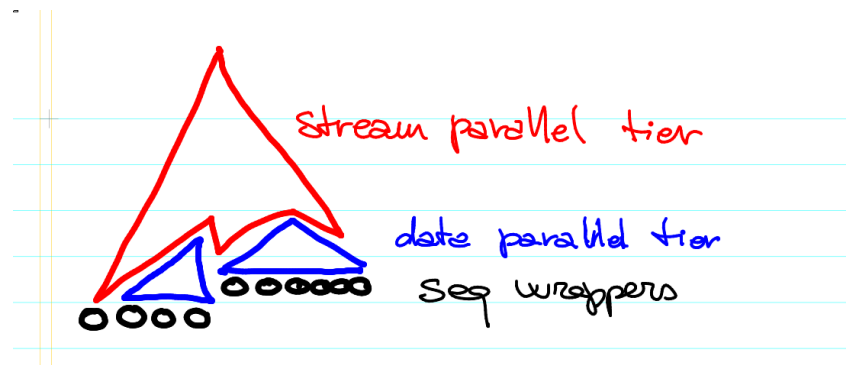


Figure 4.3 Two tier skeleton nesting

used in the upper part of the tree, data parallel ones in the lower part and sequential skeleton (i.e. wrapping of sequential, business logic code) in the leaves.

The upper part of the skeleton tree represents the first stream parallel tier and the lower part (excluding the leaves) the second, data parallel tier (see Fig. 4.3).

This choice was operated observing that:

- data parallelism with internal stream parallel patterns does not offer substantial performance improvements over the same kind of data parallelism with sequential parameters only
- data parallelism may be refined structuring its data parallel computations as further data parallel patterns
- both stream parallel and data parallel patterns eventually should be able to use sequential code parameters.

This model is still in use in several programming frameworks supporting skeleton composition, including Muesli and (in a more restricted perspective) SkeTo.

4.4 STATELESS VS. STATEFULL SKELETONS

Classical skeleton frameworks always considered, designed and provided the final user/programmer with *stateless* skeletons, that is skeletons whose functional behaviour may be completely modelled through higher order *functions*. This is mainly due to the class of parallel architectures available when the first skeleton frameworks were conceived and implemented. At that time, “parallel” architectures was a synonym of “distributed memory” architectures. Clusters, as well as *ad hoc* multiprocessors did not provide any kind of shared memory mechanisms and therefore separate processing elements had to be programmed with programs accessing disjoint rather than shared data structures. This situation has changed, as nowadays a number¹⁴ of multi/many core architectures provide quite efficient shared memory mechanisms.

Moreover, it is known that a number of cases exists where a concept of *shared state* may be useful to model parallel computations. In some cases, it may be useful to maintain in a state variable some counters relative to the current parallel computation. In other cases it is worth allowing concurrent activities to (hopefully not frequently) access some centralized and shared data structure.

¹⁴the vast majority, actually

E.G.▷ A stencil data parallel computation looking for solutions of some differential equation system will iteratively proceed up to the moment the difference between the current and the previously computed solution approximation is smaller than a given ϵ . A parallel application with this behaviour can be programmed as a map reduce, with the map computing the new approximated solution as well as the “local” difference among the current and the old approximate solution and the reduce computing the maximum absolute “local” distance. The computation ends when this value happens to be below the threshold ϵ . A much more natural way of expressing the same computation will be one where the data parallel workers update a globally shared *maxDiff* variable. This requires, of course, that each worker in the data parallel template is able to perform an atomic *update-with-local-value-if-larger-than-the-global-shared-one* operation. ■

E.G.▷ A parallel ray tracer application may be programmed in such a way all the “rays” may be computed in parallel, provided that the scene parameters (shapes, positions, surface feature of the objects, light position, etc.) may be accessed concurrently. A natural solution consists in programming a data parallel skeleton providing a copy of the scene data to all the workers in the data parallel template. A possibly more efficient solution—e.g. in case of large scene data—consists in programming the data parallel workers in such a way they can access—*read-only*—the scene data. ■

Managing and scheduling complex access patterns to a shared data structure represents a complicate and error prone activity, requiring proper planning of shared data access scheduling and synchronization. It also represents one of the classical source of bottlenecks in existing parallel applications: when shared data structures must be accessed concurrently, the concurrent activities rarely may achieve ideal speedup. The accesses to the shared data structure, in fact, make up the serial fraction that limits the speedup achieved in the parallel application (see Amdahl law in Sec. A). Therefore, shared data access has always been prohibited in structured parallel programming models based on the algorithmic skeleton concept (remember that an algorithmic skeleton has to be a well known, *efficient* parallelism exploitation pattern, according to the skeleton assessed definition).

With the growing and ubiquitous presence of shared memory multi/many core architectures, however, the problem of being able to manage some kind of shared global state is becoming again more and more interesting.

The structure of the parallel computation exposed through the skeleton composition used to model parallel applications may be used to improve the efficiency in dealing with shared state within parallel computation.

We illustrate in the following sections some principles and techniques useful to provide such shared state support in skeleton based frameworks.

4.4.1 Shared state: structuring accesses

We start distinguishing the different kind of accesses performed on shared state data structures. The particular features of each kind of access will then be used to determine the better techniques supporting the implementation of the shared state in the skeleton framework. The kind of accesses we distinguish include:

Readonly This is not actually a shared state, as no modification is made to the shared structure. The only concurrent accesses are those aimed at reading (parts of) the shared structure.

Owner writes This is another particular and quite common kind of shared state access: a number of concurrent entities access the data structure for reading but only one of

the concurrent entity is allowed to modify a single sub-partition of the shared state structure.

Accumulate In this case, the shared state data is accessed for writing by all (more than one of) the concurrent entities accessing the state, but the only kind of write operation is such that each write turns out to be a

$$x_{new} = x_{old} \oplus y$$

where $\oplus$ is a commutative and associative operator and y is a parameter provided by the concurrent entity requiring the update of the shared state variable.

Resource This is the kind of shared state access that mimics an exclusive access to a shared resource. It corresponds in obtaining a *lock* on the shared state variable, then modifying the variable and eventually releasing the lock.

These kind of accesses allow different strategies to be implemented to support shared state implementation. These strategies vary depending on the kind of architectural support available for shared memory accesses. In particular:

- *Readonly* state may be handled by copying. In case the data structure is large and shared memory is supported, accesses to the shared data structure do not need any kind of synchronization nor cache coherence needs to be ensured on the cache copies of the shared data structure (parts).
- *Owner writes* state variables may be handled in such a way the “owner” concurrent entity has allocated in its private memory¹⁵ the “owned” state variables and other concurrent activities must access these values accessing the “owner” memory. This means implementing remote accesses in case of distributed architectures or implementing some kind of cache coherence algorithm in case of shared memory target architectures.
- *Accumulate* shared state may be implemented by encapsulating the accumulated state in a concurrent activity that receives asynchronous requests *accumulate(y)* by the computation concurrent activities and applies the accumulate operation on a local copy of the state. Read request relative to the accumulated value may be implemented in the same way, may be causing a *flush* of all the write requests pending before actually reading the value to be returned back to the requesting concurrent activity. In case of target architectures providing support for shared memory, this kind of state may be implemented in a more distributed way, profiting of cache coherence mechanisms. The value of the accumulate state will therefore migrate to the places where the accumulate write requests will be generated.
- *Resource* state variables should be handled with full synchronization in order to guarantee the exclusive access to the state values.

The consequences of the usage of state variables within structured parallel computations depend on the kind of shared state and, with a relatively smaller weight, on the kind of parallel pattern used. Let us assume the kind of pattern used only impacts the number of concurrent activities accessing the shared state¹⁶. Then:

¹⁵we refer to memory as *main memory* or *caches* in this case, depending on the kind of target architecture

¹⁶this is a quite simplistic assumption, but relatively to the classification of shared states used in this Section, it makes sense

- Accesses to *readonly* shared state does not impact parallelism exploitation. A little overhead may incur in the access to readonly state variables with respect to the overhead needed to access regular variables¹⁷.
- Accesses to *owner writes* shared state will behave like accesses to *readonly* state. No parallel activities are serialized due to state access as only the owner concurrent activity may write (its part of) the state variable. A possible, small overhead may be measured when accessing-reading-shared variables if at that time the same (part of the) state variable is being written by its owner concurrent activity.
- Accesses to *accumulate* shared state will be implemented mostly asynchronously. Again, these accesses will not impair the potential parallelism: no logically concurrent activities will be serialized *unless* specific “flush/barrier” operations are provided to synchronize the accesses.
- Accesses to *resource* state variables may completely serialize potentially parallel/concurrent activities, due to the full synchronization ensured in the shared state accesses.

E.G.▷ Consider a farm, and assume we want to count the number of tasks computed by the string of workers in the farm. We may use an *accumulate* integer shared variable. The accumulator operator $\oplus$ is the actual addition operator, which is commutative and associative. Each farm worker will issue a *accumulate(1)* request after completing the current task computation. The shared state variable will only be read at the end of the computation, possibly by the farm collector or by some other concurrent activity taking place after farm termination. No serialization is imposed on worker activity due to access to this simple shared state.

■

E.G.▷ Consider a pipeline and assume each stage uses a *resource* shared state to compute the stage function. We have to distinguish two cases:

1. the access to the shared state represents a small fraction of the stage computation, or
2. it represents a consistent fraction of the computation.

In the former case, each stage access to the shared state is serialized. As a consequence, only the remaining fraction of stage computation may be actually performed in parallel. Therefore, pipe parallelism is reduced according to the Amdhal law (see Sec. A).

In the latter case, the computation of the stages of the pipeline are *de facto* serialized. The whole pipeline behaves therefore as a sequential stage with latency equal to the sum of the latency of the stages (see Chap. A).

■

¹⁷non state ones

CHAPTER 5

PARALLEL DESIGN PATTERNS

Design patterns have been introduced in the '90 to “describe simple and elegant solutions to specific problems in object-oriented software design. Design patterns capture solutions that have developed and evolved over time. Hence, they aren’t the designs people tend to generate initially. They reflect untold redesign and recoding as developers have struggled for greater reuse and flexibility in their software. Design patterns capture these solutions in a succinct and easily applied form” [26].

Parallel design patterns [43] have been introduced in early '00, with the intent to model solutions to recurring *parallel* problems. They represent a software engineering perspective on the parallel computing problems. The way they are presented—grouped in layers, with each layer including patterns of interest when looking at the parallel programming problems from a particular abstraction level—represents a kind of methodology suitable to support the parallel application developer in the design of efficient applications.

From our structured parallel programming course perspective, parallel design patterns represent a kind of “engineered way” of considering the problems related to parallelism exploitation. This is the reason why we introduce and shortly discuss parallel design patterns here.

5.1 DESIGN PATTERNS

A design pattern is a representation of a common programming problem along with a tested, efficient solution for that problem [26]. Design patterns are usually presented in an object oriented programming framework, although the idea is completely general and it can be applied to different programming models.

A design pattern includes different elements. The more important are: a *name*, denoting the pattern, a *description of the problem* that pattern aims to solve, and a *description of the solution*. Other items such as the *typical usage* of the pattern, the *consequences* achieved by

Element	Description
Name & classification	The name of the pattern and the class the pattern belongs to
Intent	The problem solved by the pattern
Also known as	The other names/terms used to describe the pattern
Motivation (forces)	The reason(s) why the pattern is introduced
Applicability	The typical situations where the pattern may be exploited
Structure	The structure of the pattern in terms of the component classes/class diagrams
Participants	The classes involved
Collaboration	The interactions among the involved classes
Consequences	The kind of consequences deriving from pattern application
Implementation	Known implementation
Sample code	Sample code or pseudo code relative to the implementation and usage of the patterns
Known uses	Examples of applications/algorithms/kernels using the pattern
Related patterns	Other patterns that may be of interest for or used with the current pattern

Table 5.1 Elements of a design pattern

using the pattern, and the *motivations* for the pattern are also included. Table 5.1 details the elements included in the description of a design pattern in [26].

Therefore a design pattern, *per se*, is not a programming construct, as it happens in case of the algorithmic skeletons. Rather, design patterns can be viewed as “recipes” that can be used to achieve efficient solutions to common programming problems.

Typical design patterns include patterns commonly used in wide range of object-oriented applications, such as

- the *Proxy* pattern, providing local access to remote services (Fig. 5.1 shortly outlines the proxy pattern as described in [26]),
- the *Iterator* pattern, providing (sequential) access to the items of a collection,

as well as more sophisticated patterns, such as

- *Mediator* pattern, encapsulating the interactions happening between a set of *colleague* objects,
- the *Strategy* pattern, providing encapsulation of a set of similar services along with methods to transparently pick up the best service provider,
- the *Observer* (or *Publish/Subscribe*) pattern, used to structure asynchronous, event based applications,
- the *Facade* pattern, providing a simplified interface to some kind of large code/class ensemble.

Overall, the programmer using design patterns to structure the sequential skeleton of his/her application will produce more robust and maintainable code than the programmer using a more traditional–non design pattern based–object oriented approach.

In the original work discussed in Gamma’s book [26], a relatively small number of patterns is discussed. The debate is open on the effectiveness of adding more and more specialized patterns to the design pattern set [1]. This situation closely resembles the one found in the skeleton community some years ago. Skeleton designers recognized that a small set of skeletons may be used to model a wide range of parallel applications [10, 22] but some

Intent	provide a surrogate placeholder for another object to control access to it
Also known as	surrogate
Motivation	defer cost of creation, initialization or access
Applicability	remote proxy, virtual proxy, protection proxy, smart reference
Structure	class Proxy calls the class RealSubject when invoked from an instance of Subject—the RealSubject—issuing a Request()
Participants	Proxy (maintains a reference to the object, provides an interface to access the object, controls the accesses to the object), Subject (defines common interdice for RealSubject and Proxy), RealSubject (defines the real object represented by the Proxy)
Collaboration	propagate requests to RealSubject
Consequences	introduces indirection in accesses to the object
Implementation	e.g. by overloading the member access operation in C++ (->)
Sample code	sample code in C++ or Java presented here ...
Known uses	accesses to remote objects (RMI/RPC), implementation of distributed objects
Related patterns	Adapter (providing a different interface to an object), Decorator (adding responsibilities to an object).

Figure 5.1 The Proxy pattern

authors claimed that specialized skeletons should be used to model application dependent parallelism exploitation patterns [34].

Recently, some questions concerning design patterns have been posed, that still have no definite answer: should design patterns become programming languages? Should they lead to the implementation of programming tools that provide the programmer with more efficient and “fast” OO programming environments?[15, 39]

Again, this is close to what happened in the skeleton community. After Cole’s work it was not clear whether skeleton should have been made available as some kind of language constructs or they should have been used only to enforce a well founded parallel programming methodology.

5.2 PARALLEL DESIGN PATTERNS

Parallel design patterns have been introduced [43] with the same aims of (sequential) design patterns, namely to describe solutions to recurrent problems in the context of interest. The context of interest in this case is obviously parallel software design rather than simple object-oriented software design.

As it happened in the case of classical design patterns, a parallel design pattern is described by means of a precise set of elements, namely:

- *Problem* – the problem to be solved.
- *Context* – the context where the pattern may be most suitably applied.
- *Forces* – the different features influencing the parallel pattern design.
- *Solution* – a description of one or more possible solutions to the problem solved by the pattern.

Sample parallel design patterns, as an example, solve typical parallelism exploitation problems, such as *divide&conquer*, *pipeline* and *master/worker* solving respectively the problem of efficient implementation of recursive, divide and conquer computations, of staged computations and of computations split in a number of independent tasks.

Parallel design patterns have been used to implement structured parallel programming frameworks very similar to the algorithmic skeleton ones [40, 29, 45] and a notable research activity has been generated that eventually led to the recognition of the fact that parallel design patterns may be a key point in the solution of the problems related to the development of efficient parallel software. The Berkeley report [9] published in late '00s clearly states that

Industry needs help from the research community to succeed in its recent dramatic shift to parallel computing.

and lists among the other principles to be followed to efficiently address the problem of designing and developing efficient software for parallel machines the following principle:

Architecting parallel software with design patterns, not just parallel programming languages.

In particular, the authors recognize that

Computer scientists are trained to think in well-defined formalisms. Pattern languages encourage a less formal, more associative way of thinking about a problem. A pattern language does not impose a rigid methodology; rather, it fosters creative problem solving by providing a common vocabulary to capture the problems encountered during design and identify potential solutions from among families of proven designs.

A comprehensive list of parallel design pattern papers and projects may be found at the web page <http://www.cs.uiuc.edu/homes/snir/PPP/>. The rest of this Section summarizes the parallel design patterns as presented in [43].

5.2.1 The parallel design pattern design spaces

One of the most interesting aspects related to parallel design patterns as described in [43] is the partitioning of the design space of a parallel application in four separate but related *design spaces*:

- The **Finding concurrency** design space, including the parallel design patterns modelling the different kinds of parallel/concurrent activities structures that may be suitably used in a parallel application.
- The **Algorithm structure** design space, including the parallel design patterns modelling recurring parallel algorithm patterns.
- The **Supporting structure** design space, including the parallel design pattern that can be used to implement the algorithmic patterns in the previous design space.
- The **Implementation mechanisms** design space, eventually including the patterns modelling the base mechanisms used to implement a parallel application.

From our algorithmic skeleton perspective, the first two design spaces correspond to what we called “concurrent activity graph” definition/design, while the last two correspond to the “implementation” of the concurrent activity graph. The importance of this design space structuring relies in the “separation of concerns” implicitly assumed in between the programmers using the parallel patterns: application programmers—the ones supposedly using the patterns in the finding concurrency and algorithm structure design spaces—will only use their (application) domain specific knowledge to find out the most proper combination of high level patterns modelling parallelism exploitation in the application at hand. And this process will not require any knowledge relative to the underlying target architecture. Then the system programmers—the ones that are experts in the target architecture features—may exploit the design patterns in the supporting structure and implementation mechanisms

spaces to properly implement the patterns used at the higher level by the application programmers. This reflects the situation in the algorithmic skeleton world where application users basically use pre-defined skeletons that the system programmers are in charge to implement efficiently onto the target architecture.

In the following sections, we will discuss in detail the different patterns in the different design spaces, pointing out, if the case, similarities and differences with similar concepts/abstractions in the algorithmic skeleton universe. Fig. 5.2 shows the global design space, outlining the different patterns and pattern groups in the four design spaces.

5.2.2 Finding concurrency design space

The patterns in the “finding concurrency design space” may be used to start designing a parallel application, right after having considered the problem to be solved with the application and identified the most computationally intensive parts to be parallelized.

There are three different groups of patterns in this space. Two of them are actually related to the ways used to implement the parallel application (the decomposition and dependency analysis patterns) and one related to the fact the “finding concurrency” space is such that a solution is often found iterating with different steps through the decomposition and dependency analysis patterns (the design evaluation pattern).

In particular, the **Decomposition patterns** are used to decompose the problem into pieces that can execute concurrently, while the **Dependency analysis patterns** help grouping the tasks to be executed and analyzing the dependencies among these tasks. The **Design evaluation pattern** is only used to “guide the algorithm designer through an analysis of what has been done so far before moving to the patterns in the algorithm structure design space” [43].

The decomposition patterns include just two patterns:

- the **Task decomposition pattern**, modelling the division of a complex application in a set of tasks that can be executed concurrently, and
- the **Data decomposition pattern**, modelling the division of the input data into subsets that can be (relatively) independently computed to obtain the final result.

The main forces influencing the design of these patterns are *flexibility*, *efficiency* and *simplicity*. Flexibility is needed to adapt the application design to different implementation requirements. Efficiency is needed as we are targeting efficient parallel application development. Last but not least, simplicity is required to help the application programmers to use to pattern in their parallel applications: too complicated patterns (or libraries, objects or APIs) have no sufficient appeal to convince the application programmers they are worth being used and exploited.

The dependency analysis patterns include three different patterns, instead:

- the **Group tasks** pattern aimed at modelling the more convenient grouping of tasks such that the management of dependencies is simplified,
- the **Order tasks** pattern aimed at figuring out how tasks (or groups of tasks) may be ordered to satisfy the application constraints related to task execution, and
- the **Data sharing** pattern aims at modelling the accesses to a shared data structure.

E.G.▷ Let’s have a look in more detail to one of these patterns, which are not as obvious as the ones in the decomposition patterns set: the order task pattern. The order task pattern considers groups of tasks, as output by the group tasks pattern and tries to find the dependencies among those groups of tasks in order to determine an order of execution of the groups of

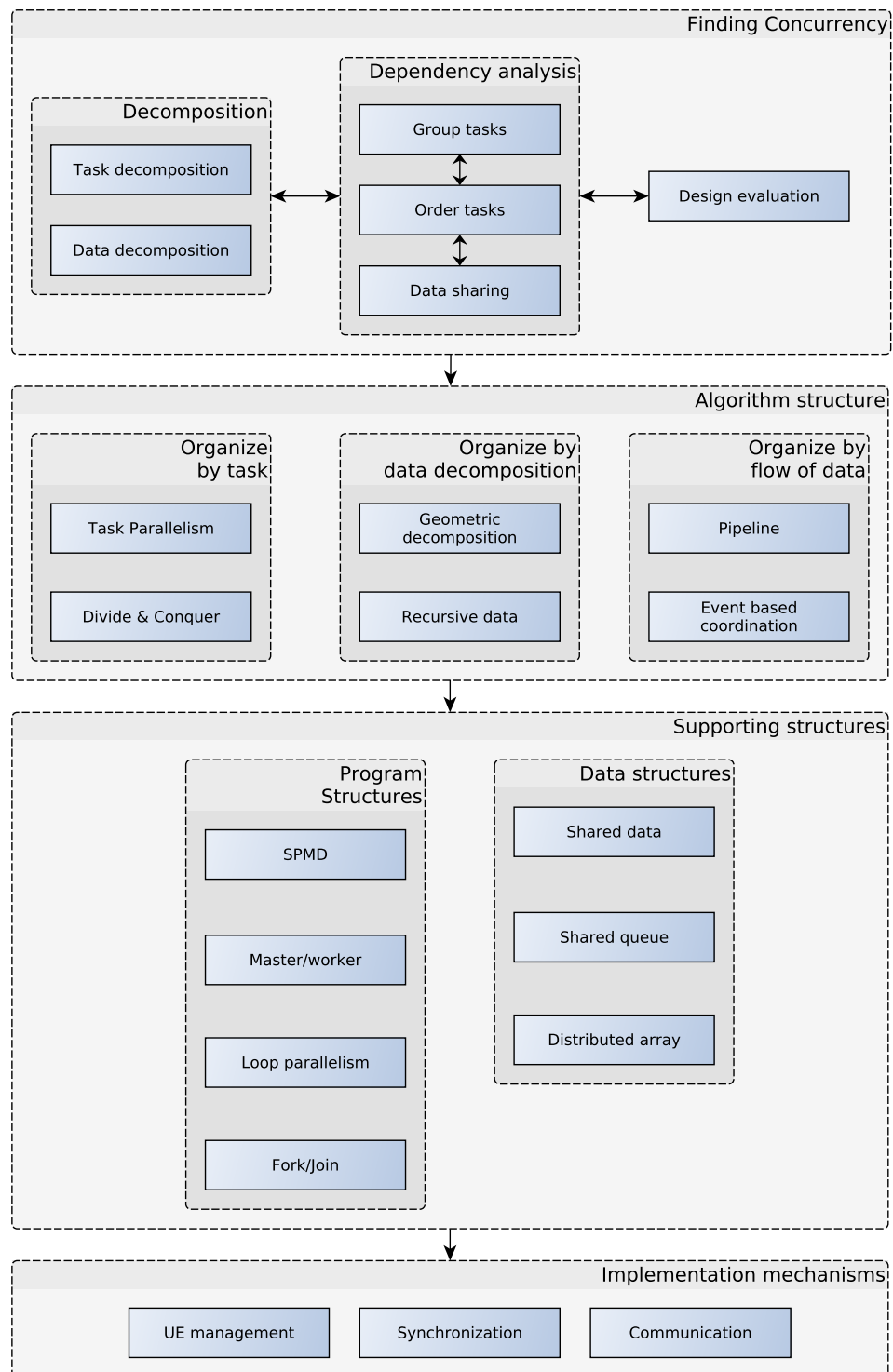


Figure 5.2 Design spaces & Parallel design patterns

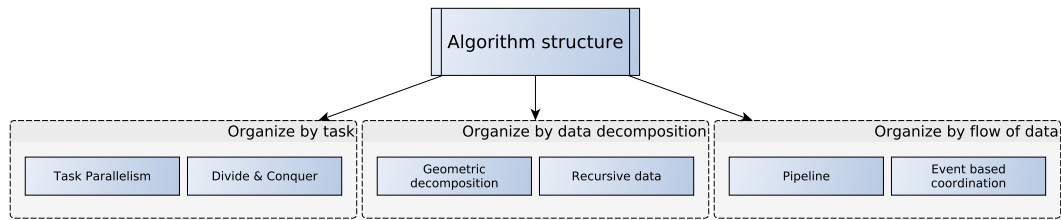


Figure 5.3 Algorithm structure design space: alternative pattern groups

tasks. In particular, temporal and data dependencies are considered. If neither temporal nor data dependencies are present, total independence is assumed, which is definitely a good feature when parallel execution of the tasks is taken into account. The order design pattern looks for a sufficient and necessary ordering, that is for an ordering that ensures that all the constraints are actually taken into account and satisfied and no unnecessary constraints are enforced. The solution proposed in [43] is to first consider data dependencies and then to consider the influence of external services (e.g. accesses to disk) that may impose further constraints with the possibility of unleashing the concurrent execution of all groups in case no data dependencies exist and no constraints come from external services. — ■

The overall output resulting from the analysis of the “finding concurrency” design space is a decomposition of the problem into different design elements, namely i) a task decomposition identifying the tasks that can be executed concurrently, ii) a data decomposition that identifies the data local to each one of the tasks, iii) a way of grouping tasks and ordering the tasks groups in such a way temporal and data dependencies are satisfied and iv) an analysis of the dependencies among the tasks.

5.2.3 Algorithm structure design space

The output from the “finding concurrency” design space is used in the “algorithm structure design space” to refine the design of our parallel application and to figure out a parallel program structure closer to an actual parallel program suitable to be run on a parallel target architecture.

There are three major ways of organizing a parallel algorithm:

- **by task**, that is considering the tasks as the base thing to be computed in parallel within the parallel algorithm,
- **by data decomposition**, that is considering the decomposition of data into (possibly disjoint) subsets as the base items to be processed in parallel within the parallel algorithm, or
- **by flow of data**, that is considering the algorithm defined flow of data as the rule dictating what are the steps to be computed concurrently/in parallel within the parallel algorithm.

The three ways of organizing the parallel algorithm are somehow alternatives. While in the “finding concurrency” design space the application programmer is supposed to go through all the groups of patterns, here the programmer is required to choose one of the three alternatives and exploit one of the parallel design patterns in the group (see Fig. 5.3).

The **Organize by tasks** pattern group includes two patterns:

- the **Task parallelism** pattern, managing the efficient execution of collections of tasks. The proposed solution to implement the pattern works out three different points: how

tasks are defined, the dependencies among tasks and the scheduling of the tasks for concurrent execution.

- the **Divide&conquer** pattern, implementing the well know divide and conquer recursive solution schema.

The **Organize by data decomposition** pattern group includes two patterns:

- the **Geometric decomposition** pattern modelling all those computations where a given “geometric shape” of the concurrent executors may be recognized (linear vector, bi-dimensional array), and the parallel algorithm may be defined by defining the kind of computation(s) being performed at the shape items (vector items, array rows, array columns, array items, ...).
- the **Recursive data** pattern, modelling those parallel computations structured after the structure of some recursively defined data structure, e.g. a list—defined as a null list or an item followed by a list—or a tree—defined as a node or a node with a (given) number of “son” trees.

Eventually, the **Organize by flow of data** pattern group also hosts two patterns:

- the **Pipeline** pattern, where the flow of data is traversing a linear chain of stages, each representing a function computed on the input data—coming from the previous stage—whose result is delivered to the next stage.
- the **Event-based coordination** where a number of semi-independent concurrent activities interact in an irregular way and interactions are determined by the flow of data among the concurrent activities.

E.G.▷ In order to fix the concepts, let’s analyze more in detail the “recursive data” pattern. The **problem** solved by the pattern is the implementation of (efficient and parallel) operations over recursively defined data structures. The **context** includes all those situations where the data contained in a recursively defined data structure (e.g. tree or list) are processed as a whole or in pieces. Tree structured data, as an example, may be processed to obtain a new data tree with all the nodes/leaves substituted by the result of a function f applied to the old nodes/leaves. Or we may process the tree structured data to search for a particular node, traversing the tree with a pre-fix algorithm. A number of problems have to be taken into account when describing this pattern. The most notable include data representation—usually pointers are used, which are neither too much portable nor practical to use for parallel processing of all the items in the data structure, as following pointers is usually a fairly sequential process, as an example—and dimensioning of the concurrent activities used to implement recursive data processing—e.g. the dept of the tree may be unknown and not computable up to the end of the computation. The **forces** driving the pattern design are different. First, we have to take into account the data representation problem. We need to define a proper representation, suitable to be used for parallel processing, may be radically different from the representation used for the recursive data structure in sequential programming. As an example a list may be represented with a vector rather than with the usual linked list data structure, to ensure the possibility to perform concurrent item accesses, even if insertion and removal of elements may require more complex implementations. Second, we must ensure the data representation is not only functional to the parallel processing needs, but it should be also easy to understand and maintain. Last but not least, a tradeoff should be looked for in between the amount of concurrency exploited in the recursive data structure processing and the amount of overhead—concurrent activity setup, communication, synchronization—introduced. The **solutions** adopted within the pattern include structuring the pattern computation in phases, namely:

- *data decomposition*: the data is restructured and decomposed as most suitable for being processed by the pattern

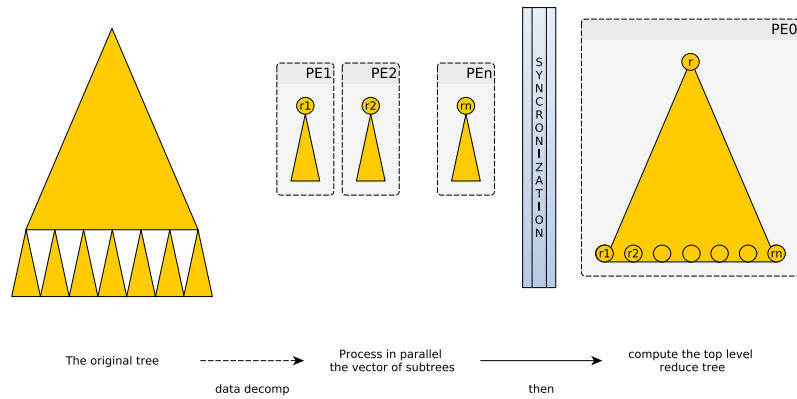


Figure 5.4 Recursive data pattern: sample usage (reduce of a tree)

- *structure*: figure out some kind of “collective”-and parallel-operation to be applied on the data decomposition results, possibly in a loop, to compute the expected results
- *synchronization*: figure out the kind of synchronization needed to implement the (loop of) collective operations leading to the final result.

As an example, consider the problem of computing a “reduce” with an operator $\oplus$ over a binary tree. We can decompose the tree into the sub trees rooted at level i , then apply sequentially the “reduce” $\oplus$ operation onto all the sub trees in parallel and eventually sequentially compute the “reduce” $\oplus$ operation on the partial results computed at the different sub trees. In this case, the data “data decomposition” phase consist in obtaining the sub tree vector out of the original tree, the “structure” phase consist in applying-in parallel-the reduce over the sub trees and the “synchronization” phase involves waiting the termination of the computation of the sub tree reduces before actually starting computing the root reduce (see Fig. 5.4). ■

5.2.4 Supporting structure design space

After having explored different possibilities to find concurrency and to express parallel algorithms in the “finding concurrency” and “algorithm structure” design spaces, implementation is taken into account with two more design spaces. The first one is called **Supporting structures** design space. This space starts investigating those structures-patterns-suitable to support the implementation of the algorithms planned when exploring the “algorithm structure” design space. Two groups of patterns are included in the design space. The first one is related to program structuring approaches-this is the Program structures pattern group-while the second one is related to the commonly used shared data structures-this is the **Data structures** pattern group. The **Program structures** group includes four patterns:

- the **SPMD** pattern, modelling the computation where a single program code is run on different input data,
- the **Master/Worker** pattern, modelling concurrent execution of a *bag of tasks* on a collection of identical workers,
- the **Loop parallelism** pattern, modelling the concurrent execution of distinct loop iterations, and

	<i>Task Parallelism</i>	<i>Divide& Conquer</i>	<i>Geometric Decomp.</i>	<i>Recursive Data</i>	<i>Pipeline</i>	<i>Event-based Coordin.</i>
SPMD	★ ★ ★ ★	★ ★ ★	★ ★ ★ ★	★ ★	★ ★ ★	★ ★
Loop Par.	★ ★ ★ ★	★ ★	★ ★ ★			
Master/worker	★ ★ ★ ★	★ ★	★	★	★	★
Fork/join	★ ★	★ ★ ★ ★	★ ★		★ ★ ★ ★	★ ★ ★ ★

Figure 5.5 Relationships between the *Supporting Structures* patterns and *Algorithm Structure* patterns (from [43]).

- the **Fork/Join** pattern, modelling the concurrent execution of different portions of the overall computation that proceed unrelated up to the (possibly coordinated) collective termination.

These patterns are well known in the parallel computing community¹⁸. The SPMD pattern is the computational model used by MPI and one of the most popular patterns used to structure parallel computations with the master/worker. Loop parallelism has been exploited in vector architectures, and it is currently one of the main sources of parallelism in both OpenMP and GPUs. Last but not least, the fork/join pattern perfectly models the `pthread_create/pthread_join` model of POSIX threads.

The **Data structures** group includes three patterns:

- the **Shared data** pattern models all those aspects related to the management of data shared among a number of different concurrent activities,
- the **Shared queue** pattern models queue data types implemented in such a way the queues may be accessed concurrently, and
- the **Distributed array** pattern, models all the aspects related to the management of arrays partitioned and distributed among different concurrent activities.

Also these patterns are well known in the parallel computing community. Shared data is managed in a number of different parallel applications and the correct and efficient management of the shared data is usually the more time and effort consuming activity in the whole parallel application development/design process. Shared queues are used to support interaction of concurrent activities in different contexts, from threads to processes and concurrent activities running on CPU co-processors. Eventually, distributed arrays are often used to implement data structures that are logically shared among concurrent activities but may be somehow partitioned in such a way single one of the concurrent activities “owns and manages” a single portion of the distributed array.

The authors of [43] classify the different patterns in this design space—in fact called “supporting structures” design space—with respect to their suitability to support the implementation of the different patterns in the “algorithm structure” design space.

As an example, Task parallelism is well supported by the four patterns in the Program structures group, whereas the Recursive data pattern is only (partially) supported by the SPMD and Master/Worker pattern (see Tab. 5.5, taken from the parallel patterns book).

In fact, by considering the “supporting structures” design space as the implementation layer of the “algorithm structure” design space, we *de facto* make a strong analogy with what happened in the algorithmic skeletons frameworks, where skeletons represent the “algorithm structure” patterns and the implementation templates represent the “supporting

¹⁸the community of researchers working in parallel computing but not necessarily experts in software engineering

Design space	#pag
Finding concurrency	21
Algorithm structure	57
Supporting structures	104
Implementation mechanisms	35

Figure 5.6 Space dedicated to the description of the four design spaces in [43]

```

int main(int argc, char * argv[]) {
    ...
    MPI_Init(&argc,&argv);

    my_id = MPI_Comm_rank(MPI_COMM_WORLD);

    switch (my_id) {
        case 0: { ... }
        case 1: { ... }
        ...
        default: { ... }
    }

    MPI_Finalize();
}

```

Figure 5.7 Sketch of sample MPI code

structures” patterns. It is worth pointing that the clear distinction between the “task parallelism” pattern (corresponding to the task farm skeleton) and the “master/worker” pattern (corresponding to one of the implementation templates used to implement task farms, maps and other kind of skeletons) has no correspondence in the algorithm skeleton framework where often the “supporting structures” master/worker pattern is claimed to be a skeleton, that is an “algorithm structure” pattern.

We want to point out here another interesting fact. Table 5.6 reports the pages used to describe the patterns in the four design spaces in [43]. The high level patterns and the low level ones are described using more or less the same space used to describe the patterns in the “supporting structure” design space. This is a clear sign that “there is more to say” on the supporting structures, as in parallel programming this is the only design space explored in depth along with the “implementation mechanisms” space. However, this latter space is a kind of “RISC” layer only providing very basic patterns used to build the full range of “supporting structures” patterns and therefore it may be described much more concisely.

E.G.▷ Consider the “SPMD” pattern. The **problem** solved with the pattern is the correct and efficient implementation of parallel computations where programmer provide a single program for all the concurrent activities. The single program may differentiate the instructions to execute depending on the concurrent activity index. The **context** where this pattern is used includes the large majority of the massively parallel applications, in particular all those developed using MPI. These applications are usually implemented with a single program such as the one outlined in Fig. 5.7. The program is executed on all the target machine processing elements. Each instance of the program receives a different integer id and the code executed at the different PEs is therefore differentiation with the switch statement. The **forces** driving the pattern design are the usual ones of most of the “supporting structures”

patterns, namely: portability, to ensure pattern reusability, scalability, to support different parallelism degrees according to the target architecture features, and efficiency, to ensure performance. A viable **solution** to SPMD pattern design consists in structuring the pattern with the phases outlined in the MPI code sketch above:

- *Initialize*: setting up all the necessary infrastructure for the pattern execution
- *Unique identifier*: getting the id necessary to differentiate the code within the unique program
- *Run the program on all the different processing elements*: run the program in such a way the all the concurrent program activities are completed
- *Finalize*: clean up everything and carefully terminate all the activities.

■

5.2.5 Implementation mechanisms design space

The second (and lower level) design space related to implementation of parallel applications is called **Implementation mechanisms** design space. This design space includes the patterns modelling the base mechanisms needed to support the parallel computing abstractions typical of parallel programming, that is:

- *concurrent activities*
- *synchronizations*
- *communications*.

In fact, the design pattern hosts only three distinct patterns:

- the **UE Management** pattern, related to the management of the **Units of Execution** (processes, threads)
- the **Synchronization** pattern, handling all those aspects related to ordering of events/-computations in the UE used to execute the parallel application
- the **Communication** pattern, handling all the aspects related to the communications happening in between the different UE implementing the parallel application.

The “UE management” pattern deals with all the aspects related to the management of the concurrent activities in a parallel application. Therefore creation, run and termination of concurrent activities are taken into account. Although in [43] only threads and processes are taken into account, the “UE management” pattern may be obviously adapted to handle the concurrent activities placed on CPU co-processors, e.g. the GPU kernels. The “synchronization” pattern deals with all aspects related to synchronization of concurrent activities and therefore covers aspects such as lock/fence¹⁹ mechanisms, higher level mutual exclusion constructs (e.g. monitors) and collective synchronizations (e.g. barriers). Last but not least, the “communication” pattern deals with the aspects related to data exchange among concurrent activities and therefore covers aspects related to different kinds of point-to-point (e.g. send, receive, synchronous and asynchronous) and collective communications (e.g. broadcast, scatter, gather, reduce).

¹⁹mechanisms ensuring that at a given point the *view* of the shared memory items is consistent across all EU involved in a parallel application

5.3 SAMPLE PARALLEL DESIGN PATTERN USAGE: IMAGE PROCESSING APPLICATION DESIGN

We illustrate the kind of design methodology related to parallel design patterns summarizing different steps in the design of a parallel applications. We chose an application whose goal is to process—in parallel—a stream of images in such a way i) a “truecolor” filter is applied to each image—the colors are normalized—and ii) the image is sharpened—an edge enhance filter is applied.

5.3.1 Application analysis

The first step is not related to the design patterns, but rather is meant to identify the actual needs of the application in terms of parallel computing. As we want to process a (possibly long) stream of images and we know the images are high resolution images, we can simply evince that the computational load will be huge due to two distinct facts: i) the large number of images being processed, and ii) the large number of pixels relative to each one of the images. Therefore our aim will be to parallelize as much as possible both the processing of different images in the stream (which are independent of each other) and of different parts of the same image (if possible).

5.3.2 Exploring the “finding concurrency” design space

We first consider the patterns in “decomposition” pattern group. As different images appearing onto the input stream are independent, we can use the “task decomposition” pattern to model parallel execution of our application over the different images of the stream.

When moving considering the problem of parallel computation of the single image, we may consider to use the “data decomposition” pattern to implement both filters, the color normalization and the edge sharpening ones.

We then consider the “dependency analysis” pattern group. Here we may use the “order task” pattern to ensure that the processed images are output in the same order the corresponding unprocessed images appeared onto the input stream. We can also consider using the “data sharing” pattern to ensure access to shared data or portions of the images when we implement the two filters. As an example, we should consider the possibility to compute some general—i.e. referred to the whole image—color parameters and then to use this parameters to normalize independently different sub-portions of the image in the color normalization filter.

The “design evaluation” pattern may be used to evaluate the concurrency found in this step, before actually moving to explore the “algorithm structure” design space. Within the pattern, it is suggested to consider three different perspectives (forces) in order to come to a good “concurrent decomposition” of our application:

Suitability for target platform Although we still move at a very high level of abstraction, some general features of the target architecture should already be taken into account: i) number of processing elements (CPU/core, co-processor cores), ii) mechanisms supporting data sharing (shared memory, distributed shared memory, message passing) and iii) overhead related to the set up of concurrent activities. In particular, we must ensure

- we do not plan to use a number of concurrent activities larger than the number of available processing elements. In our case this means we should not plan to use one processing element per input image, as an example. In fact, by recognizing (task order pattern) that the edge filter is to be applied after the color one, we can plan to use a couple of processing elements to implement the processing of

a single image, and then we may assume to use more of this processing element pairs to implement in parallel the processing relative to different images of the input stream, up to the point we use all the available processing elements or we succeed processing all the available images before new ones are available onto the input stream.

- that the planned shared data structures may be actually reasonably implemented in terms of the mechanisms available to support sharing. In our case this means that we look at the target architecture features and in case it does not support primitive sharing mechanisms (e.g. shared memory or distributed shared memory) we rely only on communications to implement our shared data structures, e.g. assuming they are encapsulated in a “sharing server” answering all the read/write requests to the shared data structures.
- and that the overhead necessary to implement all the planned concurrent activities does not exceed the benefits coming from the parallel execution of our application. In our case this means we should be careful not setting up a concurrent activity when the time spent in computing the results of the concurrent activity sequentially is comparable to the time used to set up the concurrent activity itself.

Design quality In this case we need to ensure our concurrent activity plan ensures *flexibility*, *efficiency* and *simplicity*. In our case, flexibility is ensured by the possibility to vary different parameters, such as the parallelism degree in the “task decomposition” and “data decomposition” pattern, as well as in the possibility to collapse either level, that is to apply only the “task decomposition” pattern (therefore to exploit only stream parallelism) or to apply only the “data decomposition” pattern (that is to apply only data parallelism). Simplicity comes from the clean design ensured by the proper usage of the two patterns mentioned above. Efficiency comes from the considerations relative to the “suitability for target platform” and may only be stated in principle: it is well known that a good design may be completely impaired when the wrong implementation mechanisms are chosen to implement it, and therefore the actual efficiency will be clear only after having explored the whole set of design spaces.

Preparation for the next phase We must ensure that our concurrent activity plan is suitable to be worked out by the patterns in the “algorithm structure” design space. In our case, the regular decomposition “task+data” figured out should ensure the “algorithm structure” space may be efficiently explored.

Fig. 5.8 illustrates the results of the exploration of the “finding concurrency” design space.

5.3.3 Exploring the “algorithm structure” design space

Moving to the algorithm structure design space, we decide the abstract algorithms to be used to implement the concurrent activities identified in the previous step. As an example:

- we decide to use a “task parallelism” pattern from the “organize by task” group to implement the processing of different images of the input stream
- we decide to use a “geometric decomposition” pattern from the “organize by data decomposition” group to implement the processing of the single filter on the single image
- we decide to use a “pipeline” pattern from the “organize by flow of data” group to implement the sequence of the two filters applied onto one input image.

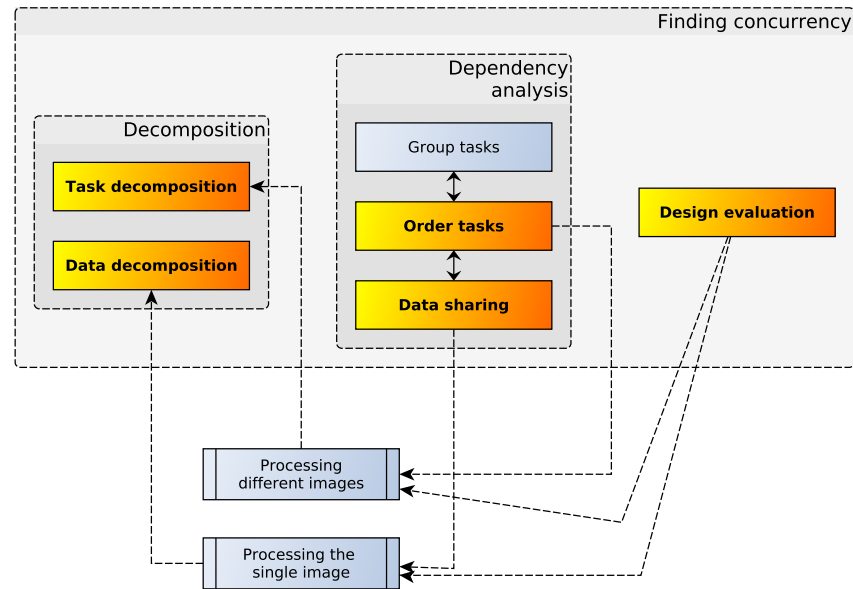


Figure 5.8 Finding concurrency design space explored: the patterns used are outlined in red/bold, the relation with the concurrent activities individuated are show as dotted lines.

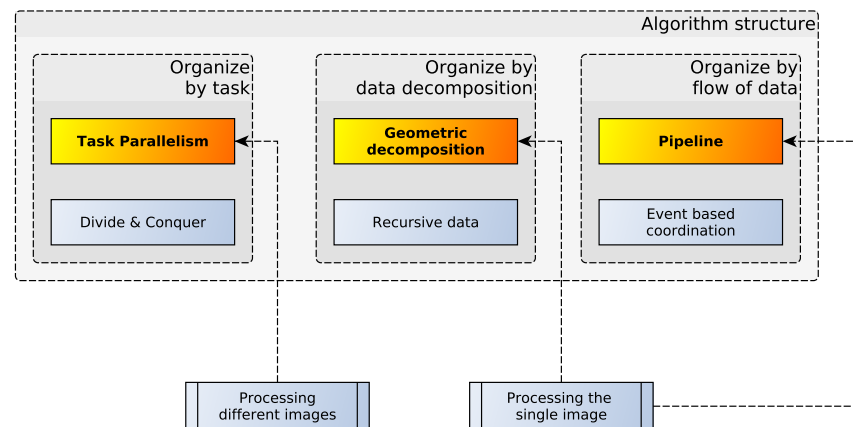


Figure 5.9 Algorithm structure design space explored: the patterns used are outlined in red/bold, the relation with the concurrent activities individuated are show as dotted lines.

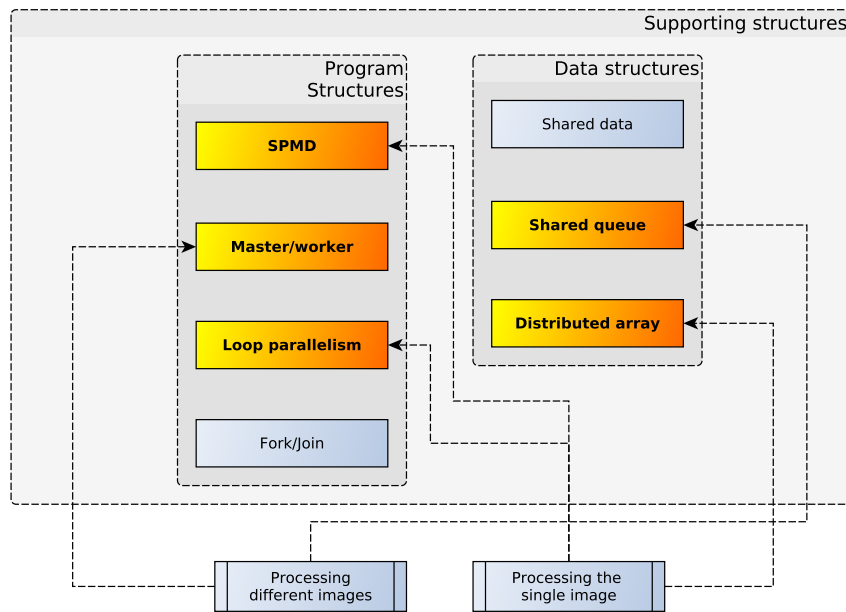


Figure 5.10 Supporting structures design space explored: the patterns used are outlined in red/bold, the relation with the concurrent activities individuated are show as dotted lines.

By picking up these patterns (see Fig. 5.9) we obviously must consider all the pattern related aspects. Remember a design pattern is a kind of “recipe” solving a particular problem. You—the application programmer—are charged of the task of instantiating the recipe to the case at hand.

5.3.4 Exploring the “supporting structures” design space

Eventually, we start considering more concrete implementation aspects by exploring the “supporting structures” design space.

As far as the “program structures” pattern group is concerned, we should look for patterns modelling the different concurrent activities individuated so far. The task parallel computation of different pictures of the input stream will be most suitably implemented using a “master/worker” pattern. Each worker will be in charge of processing a single image. The number of workers used—the parallelism degree—will determine the performance of our application. When considering the implementation of the different filter stages according to the “data decomposition” pattern, we have two choices that are worth being explored. We can use the classic “SPMD” pattern to model data parallel processing of the single filter. But we may also choose to use the “loop parallelism” pattern for the very same purpose. In case we use modern multi core architectures, the “loop parallelism” pattern may be a good choice, due to the facilities included in different programming environments (e.g. SSEx/AVX exploitation within g++ or the `#pragma parallel` for directive in OpenMP) to efficiently support this kind of parallelism. In case we use COW/NOW architectures, instead, the choice will be definitely to use the “SPMD” paradigm which is the one supported in MPI.

While considering the “data structures” pattern group, instead, we will probably eventually pick up the “shared queue” pattern to communicate (partially computed) images across different stages/workers, and the “distributed array” pattern to support data parallel implementation of the two distinct image filters.

5.3.5 Exploring the “implementation mechanisms” design space

Eventually, we have to explore the design space related to the implementation mechanisms. Let’s assume we are targeting a cluster of multi core processing elements running a POSIX compliant operating system. A possible choice of patterns in this design space implementing the patterns chosen in the “supporting structures” design space will be the following one:

- both threads and processes will be used in the “UE mechanisms” pattern. In particular, we may choose to use processes to implement the master/worker pattern and threads to implement the data parallel computation of the two filter stages.
- locks and mutexes from the “synchronization” patterns will be used to implement the “shared queue” and the “distributed array” patterns at the single (multi core) processing element level.
- asynchronous point-to-point communications from the “communication” pattern will be used to implement the steps needed to feed workers in the master/worker pattern processing different images and stages in the pipeline processing different filters on the same image.

5.4 COMPARING PARALLEL DESIGN PATTERNS AND ALGORITHMIC SKELETONS

After having discussed how parallel design patterns are structured and how they can be used to design and implement parallel applications, let’s try to set up a kind of “summary” of different features relative to both design patterns and to algorithmic skeletons, in the perspective of better understanding the pros and cons of both approaches. Fig. 5.11 summarizes how different features are supported in parallel design pattern and algorithmic skeleton (typical) frameworks.

The overall statement we can make out of this comparison is the following one:

Algorithmic skeletons and design patterns tackle the same problem(s) from different perspectives: high performance computing and software engineering, respectively.

Algorithmic skeletons are aimed at directly providing pre-defined, efficient building blocks for parallel applications to the application programmer, while design patterns are aimed at providing “recipes” to program those building blocks, at different levels of abstraction.

A large number of algorithmic skeleton frameworks are available, while design pattern related methodologies are used in different—apparently non structured—parallel programming frameworks.

From this summary two important considerations may be made:

- on the one side, the pattern methodology may be used to improve the design of parallel applications with algorithmic skeleton based frameworks
- on the other side, some formalization of the design pattern concepts such that the “solution” element of the pattern may be provided as one or more library entries/object classes could support the definition of a pattern/skeleton programming environment/framework merging the more evident advantages of both worlds.

It is worth pointing out that the design patterns listed in Sec. 5.2 may be extended considerably and with profit. A number of different parallel design patterns have been actually proposed and used, which are not included in [43]²⁰. Just to make an example,

²⁰e.g. see the ones described in the works collected at the <http://www.cs.uiuc.edu/homes/snir/PPP/> web site

	Algorithmic skeletons	Parallel design patterns
<i>Communities</i>	High performance computing	Software engineering
<i>Active since</i>	Early '90s	Early '00s
<i>Style</i>	Declarative, library calls	OO, method calls
<i>Abstractions</i>	Common, reusable, complete, nestable parallelism exploitation patterns	Abstract parallelism/concurrency patterns, Implementation patterns
<i>Level of abstraction</i>	Very high	Very high to quite low, depending on the design spaces
<i>Nesting</i>	Allowed, encouraged	Allowed, encouraged
<i>Code reuse</i>	Allowed, encouraged	Allowed, encouraged
<i>Expandability</i>	Poor/none, some frameworks support skeleton set extension through direct access to the implementation layer	Supported: new “recipes” may be added to the design spaces
<i>Programming methodology</i>	Instantiate and combine skeletons from a kind of “skeleton palette”	Figure out which patterns are worth being used, eventually implement the resulting structure on your own
<i>Portability</i>	Ensured by different implementations of the skeleton framework (or different implementation template libraries)	Ensured “by definition”: design patterns are target architecture independent, although some can be used only on certain architectures
<i>Layering</i>	Usage layering: two tier model with task parallel skeletons at top, and data parallel skeletons at the bottom level	Design layers
<i>Maintained programming frameworks</i>	Multiple, C/C++, Java, Ocaml based, targeting COW/NOW, multi cores, multi cores with GPUs	Several recent parallel programming frameworks adopted design patterns concepts (e.g. TBB [32] or TPL [30])
<i>Notable frameworks</i>	P3L, ASSIST, Muesli, SkeTo, Skandium, FastFlow, SkePu, Mallba, OSL	COPPS
<i>Automatic performance tuning</i>	Possible, demonstrated, both static (performance models and template assignment) and dynamic (behavioural skeletons)	Possible, in principle
<i>Performance</i>	Assessed. Comparable with the ones achieved using classical, non structured programming environments.	In principle, same as for skeletons.
<i>Time to deploy</i>	Greatly reduced w.r.t. classical parallel programming frameworks, both design and implementation/tuning/debugging time	Quite limited design time. Implementation time comparable to the times needed when using classical parallel programming frameworks. Tuning and debugging time reduced.
<i>Debugging</i>	Debugging of sequential code portions (only) is needed	Parallel pattern and sequential code debugging needed

Figure 5.11 Summary of features in parallel design patterns and algorithmic skeletons

consider the “pipeline” pattern included in “organize by flow of data” pattern group in the “algorithm structure” design space. The “supporting structures” design space does not provide a specific pattern suitable to be used to implement a pipeline algorithm. Although both the “SPMD” and the “master/worker” patterns may be adapted to execute “pipeline” stage tasks, a “pipeline” “supporting structure” pattern would be greatly useful and not too much difficult to introduce.

Actually, different authors—including the author of these notes—recognize that only three fundamental “supporting structures” are needed to implement any parallel application, provided a convenient number of “connector patterns” are also provided. These fundamental “supporting structure” patterns are the “string of workers”—a number of concurrent activities processing relatively independent tasks in parallel—the “chain of workers”—a number of concurrent activities each processing results from the previous one and delivering results to the next one in the chain—and feedback loops—communications channels driving back some results to previously executed concurrent activities. The “connector patterns” needed to set up proper networks of concurrent activities out of these supporting structure patterns—i.e. to manage communication and synchronization among the concurrent activities—include point-to-point and collective communications, lock/mutex/semaphores and barriers and logically shared data structures.

CHAPTER 6

SKELETON DESIGN

In this Chapter we discuss the problems related to skeleton (set) design. In particular, we want to answer two questions:

- How can we identify suitable skeletons/parallel design patterns?
- How can we design a good skeleton/parallel design pattern?

In order to answer these question we start considering again the definition of algorithmic skeleton we gave in Chap. 4:

An algorithmic skeleton is parametric, reusable and portable programming abstraction modeling a known, common and efficient parallelism exploitation pattern.

From this definition we can figure out the principles that should inspire the process of identifying and designing new skeletons.

- We should look for *parallelism exploitation patterns*. This means that parallelism is the main subject, the kind of parallelism modeled has to be clearly identified and described and, obviously, the pattern should be different from the patterns already modeled by other already defined skeletons or by a composition of already defined skeletons.
- The skeleton should be *parametric*. It should be possible to specialize the skeleton by passing a proper set of functional and/or non functional parameters. In particular all the computations differing by features not relevant to distinguish the parallelism exploitation pattern used must be expressible with the same skeleton by simply instantiating it with different parameters.
- The skeleton should be *reusable*: a number of different situations, possibly in different application domains should exist suitable to be modeled by the skeleton. In case we

have only one or two²¹ situations where the skeleton may be used to model parallelism exploitation, the skeleton is not worth to be included in the skeleton set, unless it results to be the only way to model that particular situation and the few cases where the skeleton may be used cover very common application cases.

- A skeleton is a *programming abstraction*. Therefore the new skeleton should clearly abstract all the details of the modeled parallelism exploitation pattern. Moreover, it must be provided to the final user²² in a way which is compliant to the general rules used to define the other skeletons in the current skeleton set and respecting the syntax rules inherited from the skeleton “host language”²³.
- The new skeleton should be *portable* and *efficient*: implementation should exist for the main class of parallel architectures and these implementations should be efficient, although efficiency may vary across different architectures. Take into account that we are defining a new skeleton—which is a kind of abstract concept—while the portability and efficiency are properties related to implementation, actually. The point here is that when we design a new skeleton we should be able to imagine how it can be implemented efficiently on a range of target architectures. This to avoid the situation where we have a very nice “instruction set”—the algorithmic skeleton set—but this instruction set could not be implemented efficiently and therefore no user will be prone to adopt it and to use it.
- Last but not least, the new skeleton should abstract a *common* parallelism exploitation pattern. Here we have to make some distinction. Common parallel patterns are the things we are actually looking for, when designing a new skeleton. However, the adjective “common” may be interpreted in different ways. We can intend “common” as “appearing in wide range of different applications *from different application domains*”, or we can release the “different application domains” requirement while keeping the rest of the sentence. In the former case, we will end up with true algorithmic skeletons. In the latter, we will end up with more *domain specific* skeleton sets. In principle, both true and domain specific skeleton sets are worth to be investigated and designed. We must point out that several of the skeleton frameworks designed in the past (or currently being maintained) were somehow “domain specific”, at least in their author intentions: Skipper [47, 21] has been developed to support image processing and MALLBA [2] is aimed to support optimization algorithm design, as an example.

6.1 COLE MANIFESTO PRINCIPLES

In his algorithmic skeleton “manifesto” [19] Cole lists four principles which “should guide the design and development of skeletal programming systems”. These systems can be read as principles dictating requisites on the skeletons that have to be eventually included in the skeleton set provide to the application programmer. Here we recall the Cole principles and we comment their impact on the skeleton design process.

1. Propagate the concept with minimal conceptual disruption. The principle, as illustrated in the manifesto, states that “simplicity should be a strength”. If we also take into account that following a RISC approach in the design of the skeleton set²⁴ is definitely a good

²¹more in general, just a very few

²²the application programmer

²³the language used to express sequential portions of code and data types

²⁴that is trying to define a small set of simple and efficient skeletons that may eventually be combined to achieve more complex parallelism exploitation patterns

practice, then it is evident that the development of *simple, general purpose* and *composable* skeletons is the final goal we must try to achieve. In addition, these skeletons must be provided to the application programmers in *a way as compliant as possible* with the rules and conventions of the sequential programming framework used to embed/host the skeleton framework.

2. Integrate ad-hoc parallelism. The principle was stated by Cole to point out that skeletons should support the integration of parallel forms not originally provided by the skeleton set designer. These parallel forms should be supported in such a way that they eventually integrate in the rest of the skeleton set. As an example, the user must be enabled to program its own parallel pattern and then the skeleton framework should allow the user to use the pattern as a skeleton in all the places where one of the original skeletons could be used. Cole's eSkel skeleton framework [12], as an example, provided the user with the possibility to autonomously manage a set of MPI processes allocated by the skeleton system and linked with proper communication channels to the rest of the skeletons used in the application. With respect to the different new skeleton design problems discussed in this Chapter, this principle has no particular effect. However, when considering the design of the overall skeleton set we must be careful to introduce some kind of *escape* skeleton suitable to support user²⁵ defined parallelism exploitation patterns and perfectly integrated with the rest of the skeleton set. In Sec. 11 we discuss how skeleton set expandability may be achieved in a macro data flow based implementation of a skeleton framework. In this case, the "escape" skeleton is the one where the user may directly define the macro data flow graph that constitutes the result of the compilation of the skeleton.

3. Accommodate diversity. The principle states that when designing skeletons "we must be careful to draw a balance between our desire for abstract simplicity and the pragmatic need for flexibility", that is we have to consider that simple skeleton are worth provided the simplicity of the skeleton does not impair the possibility to use the skeleton to model slightly different parallelism exploitation patterns. Here we can assume that this principle has two main consequences on the design process. On the one hand, the new skeletons should be properly defined using a significant set of parameters. Possibly, some of the parameters may be optional. The parameters should be used to differentiate the general skeleton behaviour and clear semantics should be given (functional and parallel) that states the effect of the combined usage of the parameters when instantiating the skeleton in a real application. On the other hand the new skeleton should comply the rules defined when looking for the possibility to "integrate ad-hoc parallelism", in such a way the user may extend parts of the skeleton to implement his/her slightly different pattern. As an example, when defining a farm, the user may wish to implement some particular scheduling policy, different from the system defined one, to support other kind of computations with respect to those supported by the original farm.

4. Show the payback. This principle is more related to the design of the overall skeleton set, actually. However, in order to build a skeleton framework able to "show the payback" to the final user, we should include in the skeleton set only new skeletons that i) may be used in a variety of well known situations, in different application fields, ii) should be high level, that is they should not include (too much) implementation dependent details, and iii) they should be easy to use, that is they should be perfectly integrated with the overall programming framework.

²⁵application programmer

6.1.1 Summarizing ...

If we try to summarize what we discussed above, we can obtain the following set of requisites for a new skeleton to be included in the skeleton set provided to the application programmer:

- a) it must model a common, efficient parallel pattern
- b) it must be easy to use
- c) it must be defined in terms of a comprehensive set of parameters
- d) it should tolerate/support user defined variations
- e) it must be efficiently implementable on a range of parallel target architectures.

6.2 LOOKING FOR (NEW) SKELETONS

In this Section we consider a methodology for discovering and designing new skeletons. Actually, we will distinguish two cases: one based on the analysis of existing parallel applications, and the other one based on the specific needs of a user implementing a particular application. In reality the second one constitutes an adaptation of the first one with just a different input/starting point.

6.2.1 Analysis

In this case, the idea is to look for new skeletons in existing parallel applications. Therefore the procedure may be outlined as follows:

1. We decide whether to analyze applications from different domains or from a single application domain. In the first case, the procedure will most likely terminate with the individuation of new general purpose skeletons, whereas in the second case it will most likely terminate with the individuation of domain specific skeletons. In both cases, we select a number of applications *whose code is accessible* and/or *whose parallel algorithm is clear*.
2. We “distill” from the application individuated in the previous phase the parallel patterns used to express and model parallelism. This activity may require some kind of “reverse engineering” of the source code. In some cases some support by the application designers could be necessary to avoid digging huge amounts of source code. It has to be considered that in normal parallel applications the parallel structure of the application is not exposed “per se” and therefore we need to look at the mechanisms used to implement parallelism exploitation in order to be able to re-construct (reverse engineer) the high level parallel structure of the application.
3. We select the candidate new skeletons among those distilled in the previous phase that have no correspondent²⁶ skeleton in the currently available skeleton set.
4. We perform a comprehensive analysis of the candidate new skeletons. In particular, we try to understand whether each new skeleton candidate is in case equivalent to some composition of other, new and simpler skeletons. In case, we try to understand whether these simpler skeletons are worth to be included in the skeleton set, that is if they actually model significant parallel patterns (or, possibly, significant parallel

²⁶that is, whose parallel exploitation pattern could not be modeled using the skeletons currently available either alone or in proper nesting

pattern building blocks). If the new skeletons individuated cannot be further decomposed, we investigate if the new skeleton satisfies all the requirements listed at the end of the previous section, derived from either the algorithmic skeleton definition or from Cole’s manifesto.

5. We proceed with new skeleton refinement. We design the skeleton name, syntax, parameter set and semantics. In the design of the skeleton parameter set we take into account both functional (e.g. worker functions, task and result data types) and non functional (e.g. parallelism degree, scheduling policies (if any)) parameters, and we also take into account any optional parameter that may be used to further differentiate the skeleton parallel pattern. When defining semantics, we define both functional and parallel semantics, as usual.

6.2.2 Synthesis

In this case the idea is to proceed in the definition of new skeletons starting from the particular needs of an application programmer trying to implement a specific parallel application. The process starts in case we verify that a pattern, significant and useful to implement (part of) the parallel application we are considering, cannot be expressed by any of the existing skeletons or by a composition of the existing skeletons. We then proceed as follows:

1. We perform an analysis phase such as the one listed in point 4 of the procedure outlined in Sec. 6.2.1.
2. Then we look for other cases where such a skeleton may be used. We therefore follow a procedure similar to the one listed in points 1 and 2 of the procedure outlined in Sec. 6.2.1 to individuate the application parallel patterns and we look for equal or “similar”²⁷ patterns. If a number of situations are individuated where the new pattern may be used (as is or slightly modified) we go on with the new pattern design process, otherwise we stop here.
3. We refine the skeleton definition, also taking into account the needs evidenced by the other situations where the new pattern could be eventually used, and we eventually define the new skeleton name, syntax, parameters and semantics.

6.3 SKELETONS VS TEMPLATES

An algorithmic skeleton is fundamentally a programming abstraction. A template, as introduced in this book, is an implementation of a skeleton on a given target architecture, including a process template (or a thread template) and some kind of performance model.

When looking for new skeletons it is easy to be convinced that a *recurring implementation* of some parallel pattern is a skeleton, especially during a process such as the one described in 6.2.1. We should take care of not confusing the parallel programming abstraction with the implementation, however.

E.G.▷ As an example, take into account a task farm skeleton. The programming abstraction presented to the user is such that the user can “submit” a set of independent tasks—possibly available at different times—and get them computed in parallel, with a speedup hopefully proportional to the number of parallel processing engines (processes on different processing elements, threads on different cores) used.

The emitter and collector concepts we introduced and we used several times in the book *are not* logically belonging to the task farm skeleton. Rather, they are related to a possible,

²⁷with respect to our “new skeleton” candidate

relatively efficient implementation of the task farm. In fact, we pointed out how a similar implementation may also be used to implement a map skeleton, provided the emitter splits the incoming “collection” data into items to be delivered to the workers for independent computation and the collector gathers all the results from the workers and eventually “builds” the map result. ■

The point here is particularly important if we refer back to the discussion relative to CISC vs. RISC skeleton sets we introduced in Sec. 4.3. When looking for RISC skeleton sets, we may easily end up with the idea that the “elementary” building blocks are the simplest, possibly recurring, patterns used to implement the skeletons rather than the skeletons themselves. We may consider the emitter, a string of workers, a collector, a chain of workers, a tree of workers as the base building blocks of our parallel computations. This may be reasonable in perspective, but we have to take into account that:

- the “assembly” of these building blocks requires a deeper knowledge than the one required to nest “true” skeletons. As an example, to build a map out of the emitter, string of workers and collector building blocks, we need to be aware of the decomposition level used to feed workers. In a map skeleton, this is a parameter usually left in charge of the map implementation, and therefore under the responsibility of the system programmer rather than the of the application programmer.
- the level of abstraction of these basic building blocks could not avoid taking into account the target architecture, as skeletons in principle may do. An emitter building block for a complete interconnection architecture such as an workstation cluster or a symmetric multiprocessor (multicore) will be different from the same building block for another architecture not supporting complete interconnection, such as a mesh interconnected network of workstations/cores or a NUMA multi core. In addition to the already mentioned shift of responsibilities from system to application programmers this has also huge consequences on portability of parallel programs on different architectures.

CHAPTER 7

IMPLEMENTATION

Algorithmic skeleton frameworks can be implemented according to different models. A first, important alternative is among implementations introducing new programming languages and implementations actually introducing only libraries for well know, sequential programming languages. In both cases, skeleton programming frameworks may be implemented using different software technologies. Traditionally, algorithmic skeleton frameworks have been implemented exploiting *implementation templates*. Most of the early implementation of skeleton frameworks actually exploit this technology. In late '90s, a *macro data flow* based implementation model has been proposed. More recently, implementations of skeleton frameworks exploiting *AOP* (Aspect Oriented Programming), *annotations* and *software components* have been designed and implemented. In the following sections, we will discuss these different approaches.

7.1 LANGUAGES VS. LIBRARIES

Algorithmic programming frameworks may be provided to the application programmers in two fundamentally different flavors:

- As a brand **new programming language**, hosting skeletons as proper language constructs and compiled (or interpreted) in such a way the skeleton programs may be suitably executed on the target parallel architecture at hand, or
- as a **library** written in a particular *host language*.

7.1.1 New language

In case of skeleton frameworks implemented as new programming languages, the application programmer is exposed to a new language sporting skeletons as primitive constructs. No-

table examples of this approach are the P3L [11] and ASSIST [50] programming frameworks. In these cases, the programmer writes programs that include particular statements/declarations declaring skeletons as well as the main program to be executed. As an example, in P3L, a skeleton program looks like:

```
seq filter in(Image x) out(Image y)
$c++{
    // ... c++ code producing y out of x here
$c++}

seq recon in(Image x) out(String y[])
$c{
    /* C code producing the array of strings
       y out of the Image x here */
$c}

pipeline main in(Image a) out(String c[])
    filter in(a) out(Image b)
    recon in(b) out(c)
end pipeline
```

The program, once compiled through the `p3lcc` compiler may be run as a plain Unix filter processing inputs from the standard input and delivering outputs to the standard output. The sequential pieces of code wrapped into the `filter` and `recon` sequential skeletons are compiled using plain C++ or C compilers, as indicated among the `$` and `{` characters introducing the sequential code. The program is defined by the topmost skeleton, i.e. the skeleton not parameter of any other skeleton, which turns out to be the `pipeline main` in our case. Proper variable scoping is defined in such a way the variables named across different skeleton calls (e.g. variable `c` used as output of `filter` and input of `recon`) model communication channels among skeletons²⁸.

When a new programming language is introduced to provide skeletons, a set of compiler/interpreter tools are provided such as those of Fig. 7.1. A *compiler* translates source code into intermediate/object code. This object code is then linked with proper libraries (e.g. communication libraries) and eventually run on the parallel computing resources at hand by the *linker/launcher* tool.

The compiler and the linker/loader tools may be implemented by orchestrating different compiler/linker tools, actually. In most cases, the new programming languages providing skeletons are not compiled directly to object code. Rather, they are compiled to some intermediate “source + library call” code in some *host* sequential programming language that is then compiled to object code using the host sequential language compiler. This happens to avoid re-writing sequential compilers when these are already available. The “library calls” here are usually calls to some communication library. As an example, in P3L the `p3lcc` compiler produced C + MPI “object code”. This code was eventually compiled with the `mpicc` compiler. The resulting code was eventually run on the parallel computing elements with the `mpirun` command.

7.1.2 Library

When the algorithmic skeletons are provided as libraries, a completely different approach is followed.

²⁸In case of P3L, these channels were implemented using MPI channels [16]

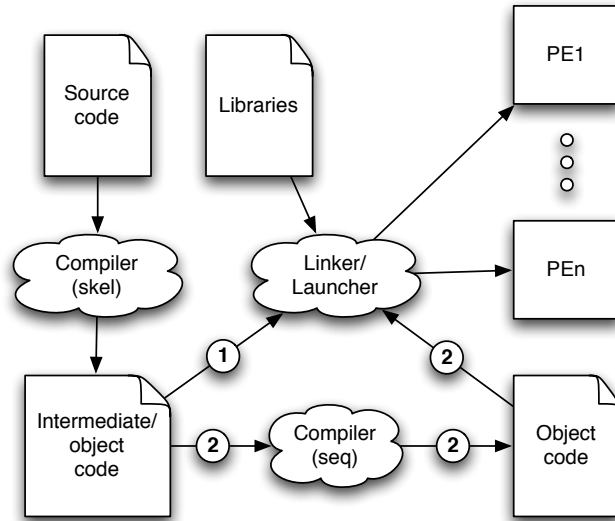


Figure 7.1 Tool chain for skeleton frameworks providing a new language (workflow ① and ② represent alternatives)

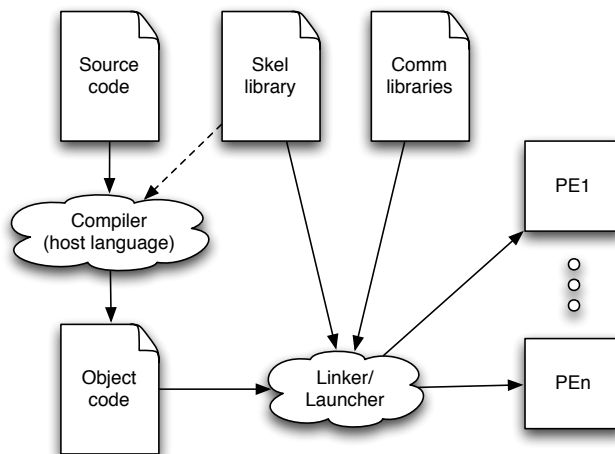


Figure 7.2 Tool chain for skeleton frameworks implemented as libraries

Skeleton programs are written in some “host sequential” programming language (host source code). Skeletons are either declared or simply called via host language library calls. The library implementation, in turn, implements the skeleton functional and parallel semantics. The skeleton library implementation may use different kinds of communications libraries (e.g. MPI, POSIX/TCP, JXTA) as it happens in the case of skeleton frameworks implemented as new programming languages. In this case the tool chain to be implemented looks like the one in Fig. 7.2.

The distinction among “declaration” and “call” of skeletons is worth to be explained in more detail. Several skeleton frameworks (e.g. Muesli[35] and Muskel[7]) *declare* the skeleton program and then a specific statement is inserted in the program to actually compute the skeleton program.

The Muskel code shown in Sec. 4.2.2 shows how this approach works. First a main skeleton is declared, using proper composition of objects of class `Skeleton`. Then an `ApplicationManager` object is used to start the computation of the program.

Other skeleton frameworks, instead, use library calls to directly execute skeleton code. The SkeTo code in Sec. 4.2.2 shows an example of map and reduce skeletons which are called for execution. In fact, the philosophy of SkeTo is such that the programmer may intermingle skeleton calls to sequential code as if they were plain, sequential library calls[48, 42].

7.2 TEMPLATE BASED VS. MACRO DATA FLOW IMPLEMENTATION

Two quite general models have been used to implement parallel programming frameworks based on algorithmic skeletons:

- *implementation templates*, and
- *macro data flow*.

The former is the model used in the firsts algorithmic skeleton framework implementations. It has been used later on and it is currently used in the implementation of the vast majority of skeleton based parallel programming frameworks (e.g. in eSkel, Muesli, SkeTo, ASSIST). The latter has been introduced in late '90. It is radically different both in the techniques used to implement the algorithmic skeletons and in the possibilities opened in terms of optimizations, fault tolerance and portability. With several variants, it has been adopted in different frameworks, including Muskel (where it has been developed first), Skipper, Calcium and Skandium²⁹.

In the following subsections we outline the two implementation technique and we point out the main similarities and differences.

7.2.1 Template based implementations

In a template based implementation of a skeleton framework, each skeleton is eventually implemented by instantiating a “process” *template*³⁰. Historically, we use the term *process template* as the typical concurrent entity run in parallel in the first skeletons frameworks was the *process*. Currently, we should better refer to the templates as *concurrent activity templates* to denote the fact the concurrent activities implementing a given skeleton actually may be a mix of processes, threads, GPU kernels etc.

A process template is a parametric network of concurrent activities, specified by means of a graph $G = (N, A)$. Each node N_i in the graph represents a concurrent activity.

²⁹actually both Calcium and its successor Skipper, use an implementation based on evaluation stacks. This notwithstanding the execution mechanism eventually looks like the one used in macro data flow implementations.

³⁰In literature, the term “implementation template” has been used as well

An arc $\langle i, j \rangle$ from node N_i to node N_j denotes a communication moving data (or pure synchronization signals) from node N_i to node N_j . Each node has associated an informal “behaviour”, that the program executed on that node. Parameters in the specification of a process template vary.

As an example, the parallelism degree of the template is normally a parameter, as well as the business code to be executed on the node or the type and number of the channels (links) insisting on the node.

Each process template is associated with some further metadata. The metadata provide further information needed to make a proper usage of the template. Examples of metadata associated to the template include:

Target architecture : a description of the target architecture(s) were the template may be conveniently and efficiently used.

Skeletons supported : a description of the skeletons that can be possibly implemented in terms of the template.

Performance model : a description of the performance achievable with the template, parametric with respect to both application dependent parameters (e.g. the execution times of the sequential portions of business logic code used in the template) and system dependent parameters (such as communication latency and bandwidth relative to the target architecture).

Let us suppose a process template is described by a tuple such as:

$$\langle name, \{Sk\}, \{Arch\}, PerfMod, G = (N, A) \rangle$$

and that at least one template exist for any skeleton provided by the skeleton framework. Then the compilation of a skeleton application can be obtained as follows:

Phase 1: skeleton tree The source code of the application is parsed and a skeleton tree, such as those described in Sec. 4.2.3, is derived

Phase 2: template assignment The skeleton tree is visited and a template is assigned to each one of the nodes in the tree. The template T_j assigned to a node of type³¹ Sk_i is such that

- C1 the template T_j is a template suitable to implement skeleton Sk_i
- C2 the template T_j is suitable to be implemented on the target architecture considered
- C3 possibly, the performance model of the template T_j demonstrates better performance than those of any other template T_i satisfying the conditions C1 and C2 above.

Phase 3: optimization The annotated skeleton tree³² is visited again and the parameters of the templates are filled up, using either heuristics or the performance models to devise correct actual parameter values.

Phase 4: code generation The “object” code for the parallel application is generated. The object code is usually built of a set of programs in a high level language with calls to suitable communication and synchronization libraries, such as C/C++ with MPI calls or OpenMP pragmas. During code generation usually makefiles and metadata needed for code deployment on the target architecture are produced.

³¹modeling a skeleton

³²each node is annotated with a template

It is worth pointing out that the very same process may be implemented in a library, rather than in a set of compiling tools. In this case:

- Phase one derives from the fact skeletons are declared as library objects, and therefore there is no need to parse source code to get the skeleton tree
- Phases 2, 3 and 4 are executed *on-the-fly* when the execution of the skeleton application is started, via the proper library call. In general, this means the “execute” call sets up a number of concurrent activities that then specialize according to the different processes needed to implement the templates. As an example, Muesli skeletons are implemented as a library and Muesli uses a template based approach. The skeletons composing the program are *declared* as C++ objects. Then a execution is invoked. The execution starts an MPI run. MPI process identifiers are used to specialize each MPI process to perform as expected by the template used to implement the application skeleton (skeleton nesting).

Considering the whole process leading to the generation of the skeleton application object code we can make the following considerations.

7.2.1.1 Template selection In many cases, only one template—the “best” one—exists for each skeleton supported by the framework. In that case the selection phase is clearly void. When more than a single template exists for a given skeleton and a given target architecture, the selection may rely on information provided by the performance models associated to the template, as we detailed in the phases listed above. Two alternative approaches may be followed, however: i) heuristics may be used (such as “template T_j is preferable in case of nestings, while $T_{j'}$ has to be preferred for standalone skeletons), or ii) a random template may be selected from those available for the skeleton and architecture at hand, but then the optimization phase may substitute the template with another one, if the case. In a sense, this second approach moves duties from the first skeleton visit to the optimization visit.

7.2.1.2 Optimization phase The optimization phase may be relatively large with respect to the other phases. Optimization activities may include computation of the optimal parallelism degree, evaluation of alternative skeleton nestings providing the same semantics with different non functional features, fine tuning of the template compositions. We mention here just three examples of these optimization activities.

Parallelism degree computation. The skeleton tree may be annotated with the templates without actually specifying their parallelism degree. Once the number of processing elements available for the application execution is known, the annotated skeleton tree may be processed to assign part of the available resources to each one of the templates. P3L implemented this phase in two steps: during the first step, the skeleton tree was visited bottom up and each template was assigned an “optimal” number of resources. Optimality was defined with respect to the template performance models. Then a cyclic process was initiated aimed at taking away the minimum amount of resource from the assignment that leave a balanced assignment (i.e. a resource assignment such that there are no bottleneck skeletons). The process was iterated up to the point the number of assigned resources was smaller or equal to the number of available resources. An alternative approach could consist in generating all the feasible assignments of the available resources to the templates in the skeleton tree, then to evaluate all the assignments with the performance models and eventually keep the one(s) getting the best results. This process is much more complex, as *all the possible* assignments are considered. The other alternative (the P3L one mentioned above) only works on “good” assignments: it starts from an optimal assignment and reduces to the number of resources actually available moving through assignments that are not optimal, but keep balanced assignment of resources to the skeleton/template tree assignments.

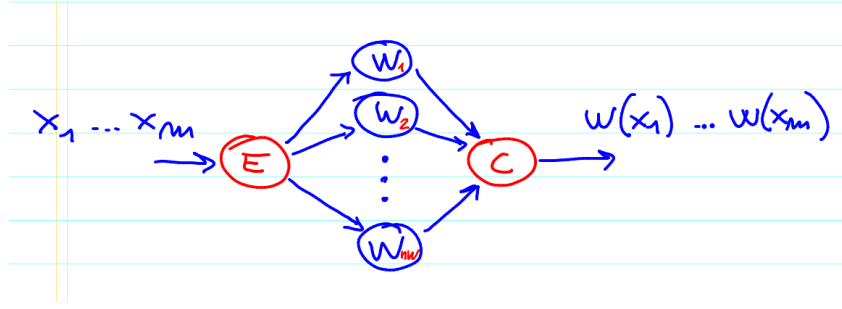


Figure 7.3 Farm template for complete interconnection architectures

Skeleton nesting rewriting. We know functional equivalences exist between skeleton trees (see also Sec. 9). As an example a

$$\text{Pipe}(\text{Farm}(\text{Seq}(s1)), \text{Farm}(\text{Seq}(s2)))$$

is functionally equivalent to a

$$\text{Farm}(\text{Pipe}(\text{Seq}(s1), \text{Seq}(s2)))$$

Now it is clear that optimization phase may decide to rewrite a skeleton tree into a functionally equivalent skeleton tree, provided that some advantage is achieved in terms of non functional properties of the skeleton tree. It is also clear that, for any rule stating the functional equivalence of skeleton tree S_i and S_j , it could be the case that with respect to a non functional concern it is better to use the first form while with respect to another non functional concern it is better to use the second form.

Template composition tuning. As it will be clearer later on, template composition may turn out into having a template T_i delivering data to template T_j such that T_j processes this data with a function $f = f_n \circ \dots \circ f_1$, T_i delivers data resulting from the application of $g = g_m \circ \dots \circ g_1$ and $g_m \equiv f_1^{-1}$. A typical example is the case of two templates where the first one builds an array out of the single rows computed in parallel by the template worker processes and the second template unpacks any matrix received in rows to submit the rows to another set of worker process³³. In this cases, the composition of the two templates may be optimized simply taking away the g_m computation from the first template and the f_1 computation from the second template.

7.2.1.3 Code generation Code generation depends a lot on the target environment considered. Algorithmic skeletons frameworks usually target environments with high level languages and communication libraries (C/C++ with MPI or OpenMP, Java with TCP/IP sockets). In this case, the concurrent activities used to implement the templates are processes or threads whose body may be produced using simple macros as well as instantiating objects and providing the properly wrapped business logic code parameters. In case lower level programming environments are targeted, the code generation may be sensibly larger. As an example, if assembly language + communication libraries were assumed as the target environment, this would require a considerable effort relative to the code generation and optimization.

7.2.1.4 Commonly used templates There are several process templates that are used in different skeleton frameworks. We describe a couple of templates to further clarify the process template concept.

³³this case may arise when we compute a $\text{Pipe}(\text{Map}, \text{Map})$ application, as an example

Farm template for complete interconnection architectures. A farm skeleton models embarrassingly parallel computation over streams. One common template used to implement the farm skeleton uses an *emitter* process, a *collector* process and a string of *worker* processes (see Fig 7.3).

- The emitter takes care of receiving the tasks from the farm input stream and to schedule them—in a fair way—to the worker processes.
- Worker processes receive tasks, compute them and deliver results to the collector process.
- The collector process simply gathers results from the workers and delivers those results onto the farm output stream.

The performance model of the process template establishes that the farm service time is given by the maximum among the time spent by the emitter to schedule a task to one of the workers, the time spent by the collector to gather a result from one of the workers and to deliver it to the farm output stream and by the time spent by one of workers to process a task divided by the number of the workers in the farm.

This farm template has several parameters:

- the parallelism degree n_w , denoting how many workers are actually included in the process template.
- the scheduling function implemented by the emitter (e.g. round robin or on demand)
- whether or not the input task ordering has to be preserved onto the output stream results (this is possibly implemented by the collector)
- the type of input tasks
- the type of output tasks
- the computation performed by each one of the workers, expressed as another, possibly sequential, skeleton

This template may be used on any architecture supporting full interconnection among processing elements. A “companion” task farm template may be obviously defined using threads instead of processes. In that case, symmetric multiprocessors can be targeted.

Farm template for mesh architectures (version 1). When targeting mesh interconnected architectures, a farm template may be defined in terms of an emitter process, a string of emitter routers, a string of workers, a string of collector routers and a collector process. The interconnection among those elements is such that the mesh interconnection available is used:

- the emitter process is placed at position (i, j) in the mesh
- the n_w emitter routers are placed at positions $(i, j + k)$ with $k \in \{1, n_w\}$
- the n_w worker processes are placed at positions $(i + 1, j + k)$ with $k \in \{1, n_w\}$
- the n_w collector routers are placed at positions $(i + 2, j + k)$ with $k \in \{1, n_w\}$
- the collector process is placed at position $(i + 3, j + n_w + 1)$

as shown in Fig. 7.4. The behaviour of the different processes in the mesh is the following:

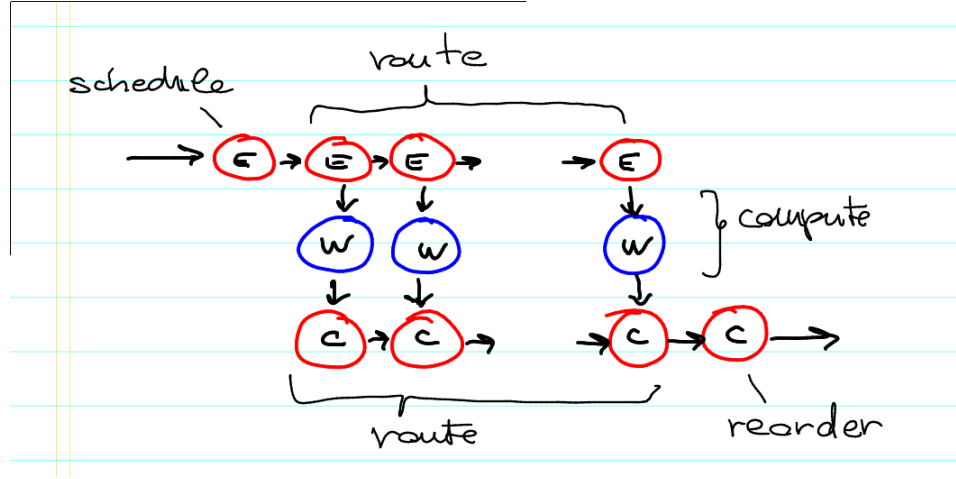


Figure 7.4 Farm template for mesh connected architectures

- the emitter process receives tasks from the task input stream and outputs passes these tasks to the first emitter router
- emitter router of index i receives $n_w - i$ tasks and passess them to the next emitter router. Then it receives another tasks and passes this tasks to the worker of index i . Eventually it starts again receiving and passing tasks.
- worker processes receive tasks to be computed from the emitter routers, compute them and deliver tasks to the corresponding collector routers
- collector routers receive tasks either from the upper worker or from the left collector router and deliver them to the right collector router (or to the collector, in case the router is the last one in the chain)
- the collector process receives results from the n_w collector router and delivers these results to the farm output stream.

The performance model of this template states that its service time is

$$T_s = \max\{T_e, \frac{T_w}{n_w}, T_c\}$$

where T_e , T_w and T_c are respectively the time spent to route a task, to compute a task or to route a result. The parameters of the template include the parallelism degree n_w as well as the parameters related to the computation performed: types of tasks and results, skeleton to be computed in the workers.

Farm template for mesh architectures (version 2). This second template is built of a number of processes arranged in a pipeline. Process i receives data from process $i - 1$ and delivers data to process $i + 1$. Each process behaves as follows:

- it receives $n_w - i$ tasks, and passes these tasks to next process, then
- it receives one task, computes it and passes the results to next process
- it receives $i - 1$ results from previous process and delivers these results to next process

This template works in three phases: during the first phase (taking $n_w T_e$ units of time) all processes receive a task to compute; in the second phase, all processes compute a task

(spending T_w units of time); in the last phase all processes route results to the last process (in $n_w T_c$ units of time), the one in charge of delivering results onto the farm output stream. As a result the template processes n_w tasks in $n_w T_e + T_w + n_w T_c$ units of time thus delivering a service time of approximately

$$T_S = \frac{n_w T_e + T_w + n_w T_c}{n_w}$$

Taking into account that the time spent to route a task or a result (T_e or T_c) is small, the service time in this case can be further approximated with

$$T_S = \frac{n_w T_e}{n_w} + \frac{T_w}{n_w} + \frac{n_w T_c}{n_w} \cong \frac{T_w}{n_w}$$

7.2.2 Macro Data Flow based implementation

In late '90, an alternative implementation method for programming frameworks providing algorithmic skeletons has been introduced [23]. Rather than compiling skeletons by instantiating proper process templates, the skeleton application are first compiled to *macro data flow graphs* and then instances of this graphs (one instance per input data set) are evaluated by means of a distributed macro data flow interpreter.

Let's start defining macro data flow graphs.

A macro data flow graph (MDFG) is a graph $G = (N, A)$ whose nodes N are macro data flow instructions (MDFI). A macro data flow instruction, in turn, is a tuple

$$\langle id, f, tokens, dests \rangle$$

where

- *id* is the instruction identifier,
- *f* represents the function computed by the instruction on the input tokens,
- *tokens* is a list of $\langle value, presencebit \rangle$ pairs, whose *value* field will eventually host some data coming from another MDFI and *presencebit* is a boolean stating whether or not the value item is significant (has already been received),
- *dests* is a list of $\langle i, id, pos \rangle$ representing the position *pos* in the token list of MDFI *id* to be filled by the *i*-th result produced from the computation of *f* on the $n = \text{len}(\text{tokens})$ values of the input tokens $f(\text{value}_1, \dots, \text{value}_n)$

The arcs of the MDFG G , $a_i = \langle s_i, d_i \rangle \in A$, model the fact that a data item (output token) produced by MDFI with $id = s_i$ is directed to a MDFI whose $id = d_i$.

A input token assignment of a MDGF G is a set of $\langle value, dest \rangle$ pairs, hosting the data values used to initialize the dest tokens in the graph.

This is nothing different from usual data flow graphs used in sequential compilers to conduct data flow analysis. The term *macro* distinguish our graphs as having instructions whose function *f* is a coarse grain computation, while data flow analysis often use fine grain data flow instructions (e.g. instructions whose functions represent simple arithmetic operators). In case of macro data flow instructions used to support skeleton implementation, the functions used are typically those computed by the sequential business code wrapped into skeleton parameters.

The computation of a MDFG G with an input token assignment ITS proceeds as follows:

- the data items in the input token assignment are moved to the tokens identified by their *dest* fields.
- while *fireable* instructions exists in the MDFG, these instructions are executed and their results are directed to the proper token destinations. A fireable instruction is a MDFI whose input tokens have all been received, that is, have all *presencebits* true.

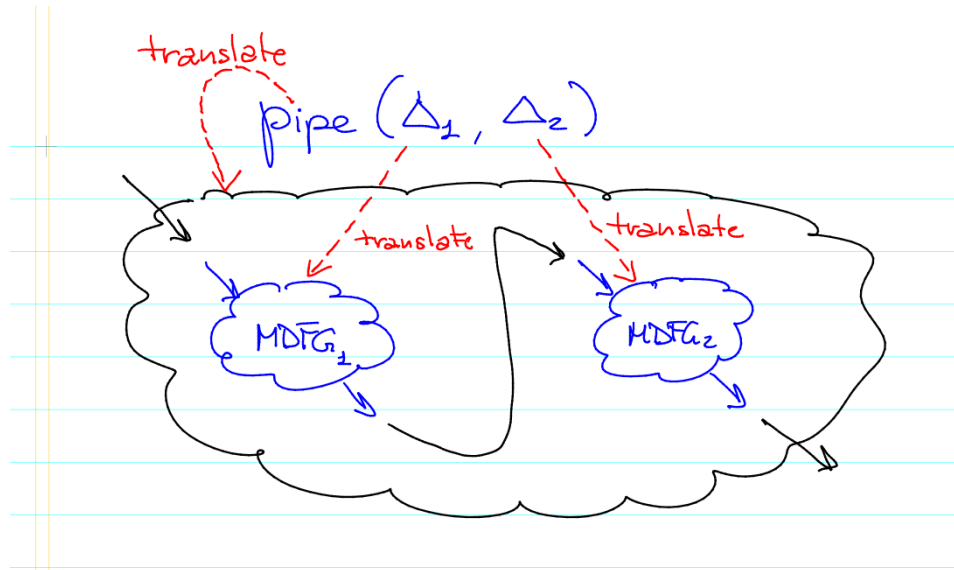


Figure 7.5 Recursive compilation of a pipeline skeleton to MDF graphs

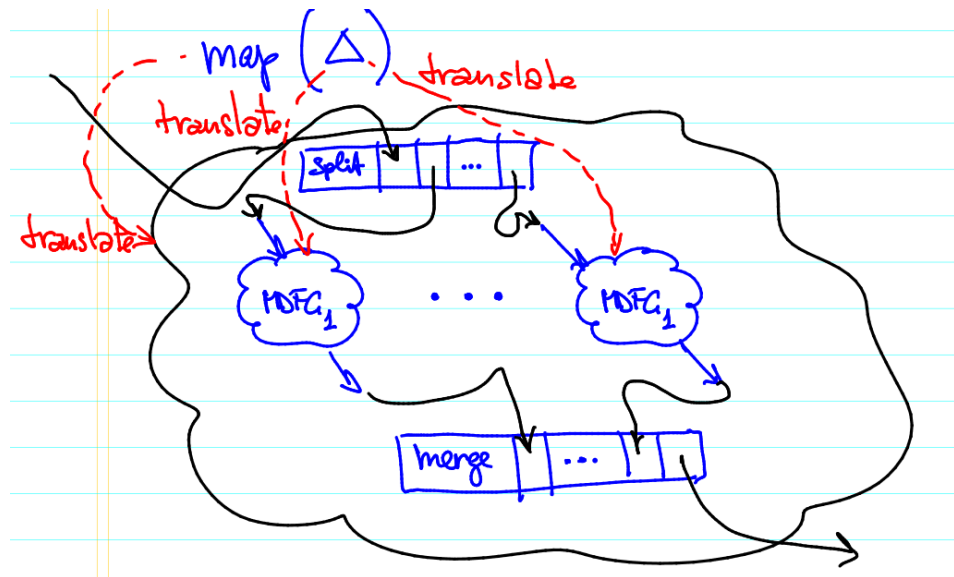


Figure 7.6 Recursive compilation of a map skeleton to MDF graphs

When a macro data flow implementation is used, each skeleton is associated with a MDFG. That graph implements the parallelism exploitation pattern embedded in the skeleton. As an example, a pipeline skeleton $\text{pipe}(S1, S2)$ will be associated with a two instruction MDFG: the first instruction, with a single output token computes the function denoted by $S1$ and delivers the result to the second instruction. This latter instruction has again a single input token and computes the function denoted by $S2$. In case S_i is actually a sequential skeleton $\text{seq}(C_i)$, the wrapped sequential code C_i will be used to compute the function of the corresponding MDFI. In case S_i is represented by another skeleton (composition), then recursively the instruction computing S_i will be recursively substituted by the graph associated to the skeleton (composition) denoted by S_i .

This process actually defines a formal recursive procedure translating a skeleton tree into a MDFG:

- **translate(Seq(Code))** returns a single MDFI computing the function defined in terms of **Code**. The instruction has as many input tokens as the input parameters of the function defined by **Code**, and has many destinations as the output parameters (results) defined by **Code**
- **translate(Skeleton(Param1,...,ParamN))** returns a graph G including as sub graphs (one or more instances of) the graphs generated by **translate(Param1)** to **translate(ParamN)** properly arranged according to the semantics of **Skeleton** (see Fig. 7.5 and Fig. 7.6)

Once the translation of a skeleton tree to a MDFG has been defined, we can define how a skeleton application may be executed:

- the application is compiled with the **translate** function defined above and a MDFG G is obtained, then *concurrently*:
 - *Input data management* For each input data item appearing on the input (stream) of the skeleton application an input token assignment is derived and a copy of the graph G is instantiated with that assignment. Then a “fresh”³⁴ graph identifier gid is computed and all the instructions in G are modified:
 - * their identifier id is substituted with the pair gid, id
 - * all destination $\langle id, pos \rangle$ in all instructions are substituted by $\langle gid, id, pos \rangle$

The resulting graph G' is inserted into a *MDFG repository*.

- *Computation* The MDFG repository is scanned and the fireable instructions in the repository are concurrently executed by means of a *concurrent macro data flow interpreter*. Results computed by the fireable MDFI are either placed in the proper tokens of the MDFG repository MDFIs or—in case of special “output” destinations stored in *dest* fields of a MDFI—passed to the output data management.
- *Output data management* The final results computed by the concurrent MDF interpreter are delivered to application output—screen, disk file, etc.

This way of implementing skeleton frameworks has a clear advantage with respect to the template based one. The process of implementing skeleton programs is split in two parts: first, we devise efficient ways of compiling skeletons to MDFGs. Then we provide an efficient way of executing instantiated MDFGs on the target architecture at hand. As a consequence:

- The two steps may be separately optimized.
- Porting the skeleton framework to a different target architecture only requires porting the concurrent MDF interpreter.
- Introduction of new skeletons only require the definition of the proper translation of the skeleton to MDFGs and it can be easily integrated in an existing framework by properly extending³⁵ **translate** function.

³⁴never user before

³⁵i.e. adding a case clause in the **translate** implementation switch

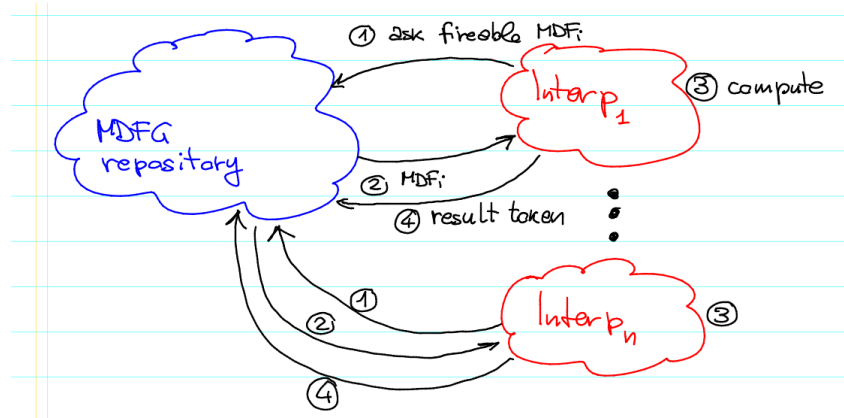


Figure 7.7 Concurrent MDF interpreter logical design

7.2.2.1 Concurrent MDF interpreter Clearly, the critical component of the skeleton framework implemented using MDF is the concurrent MDF interpreter. In order to be effective:

- The interpreter must include a suitable number of concurrent activities (*concurrent interpreter instances*), each being able to “interpret” (that is to compute) any of the MDFI appearing in the MDFG repository. This number represents the parallelism degree of the MDF interpreter and *de facto* establishes a bound for the maximum parallelism degree in the execution of the skeleton application.
- Each concurrent interpreter instance must be able to access the logically global, shared MDFG repository both to get fireable MDFI to compute and to store the computed output tokens.
- The logically global and shared MDFG repository must be implemented in such a way it does not constitute a bottleneck for the whole system. When a reasonably small number of concurrent interpreter instances is used, the implementation of the MDFG repository may be kept centralized. When larger numbers of concurrent interpreter instances are used, the MDFG repository itself must be implemented as a collection of concurrent and cooperating sub repositories.

The logical architecture of the concurrent MDF interpreter is outlined in Fig. 7.7.

7.2.2.2 Optimizations The efficient implementation of a distributed/concurrent (macro) data flow interpreter is as challenging as the efficient implementation of a template based skeleton framework. Several different optimizations may be introduced, aimed at improving the efficiency of the interpreter:

Scheduling optimizations Instructions scheduled on remote nodes include the input tokens to be processed. In case of large tokens, scheduling of MDFI to the concurrent interpreters may include special output tokens (dest) flags, telling the concurrent interpreter to maintain a copy of the result token and to send a unique *result identifier* as a result token instead of (or in addition to) sending the data back for proper delivery to the next MDFI input token. When an instruction is scheduled with this result in the list of its input tokens, it is preferably scheduled on the concurrent interpreter instance already having the input token data and therefore the communication overhead is reduced. In case of frameworks targeting shared memory architectures, such

as SMP multicores, the same technique may be used to direct computations to cores already having in their caches a valid copy of the input tokens³⁶.

MDFG linear fragment scheduling In case a linear chain of MDFI is present in a MDFG, once the first MDFI of the chain has been scheduled on concurrent interpreter i , all the subsequent instruction of the chain are also scheduled to the same concurrent interpreter. This allows copies of output tokens to the MDFG repository and subsequent copies of input tokens from the repository to the concurrent interpreter to be avoided.

7.2.3 Templates vs. macro data flow

Template and macro data flow implementation of skeleton frameworks both show pros and cons:

- Template based implementations achieve better performances when computing tasks with constant execution time, as the template based implementation has less run time overhead with respect to the macro data flow based implementations. On the other side, computations with tasks with highly variable computing times are more efficient on macro data flow based implementations as these implementations sport better³⁷ load balancing properties with respect to template based implementations.
- Porting of macro data flow based frameworks to new target architectures is easier than porting of template based implementations. In fact, MDF implementation porting only requires the development of a new MDF concurrent interpreter, whereas porting of template based implementations requires a complete rewriting of the skeleton template library, including performance models of the templates.
- Template based implementation are more suited to support static optimization techniques, such as those rewriting skeleton trees, as a substantial compile phase is anyway needed to implement skeletons via templates. The very same optimizations in MDF based implementations require some additional effort due to the fact the run time has actually no more knowledge of the skeleton structure of the program it is executing: it just sees MDFI from a MDFG.

7.3 COMPONENT BASED SKELETON FRAMEWORKS

Algorithmic skeletons may obviously be interpreted as building blocks for parallel applications. Therefore it is natural to try to migrate the algorithmic skeleton concept to the software component scenario.

Software component are usually intended as black boxes with a well defined interface defined in terms of provide ports—ports providing some kind of service—and use ports—ports declaring some kind of needed, external service. Each software component has associated—mostly informally—a semantics, defining the kind of services provided by provide ports in terms of the services used through the use ports. A framework supporting software components supports application development via composition of system-defined as well as user-defined components. The user picks-up the components modelling the blocks of his application and eventually links those components in a component “assembly” (a graph of components, in the software component jargon) by properly pairing provide and use ports.

³⁶in both cases, we refer this technique as *affinity scheduling*

³⁷an also achieved more easily

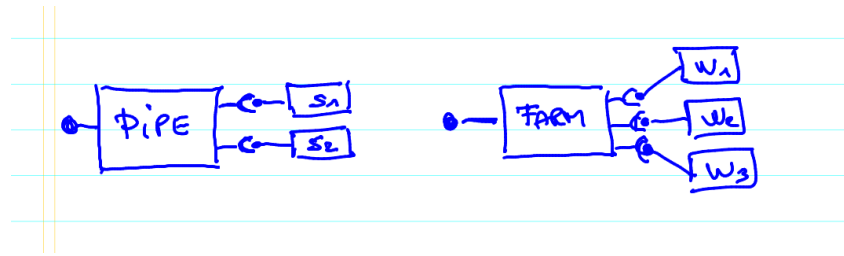


Figure 7.8 Sample skeleton components (not composite)

Any algorithmic skeleton can therefore be viewed as a *parallel component providing* as a service the higher order function modelled by the skeleton and *using* other the services provided by components modelling its skeleton parameters.

The simpler skeleton component is clearly the one wrapping some legacy code. It only has a provide port. Through this port, a computation of the wrapped code onto some kind of input parameters may be asked. The computation happens sequentially. Then other skeleton components may be defined. As an example, a pipeline component will have one provide port and two use ports, as depicted in Fig. 7.8 (left). Or a farm component may be defined with one provides port and a number of use ports, each interfacing the farm component with a worker component, as depicted in Fig. 7.8 (right).

These components modelling algorithmic skeletons may be used as other more classical components within the software component framework, provided the component framework supports compositionality. In this case, the composite component pipeline is perceived as a monolithic component from any other component invoking the services provided by its provide ports. Only the system programmer implementing the skeleton component actually perceives the structured internals of the skeleton components.

Skeleton frameworks have rarely been provided as software component frameworks, with a notable exception: GMC Behavioural Skeletons (GCM/BS see Sec. 10.1). GCM stands for Grid Component Model and it is a component model specially designed to support grid computing within the CoreGRID project³⁸

GCM/BS exploited all the positive features of a software component framework to provide the programmer of parallel component applications with skeleton composite components (the behavioural skeletons). These components not only implemented all the parallelism exploitation details related to the modelled skeleton in way that turns out to be completely transparent to the component application programmer, but also included advanced features that allowed non functional concerns such as performance tuning and optimizations to be completely managed by the skeleton component, without any kind of support/intervention required to the component application programmer.

³⁸CoreGRID is an EU funded Network of Excellence (NoE) that ran in the years from 2005 to 2008 (see www.coregrid.net). GCM has been developed within the Programming model Institute of CoreGRID. The reference implementation of GCM has been developed within the GridCOMP EU funded project (gridcomp.ercim.org) on top of the ProActive middleware.

CHAPTER 8

PORTABILITY

Portability has different meanings at different abstraction levels. *Application portability* refers to the possibility to port an existing application from a machine to another, possibly different, machine. Portability is fully achieved in this case, when the application may be run unchanged on the second architecture once it has been designed, developed and debugged for the first architecture. Application portability may be achieved different ways, including:

- *recompiling* of the source code, or
- simply *re-running* the original compiled code.

In the latter case, the two machines targeted will obviously be *binary compatible*, that is they will accept the same kind of object code files.

When considering programming frameworks, instead, *portability of the programming framework* across different architectures is fully achieved when the programming framework running on machine *A* may be moved and operated on machine *B*. This in turn, implies that application portability will be achieved, for all those applications running on top of our programming framework.

As for application portability, programming framework portability may be achieved in different ways:

- The easiest one is portability through *recompiling*. We take the source code of our programming framework, written in a language L_x and we recompile the whole source code of the programming environment on the second machine, provided a compiler for L_x is available on the second architecture.
- A more complex way of porting a programming environment requires both recompilation and rewriting of some parts of the programming framework implementation.

In particular, all those parts that were specifically implemented for solving problems peculiar of architecture *A* and whose solutions do not apply to architecture *B*.

- In the worst case, portability could simply not be achieved, as the features of the target architecture *B* are radically different from the features of target architecture *A*. In this case, most of the programming environment parts must be re-written almost from scratch. Probably, only the higher level tools, such as the parsers, may be preserved unmodified.

Portability of programs is an issue typical of parallel programming. In the sequential programming world it has a minor impact. The problem is mostly determined by the different features sported by different parallel and distributed architectures. These differences may concern the interconnection network between parallel/distributed processing elements (full interconnection, regular interconnection topology networks, etc.), the presence or the absence of some level of sharing in the memory hierarchies of the processing elements, the kind of accelerators available (e.g. GP-GPUs or FPGAs). This is not the case, or at least, the differences between different architectures are not so evident, when sequential architectures are considered. Most of the sequential architectures around³⁹ support very similar assembly code sets and as a result, the only significant differences between different architectures consist in the presence of particular functional units (e.g. SIMD units) speeding up data parallel computations. As a consequence, porting an application from architecture *A* to architecture *B* in most cases only requires a re-compilation of the source code of the application and, in a smaller but consistent number of cases, the object code of the application compiled on machine *A* may be simply moved and run to machine *B* without any kind of (or with minor, performance related) issues.

We are ready now to introduce a distinction between two different kinds of portability:

Functional portability Functional portability concerns the portability of the functionality of the application or programming framework. Functional portability of an image filtering application means that we can move/recompile the application to the new machine preserving the possibility to filter images obtaining the same set of filtered images we previously get from the application running on the first architecture.

Performance portability In addition to functional portability, we may be interested in portability of non functional properties of our applications/programming environments. Non functional features are those features that do not concur directly in the result computed by our applications (the function of the input data) but rather concur to *the way* these results are computed. As an example, performance, security, power management, fault tolerance are all non functional features of applications/programming environments. Performance portability guarantees that a program with a given performance on machine *A* may be ported to machine *B* preserving a comparable performance. The term “comparable”, in this case, may be considered modulo the architectural differences between the two architectures. In other words, performance portability must guarantee that the best performance possible for the algorithms used relatively to the features of the target architecture are eventually achieved.

Now when considering performance portability in addition to functional portability new issues must be taken into account. Simple recompilation of an application on a new target architecture can give you functional portability but may be it does not guarantee performance portability.

³⁹it is worth pointing out that these architectures actually do not exist anymore as single chips, but we refer here to usage of a single core as a sequential processing element

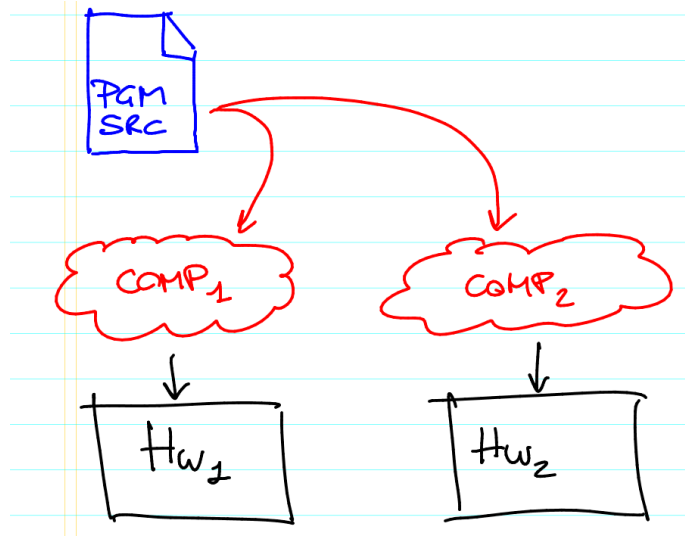


Figure 8.1 Portability through recompiling

In the sequential programming world, we are used to get performance portability for free. If you write a program for a PowerPC and then you recompile the very same code on an Intel processor, you'll get performance portability in addition to functional portability.

But in the parallel programming world things are a little bit different. If you write an MPI program using barriers and you run it on a machine with hardware support for broadcast⁴⁰, in case you move that program to a machine without the hardware support for broadcast, you will get definitely worst performances.

In the following sections we will discuss issues and solutions relative to both functional and performance portability of skeleton based programming frameworks. We distinguish two main approaches to portability, namely i) portability achieved through re-compiling and ii) portability supported through virtual machines.

8.1 PORTABILITY THROUGH RE-COMPILING

Portability through re-compiling relies on the existence of a set of compiling tools that may be used to re-compile the whole programming framework implementation on the new target machine.

8.1.1 Functional portability

In order to be able to functionally port a skeleton framework from a target architecture to another target architecture, what has to be guaranteed is that the programs running on the first architecture also run on the second one and compute the same results.

Let us consider what happens of functional portability when using template or macro data flow based implementations of skeleton programming frameworks.

Template based frameworks. In a template based skeleton framework, implementation of skeletons is achieved by instantiating process templates. Provided that the new target architecture:

⁴⁰as an example, on a machine whose cores/nodes are interconnected using a mesh network plus a global bus for synchronizations

- supports the language used to implement the framework,
- supports the same kind of concurrent activity abstractions (e.g. processes, threads or kernels) used to implement the templates on the first architecture, and
- supports the communication and synchronization mechanisms used in the original process templates

then functional portability through re-compiling may be achieved.

The first requisite above is obvious. If you can't compile just a single component of the programming framework because no compiler exists on the new target architecture for the language used to write that component, the whole porting fails. Such a situation requires re-writing the component in another language for which a compiler exists, instead.

The second and third requisite could be handled in a different way. If some base mechanisms essential for framework implementation is not present, some new component must be designed, developed and integrated into the ported framework capable of emulating the missing mechanisms. Alternatively, the templates using the missing mechanisms must be completely rewritten in terms of the mechanisms presents on the new target architecture.

Macro data flow based frameworks. In a macro data flow based skeleton programming framework, functional portability may be achieved if

- a compiler for the language used to implement the framework is available,
- the mechanisms used to implement the macro data flow interpreter are supported on the new target architecture.

Therefore, as far as functional portability is concerned, we can conclude functional portability may be guaranteed if template implementation or concurrent MDF interpreter implementation may be conveniently ported onto the new architecture in the two cases. As a macro data flow concurrent interpreter will be in general modeled by a single parallelism exploitation pattern, while the templates in a template library model more parallelism exploitation patterns, we can also conclude that functional portability is slightly easier in MDF based skeleton frameworks with respect to template based frameworks

8.1.2 Performance portability

When dealing with performance portability, things are a little bit more complex.

Template based frameworks. In a template based skeleton framework performance portability is mainly achieved through proper restructuring of the templates used to implement the skeletons and the skeleton compositions. The general structure of the programming framework tools could be mostly left unmodified provided that new templates, or modified/adapted versions of the existing ones, are adopted for the new architecture targeted. As an example, let us consider the “farm template for complete interconnection architectures” of Sec. 7.2.1.4. In case we move to target an architecture with mesh interconnection of internal cores such as the Tilera multi-core, we must take into account the farm templates such as those discussed later on in the same Section (“farm templates for mesh architectures”). This was not necessary in case functional portability is only considered. In that case, we could have set up further communication library emulating the full interconnection required by the first template on a mesh architecture. This introduces more overhead in the implementation, but as we are only interested in functional portability, it is sufficient to guarantee portability.

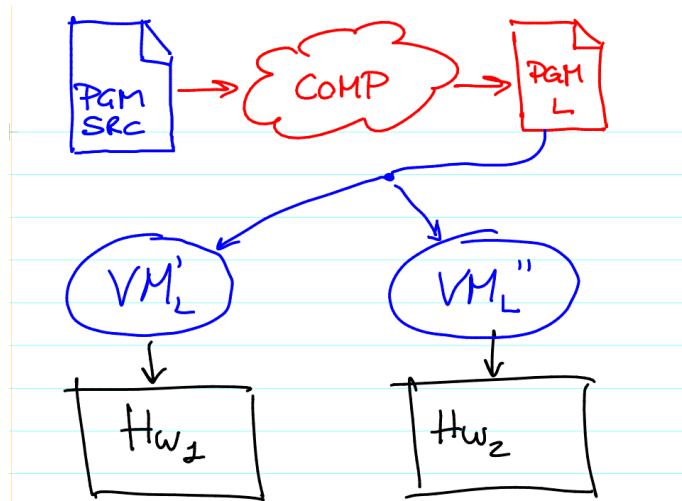


Figure 8.2 Portability through virtual machines

It has to be taken into account that depending on the new architecture features the design of a new template or the re-adaptation of an existing template for any one of the skeletons provided by the framework may require a huge design and implementation effort.

MDF based frameworks. In this case performance portability is mainly guaranteed by an efficient porting of the concurrent MDF interpreter. As again stated in Sec. 7.2.2, the efficient implementation of a concurrent MDF interpreter does require a substantial effort. However, once this effort has been spent, the whole framework may be ported seamlessly to the new architecture, as the features of the MDF graphs generated when compiling the skeletons will not impact performances on the new machine if the concurrent MDF interpreter has been successfully “performance” ported.

Therefore, in case of performance portability, we can conclude that it is slightly easier to achieve it in case of template based frameworks, as the general task is split into smaller tasks (the implementation of the single templates) rather than being kept unique (the implementation of an efficient concurrent MDF interpreter).

8.2 PORTABILITY THROUGH VIRTUAL MACHINES

Up to now, we have only considered portability assuming our programming frameworks are recompiled when moving to a new target architecture. There is an alternative way of implementing portability, however, which exploits virtual machines.

Let us assume that a skeleton programming framework exists targeting the Java Virtual Machine. The framework itself may be a library based or a new language eventually compiled down to Java, template based or macro data flow based. It does not matter. The important thing is that the skeleton applications eventually run as JVM byte-code. Alternatively, let us assume that a skeleton programming framework exists targeting C+MPI. Again it does not matter how the framework is implemented nor the programming model (library or new language) it exposes to the skeleton application programmer. The important thing is that the skeletons applications are eventually run as MPI applications.

In both cases, the functional portability of the programming frameworks may be simply guaranteed by the portability of JVM or MPI. What you need to port your programming

environment on a new target architecture is an implementation of the JVM or an implementation of MPI on the new target architecture.

What about performance portability, then? In general performance portability could not be guaranteed for the very same reasons listed when discussing compiler based framework portability. Virtual machine only guarantees portability and efficiency of the virtual machine mechanisms. The efficiency achieved on a single architecture is the “best” available for those mechanisms. However, this does not guarantee that the final framework efficiency is convenient. Again, let us consider porting of a programming framework from a machine supporting hardware broadcast and multi-cast mechanisms to a machine that does not support in hardware such mechanisms. Any template⁴¹ relying on the efficiency of broadcast/multi-cast mechanisms will suffer from severe performance penalties on the second machine. As a consequence, true performance portability will be achieved only by rewriting the templates⁴² in such a way they properly exploit the mechanisms available on the new target architecture.

8.3 HETEROGENEOUS ARCHITECTURE TARGETING

Targeting heterogeneous architectures poses problems similar to those arising when porting programming environments across different architectures. An heterogeneous architecture is such that at least two different kind of processing elements exist, that can be used to execute parallel/distributed activities. Sample heterogeneous architectures include multi-core CPUs with attached GPUs as well as interconnections⁴³ of different multi-core processing elements, e.g. a mix of Intel Core and IBM PowerPC processors interconnected via Ethernet.

Heterogeneity poses several problems, that partially overlap to those analyzed when dealing with portability issues:

- *Specialized compilers* must be used to target different nodes in the heterogeneous architecture. This means that the very same process template must be compiled with different tools depending on the choices made in the mapping phase, that is when “processes” are bound to processing elements.
- *Specialized templates* the templates used to compile/implement skeletons may differ for different processing elements. As an example, a map template used on plain multi-cores may radically differ from the one used on CPUs with an attached GPU. In this latter case, data parallel kernel executions may be delegated to the GPU, achieving a sensible speedup.
- *Specialized concurrent MDF interpreter instances* must be available to be run on the different processing elements used to implement the overall MDF interpreter.

In addition to requiring specialized compiler and templates, however, heterogeneity also raises some problems in the high level part of the compiling tools relative to template based programming frameworks. In particular, when templates are chosen to be associated to the skeletons appearing onto the application skeleton tree (see Sec. 7.2.1) decisions have to be made relatively to the possibility to choose templates that only work on a given subset of the target architecture. As template assignment has eventually direct consequences on the final performance of the application, this is a critical point.

E.G.▷ Consider the case of architectures build of the interconnection of multi-core nodes, where only a part of the multi-core nodes are equipped with GPUs. Let us assume the application to implement has a skeleton tree such as the following:

⁴¹but this is true also taking into account the implementation of the concurrent MDF interpreter

⁴²or the concurrent MDF interpreter

⁴³by means of standard interconnections networks

Pipe (Map (Seq (C1)) , Map (Seq (C2)))

Let us suppose we have available map templates both for multicore only and for multi-core+GPU processing elements interconnections. Whether or not the GPUs are more efficient of CPUs on a given computation could not be decided without actually looking at the input data of the map skeletons. Regular data⁴⁴ will probably better perform on GPUs than multi-cores. Furthermore, in case of unbalance of the pipeline stages, it may be the case that one of the stages does not need an extreme parallelization, as the other stage is actually the pipeline slowest stage and therefore it is the other stage that eventually determines the performance of the whole pipeline.

Now when compiling this program on the heterogeneous architecture, we have basically two possibilities related to the template choice:

- We proceed as usual, looking at the performance models of the templates in order to establish which template composition will best implement the skeleton tree at hand. In other words, we completely implement the template assignment during the first phases of the process leading to the generation of the skeleton application object code. Or alternatively
- We perform a preliminary template assignment and later on, at *run time*, we possibly change the choice made in such a way we succeed to exploit the behaviour data gathered while executing the program⁴⁵. This implements a kind of *self*-management of the template assignment process. Further details on this will be given in Sec. 10.1.

■

A different set of problems arise when targeting heterogeneous architectures in a macro data flow based framework. In this case, the compilation of the skeleton application to MDF graphs should not be changed. What changes is the implementation of the concurrent macro data flow interpreter. In particular:

- Different interpreter instances have to be developed, such that each instance is optimized for one of the heterogeneous processing elements. It is possible that different (heterogeneous) processing elements may run the same kind of interpreter instance (e.g. processing elements with slightly different processors). In the general case, different instances of the interpreter may support the execution of different subsets of MDF instructions: as an example, GPU powered interpreter instances may support the execution of data parallel macro data flow instructions, while more classical, non GPU powered architectures only support such instructions through execution of explicitly parallel MDF graphs (that is, not in the execution of a single MDF instruction).
- The scheduling of fireable MDF instructions to the concurrent interpreter instances has to be consequently modified. Being functionally different instances of the interpreter running on different heterogeneous nodes, specialized scheduling functions must be implemented as the set of remote interpreters is not anymore “anonymous”.
- In turn, the specialization of the scheduling policies to accomplish the diversity among available concurrent interpreters may require a more complex management of the logically centralized MDF graph repository. In case of distributed implementation of the repository with load balancing guaranteed through fireable instruction stealing, also the stealing strategies must be adapted in such a way the existence of different interpreter instances is taken into account.

⁴⁴that is data that lead to identical executions in the subcomputations of the map

⁴⁵such data may make evident that the GPU template was the best one or vice versa, as an example

8.4 DISTRIBUTED VS MULTI-CORE ARCHITECTURE TARGETING

Historically, skeleton based programming frameworks have been developed to target distributed architectures⁴⁶. The skeletons frameworks Muesli, Muskel, ASSIST, SkeTo (just to name those still maintained) have all been designed and developed to target clusters and network of workstations. Later on, some of these frameworks have been redesigned and improved in such a way multi core architectures could also be exploited. At the moment being, Muesli [17], Muskel [8] and Skandium [38] all support multicore machine targeting.

Multi core targeting is basically supported in two distinct ways:

Architecture modeling changes We assume that each multi core node in the target COW/NOW architecture is itself a network of processing elements. As a consequence, a number of processes—from the template composition implementing the skeleton program—or of concurrent interpreters—in case a MDF implementation is used—equal to the number of supported cores is placed on the single multi core.

This implies slight modifications in the skeleton framework implementation. Basically only requires to expose different cardinalities for the processing elements hosted in the target architecture, in such a way multiple cores are counted as multiple processing elements.

Of course, in case the templates used may optimize communications depending on the features of the links interconnecting nodes, the links between “virtual processing elements” modeling cores of the same processor should be modeled as “fast” links⁴⁷.

Template/MDF interpreted specialization Alternatively, and depending on the kind of implementation adopted in the framework, either the implementation templates of the skeletons, or the structure of the concurrent MDF interpreter instance should be changed to take into account the existence of multi core nodes in the target architecture.

When considering a *single* multi core as the target architecture of our skeleton framework, similar choices may be taken, but in general, better optimization can be achieved using the “specialization” strategy rather than the “architecture modeling” one.

In any case, the design of the templates or the design of the MDF interpreter must tackle problems that are quite different from the ones significant when COW/NOW architectures are targeted, as reported in Table 8.1.

In the following sections we briefly discuss the issues related to COW/NOW and multi-/many core targeting.

8.4.1 Template/MDF interpreter implementation

On distributed architectures, the main mechanism exploited to implement concurrent activities is the *process* abstraction. Processes are used to implement concurrent activities in templates as well as those related to MDF interpreter implementation. As a consequence, *no memory is shared* among the different concurrent entities used to implement the skeleton

⁴⁶We refer in this section as “distributed architectures” those resulting from the interconnection through some kind of interconnection network of complete processing elements, such as the cluster of workstations, the network of workstations and the computational grids. We refer with the term “multi/many core” those architectures build out of multiple cores, possibly on the same chip, sharing some level of the memory hierarchy, such as Intel Core processor or symmetric multi processors (SMP) such as the shared memory machines hosting different multi core chips on several distinct boards.

⁴⁷that is links using specialized inter-thread communication mechanisms rather than slower inter-process mechanisms

	Distributed	Multi/many cores
Templates or MDF interpreter	Process based	Thread based
Communication & synchronization	Inter PE (TCP/IP)	Inter Thread (Shared mem + locks & semaphores)
Exploiting locality	Process mapping	Thread pinning Memory access
Optimization opportunities	Communications Process mapping Communication grouping	Shared memory accesses Thread pinning Memory layout (padding)

Table 8.1 Problems related to template/MDF interpreter implementation for distributed/shared memory architectures

framework. All the interactions among the concurrent activities must be implemented via explicit communications.

When targeting multi cores, instead, the main mechanisms exploited to implement concurrent activities is the *thread* abstraction. As a consequence, interactions among different concurrent activities may be implemented through shared memory *and/or* via explicit communications as in the case of concurrent activities implemented through processes.

Using processes may seem a little bit more complicate than using threads, due to the fact no shared memory abstraction is present and all the interactions must be implemented via explicit communications. However, while threads support “more natural” interactions via shared memory, they also support default sharing of *all* the memory space. Therefore it is easy to introduce errors and inefficiencies⁴⁸.

E.G.▷ A simple example will better illustrate the case. When implementing a map template using threads the naive implementation of accesses to the partitions of the original, collective data may introduce false sharing that could impair the actual speedup achieved in the execution of the map template. In particular, a map template with threads operating onto sub items of a vector whose items are of size smaller than the cache line size implies the arising of a false sharing situation. Items i and $i + 1$ of the vector will be processed by different threads but they will actually be allocated to the same cache block. As a consequence, two accesses updating positions i and $i + 1$ at different cores will imply the start of a cache coherency protocol which is actually useless, due to the fact that thread processing position i will never need accessing position $i + 1$ and vice versa⁴⁹. ■

8.4.2 Communication & synchronization

Communications and synchronizations represent a crucial matter in parallel and distributed programming. Therefore they play a fundamental role in both template and MDF based skeleton framework implementation.

When targeting distributed architectures, the main communication mechanisms are those provided by the interconnection network used to host the target architecture processing elements. In common COW/NOW architectures, either Ethernet or some kind of more advanced interconnections such as Myrinet [46] or Infiniband [31] are used. When using

⁴⁸in this sense, inefficiencies can be seen as particular kind of “non functional errors

⁴⁹after the termination of the map, of course, this may happen, but not within the execution of the map. Within the map thread processing item i may at most require the access to current (not the new one) value of position $i + 1$, which does not require cache coherency.

standard Ethernet, the TCP/IP protocol stack is used for communications, either at the level of transport protocols (TCP, UDP) or at the network level (IP). Streams and datagrams are used to implement both communications and synchronizations. Synchronizations are usually implemented via small messages⁵⁰. When using more advanced interconnection networks, proprietary communication and synchronization mechanisms are used instead. TCP/IP protocol stack is also usually provided on top of these basic, proprietary mechanism, often resulting in quite large overhead when using it to implement communications or synchronizations⁵¹. It is worth pointing out that both Ethernet and the more modern and performant interconnection networks support some kind of primitive broadcast or multi cast communications. These mechanisms are fundamental for the design of both efficient templates and optimized concurrent MDF interpreters. As an example, in the former case, they can be used to support data distribution in data parallel templates, while in the latter case, they can be used to support load balancing mechanisms in distributed MDF graph pool implementation as well as different coordination mechanisms supporting the distributed/parallel implementation of the interpreter, such as fireable instruction stealing or fault tolerance discovery mechanisms.

When targeting multi/many core architectures, instead, accesses to shared memory are normally used to achieve an efficient implementation of both communications and synchronizations. Communication is implemented writing and reading to properly synchronized buffers in shared memory. Synchronization, including the one needed to implement the communication buffers is implemented using locks, semaphores or any other synchronization primitive available. In this case, it is fundamental that very efficient synchronization mechanisms are used, due to huge amount of cases where this mechanisms are used and to the relatively *fine grain*⁵² computations involved in many cases in the synchronization operations. Some skeleton framework tackle this issue since the very beginning of the framework design. Fastflow is entirely build on a very fast and efficient single producer single consumer queue which has been demonstrated to be correct while being lock free and fence free on common cache coherent multi cores such as the ones currently provided by Intel and AMD. The implementation through shared memory of communication and synchronization primitives naturally supports the implementation of the collective broadcast/multi cast operations.

8.4.3 Exploiting locality

Locality is the property of algorithms of performing “near” data accesses during program execution. In other words, very often algorithms are such that if a data structure or a portion of code is accessed at time t_i the probability of another access to the data/code in the near future (time $t_{i+\Delta}$) is very high (temporal locality) as well as the probability of accessing locations close⁵³ to the ones just accessed (spatial locality). Locality is the property that makes current architectures work. Without locality, caches will be useless. Without locality most of the memory access⁵⁴ would be in main memory, whose access times are order of magnitude larger than the access times of caches. Ensuring locality exploitation is therefore fundamental for efficient implementation of algorithms and, therefore, of sequential programs.

⁵⁰messages hosting a very small payload, may be just one byte, which is not significant as payload as the send and receive of the message have associated the synchronization semantics.

⁵¹Windows support for Infiniband provides a TCP/IP stack implementation on top of Infiniband. The stack introduces a sensible overhead with respect to the one introduced by primitive Infiniband mechanisms.

⁵²that is small amount of computation in between successive synchronization operations

⁵³whose address is not too different

⁵⁴instruction fetches *and* data accesses

Locality is even more important in parallel programming. Considering the parallel/distributed/concurrent activities usually are performed on processing elements that are relatively far from each other—in terms of the time spent to exchange data—it is clear that a program not exploiting locality will never be efficient.

When considering process based implementations of skeleton frameworks this is particularly evident. If we design a template where data is continuously moved to and from different processing elements because of poor locality optimization, this program will spend a lot of time moving data rather than actually computing results.

E.G.▷ As an example, suppose we implement a pipeline template where each stage gets tasks to be computed from an emitter/collector process and delivers results to the same process. In this case to compute a k stage pipeline on a stream of m tasks we need $2km$ communications as each stage must first receive a task to compute and then send the result back to the emitter/collector process. If we use the classical “linear chain” of k processes to implement the stages, the communications needed to compute the same stream are just one half mk . In the former template, we did not exploit the locality in the access of partially computed result $f_h(f_{h-1}(\dots(x_i)\dots))$ which is only accessed by stages S_h (the producer) and S_{h+1} (the consumer). ■

However, also in case of thread based implementations, locality plays a fundamental role.

E.G.▷ When working with threads and shared memory, we can consider a different example. Let suppose we want to implement a map template working on vectors. Therefore we distribute over a thread pool the computation of

$$\forall i : y_i = f(x_i)$$

If we distribute map iterations over threads such that iteration i is placed on thread $t = i \% n_{thread}$ we actually will achieve a much worse performance than in case we distribute iterations on threads such that iteration i is scheduled on thread $t = i / n_{thread}$, as this latter distribution allows to achieve a much better cache management due to the higher locality in the memory accesses⁵⁵. ■

Therefore, when targeting both distributed COW/NOW and multi/many core architectures, preserving and exploiting the locality proper of the parallel skeletons used is a fundamental issue. Many of the “optimizations” listed in the next section are *de facto* aimed at exploiting locality in parallel computations.

8.4.4 Optimizations

Several optimizations may be taken into account when targeting (porting to) distributed/multi core architectures. Actually, being the parallel structure of our program completely exposed through the used skeletons, any known optimization technique relative to the particular patterns modeled by the skeletons may be used. As already discussed this is one of the fundamental advantages of the algorithmic skeleton approach to parallel programming.

Locality preserving/exploiting optimizations are probably the most important ones. In this context, we should mention different techniques:

- Allocation of the concurrent activities compiled from the application skeleton composition in such a way locality is preserved. This includes mapping communicating

⁵⁵as an exercise, try to imagine the trace of data addresses generated towards the caches from two different threads (running on two different cores).

processes on “near”—from the viewpoint of communication overheads—processing elements as well as pinning⁵⁶ thread to cores in such a way threads accessing the same data structures share the largest portion of memory subsystem (e.g. second level caches).

- Grouping of concurrent activities on the same processing element, in case these activities happen to be *tightly coupled*, that is they share a lot of data structures or they send each other a large amount of data. This kind of grouping decreases the parallelism degree of the application, in general, but may lead to advantages in terms of a smaller number of communications or smaller overhead in shared data access that possibly overcome the losses due to the smaller parallelism degree.

Other common optimization techniques impact the way concurrent activities interact. Communication optimizations belong to this class of optimizations, as well as shared memory access optimizations.

Communication optimizations. Communication optimizations relate to the way communication among different concurrent activities are performed. In presence of a large number of communication between two processing elements of the target architecture, for instance, *communication grouping* may be used. With this technique, a number of communications happening in the same time interval and moving data from processing element PE_i to processing element PE_j are implemented as a single communication event with a larger message body. As communications between separate processing elements usually require some setup time plus a time directly proportional to the length of the message, this technique allows $k - 1$ setup times to be spared when k communications are performed. Taking into account that communication setup may be quite large (tens of microseconds when using Ethernet) this represents a sensible advantage. Of course, this means communications must be directed to a kind of “communication manager” that analyzes the traffic in pre-defined time chunks and possibly decides to group different communications into a single one. This also implies that the latency perceived by the single process may be larger than the one achieved when not using grouping, although the performance of the overall communication system is better.

Shared memory access optimizations. Shared memory access optimizations consist in a quite different set of techniques. To avoid problems while accessing shared variables⁵⁷ variable layout in memory may be optimized as well as the usage of names to denote shared variables. Variable layout in memory may be optimized to avoid that items of the same data structure accessed by concurrent activities of the same template share the same cache line. This avoids the penalties of the cache coherence mechanisms while preserving program fairness at the expense of a (slightly) larger memory footprint. The technique mainly consists in inserting *padding data* between consecutive items (or blocks of items) of the shared data structure in such a way accesses formerly referring to the same cache line become accesses referring to different cache lines.

E.G.▷ Consider a vector of integers. Assume cache lines may host 8 integers (32 bytes). Assume also the vector is processed by a map computing $x_i = f(x_i)$ and that each one of the workers in the map get chunks of 10 vector items to be processed. Worker 1 will process two cache line of data: the first one—assuming the vector is cache aligned in memory—will be entirely read and written. In the second line, worker 1 will just access the first and second elements, while the rest of the vector items in the line will be accessed by worker 2. This situation is

⁵⁶pinning is the operation aimed at fixing thread execution on a given core

⁵⁷we assume shared memory architectures support cache coherence

a false sharing situation. When writing the new value for x_8 (the first element of the second cache line) worker 1 will probably cause the start of a cache coherency protocol aimed at propagating the new value to the cache line hosted on the processor where worker 2 has been scheduled, that in the meanwhile is processing $x_{10}, x_{11}, \dots, x_{19}$ ————— ■

Renaming of variables is used instead to allow better optimizations to be implemented in the sequential language compiler when accessing potentially shared data structures. By properly renaming variables that will eventually be accessed—for writing—by different concurrent activities due to the template/skeleton semantics we avoid the renamed variables be marked as “shared” and therefore accessed through time consuming access synchronization mechanisms.

CHAPTER 9

REWRITING TECHNIQUES

Rewriting techniques have been exploited in a variety of contexts. Functional programming languages traditionally provide the necessary concepts to support program rewriting. Let us take into account our second order modeling of skeletons introduced in Chap. 4. We defined a map skeleton. Looking at the definition of this skeleton we can easily get convinced that (being $\circ : (\alpha \rightarrow \beta) \rightarrow (\beta \rightarrow \gamma) \rightarrow (\alpha \rightarrow \gamma)$ the functional composition and “ $:$ ”: $(\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta$ the operator applying a function to some input data⁵⁸)

$$(\text{map } f) \circ (\text{map } g) \equiv \text{map}(f \circ g) \quad (9.1)$$

In fact, when the left function is applied to a data structure such as $\langle x_0, \dots, x_n \rangle$ (let us suppose the x_i here are items in a vector, as an example), we get

$$\begin{aligned} (\text{map } f) \circ (\text{map } g) : \langle x_0, \dots, x_n \rangle &= (\text{map } f) : \langle g(x_0), \dots, g(x_n) \rangle = \\ &= \langle f(g(x_0)), \dots, f(g(x_n)) \rangle \end{aligned}$$

whereas when the right function is applied to the same data structure, we get

$$\begin{aligned} \text{map}(f \circ g) : \langle x_0, \dots, x_n \rangle &= \langle (f \circ g) : x_0, \dots, (f \circ g) : x_n \rangle = \\ &= \langle f(g(x_0)), \dots, f(g(x_n)) \rangle \end{aligned}$$

which is the same result.

More rewrite rules may be proved correct when considering skeletons as introduced in this book. As an example, assuming we denote with Δ skeletons or skeleton compositions, we may simply verify that, for any skeleton Δ_i

$$\text{farm}(\Delta_i) \equiv \Delta_i \quad (9.2)$$

⁵⁸the first “ $:$ ” is the apply operator, the second one (without quotes) is the usual “type” operator

The *farm* skeleton, in fact, does only impact the parallelism degree used to compute a given program, not the functional semantics of the program computed through the *farm*. For the very same reason, we can state that

$$\text{pipe}(\Delta_1, \Delta_2) \equiv \text{pipe}(\text{farm}(\Delta_1), \text{farm}(\Delta_2)) \quad (9.3)$$

By further rewriting the last expression, using the 9.2 rule, we get another interesting example of skeleton tree rewriting, namely:

$$\text{farm}(\text{pipe}(\Delta_1, \Delta_2)) \equiv \text{pipe}(\text{farm}(\Delta_1), \text{farm}(\Delta_2)) \quad (9.4)$$

It is worth to point out that rewriting rules 9.1 and 9.2 are *primitives* while the rewriting rule 9.3 and 9.4 are clearly derived applying rewriting rule 9.2.

The importance of rewriting rules such as 9.1 and 9.2 is mainly related to non functional behaviour of your skeleton program.

E.G.▷ Let us assume a programmer has written a program, whose parallel structure is modeled by the skeleton tree

$$\text{pipe}(\text{Seq}(C_1), \text{Seq}(C_2))$$

and let us suppose the monitored performance values state that $T_{S_{C_1}} = t_1 < t_2 = T_{S_{C_2}}$. Then an easy way of improving the program “non functional” behaviour—the performance—is to apply rule 9.2 and transform the program as

$$\text{pipe}(\text{Seq}(C_1), \text{farm}(\text{Seq}(C_2)))$$

Mostly likely, this could require some minor change in the source code of the skeleton program but has a notable impact on program performance eventually achieved on the target architecture⁵⁹ ■

The rewriting rules just discussed should be read also in a more structured way, depending on the target architecture considered.

E.G.▷ Consider rule 9.1 and assume we want to implement a program operating on a stream of input tasks by applying to each one of the tasks a data parallel skeleton, as an example through the skeleton program

$$\text{pipe}(\text{map}(\Delta_1), \text{map}(\Delta_2))$$

If we assume:

1. that a “sequential composition” skeleton exist, $\text{comp}(\Delta_1, \Delta_2)$ performing in sequence, on the same processing elements, the computations of its parameter skeletons Δ_1 and Δ_2 , and
2. that a single stage *pipe* may be used, representing the fact the computation modeled by the single parameter skeleton of the pipe is to be performed on a stream of input data,

then we can state that

$$\text{pipe}(\text{map}(\Delta_1), \text{map}(\Delta_2)) = \text{pipe}(\text{map}(\text{comp}(\Delta_1), (\Delta_2))) \quad (9.5)$$

Now, either left hand side or right hand side expressions may be used to implement the same computation. What are consequences on the non functional features of our skeleton program?

⁵⁹We are assuming here that the inter arrival time of tasks to the pipeline is small enough in such a way it does not constitute a bottleneck and that the *farm* implementation is feasible, that is, the time spent to schedule tasks to the *farm* workers and to gather from these workers results is not larger than the time spent to compute the single result. If either of these assumptions does not hold, than the rewriting is useless, of course.

- If left hand side expression is used:
 - * Computation of $\text{map}(\Delta_1)$ and $\text{map}(\Delta_2)$ happen concurrently on different items of the input stream. As a consequence, we expect a better service time, being it expressed as the maximum of the service times of the two maps rather than being computed as the sum of the two
 - * More communication are involved. We have to distribute the input tasks to the workers of the first map, than to distributed the results of the first map to the workers of the second map and eventually to collect the final result from the workers of this second map.
- If the right hand side expression is used:
 - * The computation of each input data item happens serially in a single map, therefore the latency (and the service time) is higher, but
 - * Less communication overhead is needed to implement the parallel computation (we omit the communications in between workers of the first map and second map, that in this case happen to be the same).

All in all we can conclude that the left hand side program is *finer grain* than the right hand side. Depending on the target architecture at hand *and* on the features of the surrounding skeletons (if any), one of these two equivalent programs may result “better” than the other one. ■

9.1 SKELETON NORMAL FORM

An important consequence of the rewriting rules discussed in the previous section is the “normal form” concept for skeleton programs developed in late '90.

When considering a skeleton rewriting system, such as the one defined by rewriting rules listed in the previous section, we may be interested in defining the “better” rewriting—with respect to some non functional feature of our program—possible of any given program Δ .

This “better” rewriting is called “normal form” to represent the fact that this is the alternative⁶⁰ program representation that would normally, that is in absence of special situations, considered for the implementation.

A normal form for a small subset of stream parallel skeletons has been defined in [5]. The skeleton framework considered only includes pipeline, farm, sequential composition and sequential wrapping skeletons, denoted, respectively, by `pipe`, `farm`, `comp` and `seq`, as usual.

The rewriting rules considered basically only include farm elimination/introduction, which is the rewriting rule 9.2 above.

Authors also consider a simple performance modeling for the skeletons included in the framework. Their performance modeling states that

$$\begin{aligned}
 T_S(\text{pipe}(\Delta_1, \Delta_2)) &= \max\{T_S(\Delta_1), T_S(\Delta_2)\} \\
 T_S(\text{farm}(\Delta)) &= \max\{T_{\text{emitter}}, T_{\text{collector}}, \frac{T_{\text{worker}}}{n_w}\} \\
 T_S(\text{comp}(\Delta_1, \Delta_2)) &= L(\Delta_1) + L(\Delta_2) \\
 T_S(\text{seq}(C)) &= L(C)
 \end{aligned}$$

with $L(x)$ being the latency of x and T_{emitter} , $T_{\text{collector}}$ and T_{worker} being the service times of farm emitter, collector and workers processes.

Under these assumptions, the “normal form” of any skeleton program Δ is given as follows:

⁶⁰with respect to the rewriting system considered

- the set of the “leaves” of the skeleton tree Δ is computed, as they appear left to right in the skeleton tree. This set, $C_1, \dots, C_n$ is the set of the sequential wrapping skeletons appearing in the skeleton program. This set is called *fringe* and it can be recursively defined on the structure of the skeleton program as follows:

- $\text{fringe}(\text{seq}(C_i)) = C_i$
- $\text{fringe}(\text{comp}(C_1, \dots, C_n)) = \{C_1, \dots, C_n\}$
- $\text{fringe}(\text{pipe}(\Delta_1, \Delta_2)) = \text{append}(\text{fringe}(\Delta_1), \text{fringe}(\Delta_2))$
- $\text{fringe}(\text{farm}(\Delta)) = \text{fringe}(\Delta)$

with *append* defined as usual, appending at the end of the first parameter the items relative to the second parameter.

- the “normal form” of the original program is given by

$$\mathcal{NF}(\Delta) = \text{farm}(\text{comp}(\text{fringe}(\Delta)))$$

E.G.▷ Consider the program

$$\text{pipe}(\text{seq}(C_1), \text{farm}(\text{seq}(C_2)))$$

The left to right set of leaves is C_1, C_2 and therefore the normal form is

$$\text{farm}(\text{comp}(C_1, C_2))$$

■

Under the assumptions made by the authors in [5], the normal form guarantees the “best” service time for a stream parallel skeleton composition. This means that the following inequality holds, for any legal stream parallel skeleton composition Δ :

$$T_S(\mathcal{NF}(\Delta)) \leq T_S(\Delta)$$

The rationale behind the concept of normal form in this case⁶¹ is the reduction of the overheads related to communications, taking into account that the coarser grain the workers are in a farm, the best speedup and efficiency you can get out of it. The overall result of applying the normal form can therefore be stated as follows:

whenever you have to parallelize a stream parallel program (skeleton composition) the best thing you can do is to “glue up” the largest grain worker by merging all the sequential portions of code needed to produce the final result, and then you can “farm out” the coarse grain worker.

It is clear that the normal form as defined above also minimizes *latency*. If we consider the two equivalent skeleton programs Δ and $\mathcal{NF}(\Delta)$ the absolute wall clock time spent to compute $f(x_i)$, being f the function computed by Δ and x_i the generic input stream item will be smaller when computing f as $\mathcal{NF}(\Delta)$ than as Δ , due to the smaller communication overheads incurring.

E.G.▷ It is worth pointing out that the usage of the term “normal form” in this context does not comply with the “normal form” definition in rewriting systems. In rewriting systems the normal form is the term to which a given term may be reduced that could not be anymore re-written. In our skeleton context, normal form represents the “more performant” rewriting of a given term, which is the term that cannot be re-written any more without incurring in some kind of performance penalty. ■

⁶¹stream parallel skeletons only, stateless, with this particular performance model of the implementation of the skeletons provided to the programmer

9.2 MODEL DRIVEN REWRITING

Most of the rewriting rules given in Sec. 9 are bi-directional, that is, they can be used to rewrite the left hand side skeleton expression into the right hand side one or vice versa.

If we consider a skeleton program Δ and:

- the target architecture,
- the implementation of the skeletons appearing in Δ
- the performance models associated to the implementation of the skeletons appearing in Δ

the problem of finding the “best” rewriting of Δ (with respect to some performance measure $\mathcal{M}$) is quite difficult.

If a set of rewriting rules $\mathcal{R} = \{\Delta_1 \rightarrow \Delta_2, \Delta_3 \rightarrow \Delta_4, \dots\}$ exist, a program Δ may be used to generate the *rewriting tree* $RwTr(\Delta, \mathcal{R})$ such as:

- Δ is the root of the tree $RwTr(\Delta, \mathcal{R})$
- if a rule $\Delta_i \rightarrow \Delta_j$ applies to node $N_k \in RwTr(\Delta, \mathcal{R})$ (that is N_k is a skeleton expression Δ_{N_k} with a sub term matching Δ_i), then N_k is linked to a son node where the occurrence(s) of Δ_i in Δ_{N_k} have been replaced by Δ_j .

The $RwTr(\Delta, \mathcal{R})$ may be infinite. We therefore use the performance models relative to $\mathcal{M}$ to restrict the number of nodes in the tree as follows⁶²:

- we start with $RwTr(\Delta, \mathcal{R})$ being made of the only node Δ with no son nodes.
- we expand the leaves in the tree by one step, i.e. we consider, for each leaf node N_k as many new son nodes as the number of rules in $\mathcal{R}$ matching the skeleton tree of N_k and obtained rewriting the skeleton tree of N_k with these rules.
- we evaluate the performance measure of interest $\mathcal{M}$ on all the new leaves and we flag all the new nodes with the corresponding $\mathcal{M}$ values. Then, for any new node N_i added (with its associated performance measure p_i)
 - if $p_i \geq \min\{p_j \mid N_j \in \{set\ of\ leaves\ in\ the\ previous\ iteration\ tree\}\}$ we stop expanding N_i
 - otherwise, we insert the new node N_i in the skeleton tree
- if the tree has at least one new node, we cycle expanding the tree leaves again.

The tree generated this way is finite as we assume to have available a finite number of processing resources to implement the skeleton tree and any performance measure chosen will be eventually bound by the amount of resources available. As the process of building tree nodes is monotonic, we will eventually reach the performance limit and therefore terminating the generation of the rewriting tree.

In the final tree, the leaf node N_j with the better associated performance value p_j will eventually represent our “best rewriting” of Δ with respect to the set of rules $\mathcal{R}$.

Structured parallel programming models have been investigated for a long time in two different research areas, namely HPC (as algorithmic skeletons [20, 28, 24]) and software engineering [44]. Recently part of the concepts have been inherited and exploited in widely used parallel programming frameworks, including Intel TBB, Microsoft Parallel Patterns

⁶²this is a *branch-and-bound* like approach

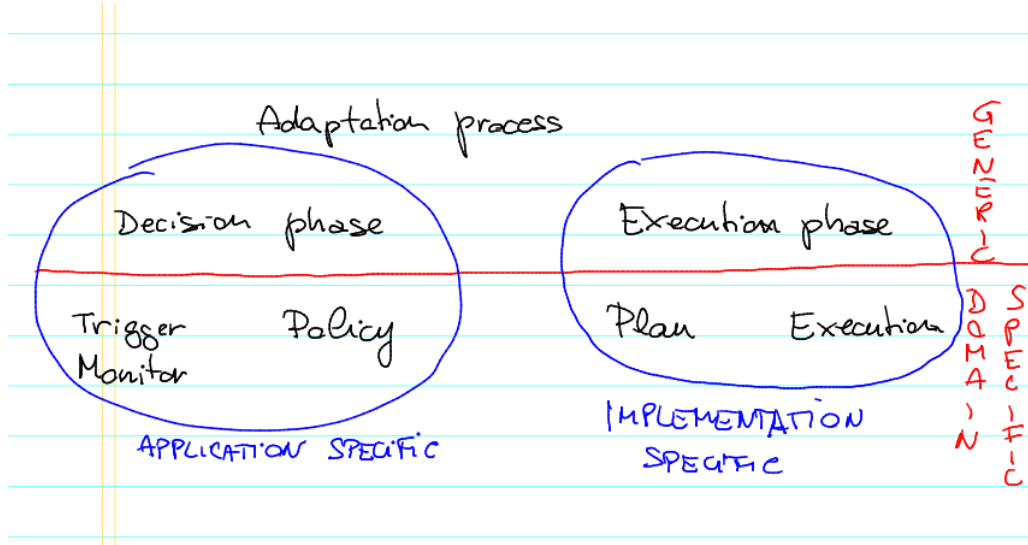


Figure 9.1 General adaptation process

library and, in part, by OpenMP. In a structured parallel programming framework parallel applications are developed using programming abstractions provided as proper programming constructs of the host programming language. C++ frameworks (FastFlow⁶³, Muesli⁶⁴, SkeTo⁶⁵) provide classes modelling parallel patterns, functional frameworks (OSL [37]) provide higher order functions modelling patterns, and so on. These programming abstractions encapsulate most (all) of the details needed to be managed to exploit the parallel patterns. The task of the parallel application programmer therefore mainly consists in picking up the more suitable pattern or pattern composition among those available/feasible. In a number of significant cases, the framework offers a variety of patterns, each with a number of different non functional parameters (parallelism degree, scheduling policy, load balancing policy, communication mechanisms, etc.) and the very same parallel application may be implemented using different pattern compositions.

Background. Different refactoring rules are known that establish *functional* equivalences among different patterns. These rules may be used to derive additional functionally equivalent pattern implementations for a given parallel application. A notable example is the so called *map fusion* rule. It states that *the sequence of two map patterns, one applying function f and another one applying function g over all the items of a given collection (vector, array, tree, ...) is equivalent to a single map pattern applying the composition of f and g , that is $\text{map}(f) \circ \text{map}(g) \equiv \text{map}(f \circ g)$. When used left to right the efficiency of an application is improved as the grain of the computations to be executed in parallel increase. As a consequence, the communication and synchronization overheads are better tolerated. SkeTo designers managed to thoroughly use the rule to improve parallel application performances [41].*

Map fusion rule and the other well known refactoring rules available *de facto* turn the problem of designing an efficient parallel implementation of an application into the problem of exploring a space of solutions to figure out which one is (mostly likely) the *better* one. The application programmer may consider one initial “solution” and then he can use the available

⁶³calvados.di.unipi.it/fastflow

⁶⁴muesli.uni-muenster.de

⁶⁵sketo.ipl-lab.org

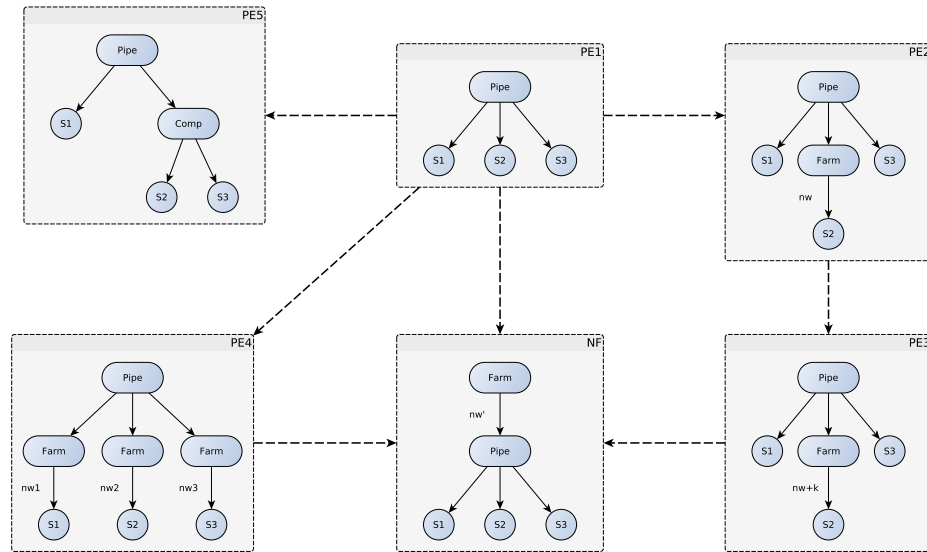


Figure 9.2 Sample pattern expression refactoring

refactoring rules to generate one or more alternative, functionally equivalent solutions only differing in terms of their non functional features.

Fig. 9.2 shows a few refactoring possibilities related to a simple parallel application structured after a three stage pipeline pattern (pattern expression PE1 at the center-top). In case the second stage turns out to be the pipeline bottleneck, it may be parallelized through the introduction of a farm ($PE1 \rightarrow PE2$). This is a transformation affecting the parallel structure of the application. In case we bump into some hot spot and the parallelism degree nw figured out to parallelize the second stage does not fit anymore our needs, we may increase it ($PE2 \rightarrow PE3$). In this case the transformation is only related to the *non functional* parameters of the application, whose parallel structure is left unchanged. The parallel pattern composition used in NF (normal form) is known to be the “best” possible composition of stream parallel patterns, and may be obtained transforming any of the other pattern expressions in Fig. 9.2 (e.g. $PE1$, $PE3$ or $PE4$). The transformation $PE1 \rightarrow PE4$ possibly optimizes pipeline stage load balancing and it determines a change in the parallel structure of the application. Eventually, $PE1 \rightarrow PE5$ reduces the number of resources utilized to execute the pipeline (“comp” being the sequential composition pattern) in case the second and third stage of the pipeline have a total latency not superior to the latency of the first stage. This simple example clearly shows that the productivity and the efficiency of the application programmer may be greatly increased if proper tools are provided to explore the space of alternative pattern expressions and to evaluate their non functional properties *before* actually coding them.

9.3 ff-RPL SHELL DESIGN

ff-RPL main inspiration comes from Mathematica software: a shell is made available, where users experiment different tools available to come up with the solution of their problem. Similarly, ff-RPL is a shell where pattern expressions may be defined, refactored or optimized according to well known rules. In addition, the non functional properties of the pattern expressions may be evaluated and eventually working C++ code may be generated out of any of the pattern expressions. A history mechanism provides a record of all the steps that lead to a given pattern expression thus providing some kind of “recipe” to derive the “good”

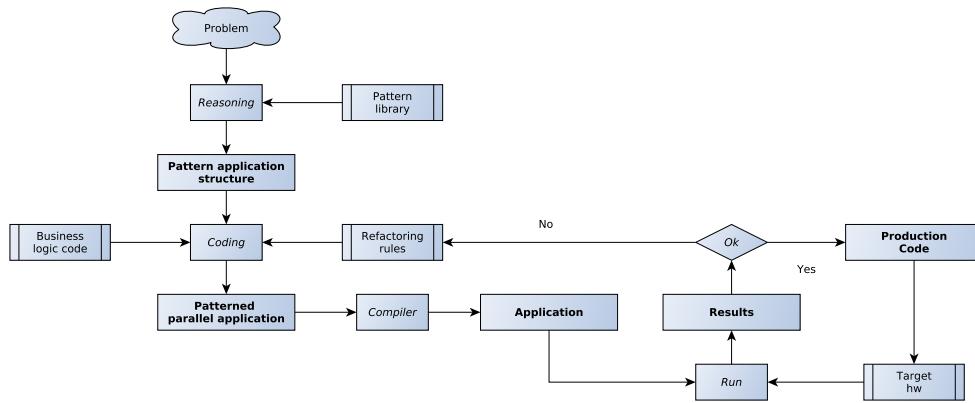


Figure 9.3 Workflow for structured parallel programming application design (Initial pattern expression is designed taken into account the available patterns. The cycle in the lower part represents the tuning activity. Tuning exploits refactoring rules.)

parallel solution out of the initial one. Overall, ff-RPL will support the structured parallel application programmers in all the phases needed to design, implement and tune a parallel application, which we tried to summarize in Fig. 9.3.

Before detailing the ff-RPL shell design, let's have a look to what the shell actually provides through a very simple but significant example, leveraging the map fusion rule. We open the shell, we declare two sequential stages with service time (and latency) of 10 and 20 units of time, respectively. Then we declare a main as a pipe of maps and a main2 as a map of comp. The service time of the main2 is smaller than the one of main, using the same amount of resources (32 processing elements):

```

rplsh> stage1 = seq(10,true)           declare a sequential stage with latency of 10 units of time
rplsh> stage2 = seq(20,true)           declare a sequential stage with latency of 20 units of time
rplsh> maps1 = map(stage1,16)          define a map with 16 workers
rplsh> show maps1
[0] : map(stage1) with [ nw: 16]
rplsh> maps2 = map(stage2,16)          define a map with 16 workers
rplsh> main = pipe(maps1, maps2)        define a pipeline of the two maps
rplsh> show main by servicetime        ask servicetime
1.250000 [0] : pipe(maps1,maps2)        its 1.25 units of time
rplsh> fatw = comp(stage1,stage2)      define seq composition of the two seq stages
rplsh> main2 = map(fatw, 32)           declare a map with 32 fat workers
rplsh> show main2 by servicetime        ask servicetime
0.937500 [0] : map(fatw) with [ nw: 32] its 0.93 units of time
rplsh>

```

As you can see, the usage of the shell verbs looks intuitive and we successfully prove the map fusion rule works as expected. Ideally, once finished and once decided main2 is the correct solution, we may have a look at what steps have been performed to come to this solution and we may ask the shell to generate the code (see details in Sec. 9.6).

The ff-RPL shell design is outlined in Fig. 9.4. The command loop understands and executes the set of *verbs* in Fig. 9.5. These verbs may be used to a) update the environment, b) call one or more evaluators of the supported non functional features, c) apply a refactoring rule (or a set of rules), d) apply an optimization rule and e) import business logic code, and f) generate FastFlow code out of any pattern composition.

Some of the verbs may produce multiple outputs. As an example, asking the shell to refactor a pattern expression with all the available refactoring rules produces a set of new pattern expressions. These result expressions are associated to a vector with the same name of the original expression, that can be easily accessed by using the C++ vector access square brackets (see example in subsec. 9.5).

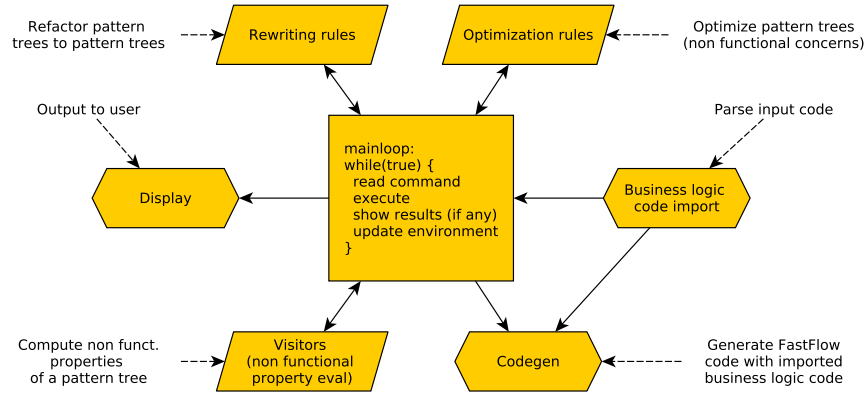


Figure 9.4 Shell design. The main loop uses additional components to implement shell commands. Text outside the colored boxes explains main function of the boxes.

Verb	semantics
set	setup in the environment different global parameters, including time spent in pattern service nodes (emitters, collectors, scatterers and gatherers, etc.), the dimension of the streams and of the data structures processed by data parallel patterns.
annotate	associate different annotations to a pattern, to denote associated service time, latency, parallelism degree, etc.
rewrite	rewrite a pattern expression with one or more of the rules available (see Sec. 9.4)
optimize	optimize a pattern expression according to the rules available (see Sec. 9.4)
expand	rewrite a pattern expression by rewriting its nested names with the associated pattern expressions.
show	displays different information relative to a pattern expression (structure, non functional properties computed with the available models, etc.).
history	displays the steps that led to the current pattern expression, including refactoring, optimization and annotation steps beside “composition” statements used
import	parses business logic code providing proper sequential code wrappers to the shell user
load	executes commands taken from an external RPL script file in the current environment
gencode	generates FastFlow code relative to a generic pattern expression ready to be compiled and executed.

Figure 9.5 ff-RPL verbs

9.4 SUPPORTED FEATURES

ff-RPL implements several refactoring and optimization rules, managing a small but complete set of parallel patterns. In the following, we outline the supported pattern set, the refactoring and the optimization rules, and eventually we discuss the **FastFlow** code generation features implemented in ff-RPL.

Patterns. The shell currently supports different kinds of patterns: *sequential* patterns, including the sequential code wrapper `Seq` and the sequential composition `Comp` patterns, *stream* parallel patterns, including (binary) pipeline (`Pipe`) and task farm (`Farm`) and *data* parallel patterns, including (binary) `Map` and `Reduce`.

Refactoring rules. In addition to the refactoring rules already mentioned in [27] (`pipe` and `farm` introduction and elimination ($\text{pipe}(\Delta_1, \Delta_2) \equiv \text{comp}(\Delta_1, \Delta_2)$ and $\text{farm}(\Delta) \equiv \Delta$), `pipe` and `comp` associativity ($\text{pipe}(\text{pipe}(\Delta_1, \Delta_2), \Delta_3) \equiv \text{pipe}(\Delta_1, \text{pipe}(\Delta_2, \Delta_3))$ and $\text{comp}(\text{comp}(\Delta_1, \Delta_2), \Delta_3) \equiv \text{comp}(\Delta_1, \text{comp}(\Delta_2, \Delta_3))$), `map/comp` and `pipe/map` distributivity ($\text{map}(\text{comp}(\Delta_1, \Delta_2)) \equiv \text{comp}(\text{map}(\Delta_1, \Delta_2))$ and $\text{map}(\text{pipe}(\Delta_1, \Delta_2)) \equiv \text{pipe}(\text{map}(\Delta_1, \Delta_2))$)) we added the following new rules:

- `mapcollapse` : $\text{map}(\text{map}(\Delta)) \rightarrow \text{map}(\Delta)$ flattening data parallel computations
- `farmcollapse` : $\text{farm}(\text{farm}(\Delta)) \rightarrow \text{farm}(\Delta)$ avoiding double scheduling gathering extra nodes in farms⁶⁶
- `pipedel` : $\text{pipe}(\Delta) \rightarrow \Delta$ and `compdel` : $\text{comp}(\Delta) \rightarrow \Delta$ that only solve some problems related to the cases of monadic sequential composition and pipeline patterns result of other refactorings and optimizations.

These refactoring rules represent well known functional equivalences between different pattern compositions [4, 6] and they can be used as parameters in the `rewrite` verb in order to generate different functionally equivalent solutions.

Optimization rules. As far as the optimization rules are concerned, in [27] we considered `PATopt` ($\text{PAT} \in \{\text{farm}, \text{pipe}, \text{map}, \text{reduce}\}$) and `maxresources` rules optimizing respectively the worker number in farms and maps, the stage number in pipelines, the parallelism degree in reduce and computing the best usage of the available computing resources in arbitrary pattern compositions. Here we extend the optimization rules in two distinct ways:

- we re-designed the `rewrite` verb in such a way it accepts an optional `rec` flag and consequently recursively applies the rule on target pattern expression, and
- we added a `twotier` optimization rule that implements the typical two tier structure of parallel computation (outer stream parallel patterns, inner data parallel patterns [36]) by removing the stream parallel patterns parameters nested in data parallel patterns.

The optimization rules supported can be used as parameters in the `optimize` verb in order to optimize either the performance or amount of resources used by a pattern. They either modify the annotations associated to the patterns (*e.g.* `pardegree` in a `farm`) or they change the pattern expression structure (*e.g.* merge fastest stages of a pipeline). At the moment, the only metrics of interest for the optimizations are related to the service

⁶⁶this is actually a shortcut for the `farmelim` rule

```

// stream generator template
template <typename Tout>
class source {
public:
    // true for End-Of-Stream
    virtual bool has_next()=0;
    // next stream item (if not EOS)
    virtual Tout* next()=0;
};

// pure function code template
template <typename Tin, typename Tout>
class seq_wrapper {
public:
    // stream filter computation
    virtual Tout compute(Tin& input)=0;
};

// stream consumer template
template <typename Tin>
class drain {
public:
    // absorb one stream item
    virtual void process(Tin * x)=0;
};

```

Figure 9.6 Business logic code templates (stream generator/absorber (top) and business logic code sequential wrapper (bottom))

time and resource usage. Service times are computed using simple analytical performance models such as those considered in [13, 5]. The pattern expression of interest is modified according to estimations based on implemented performance models and by using user-defined annotations and/or global environment variables (*e.g.* `farmopt` optimizes the farm in such a way that the estimated service time equals the emitter time).

Code generation. `ff-RPL` supports generation of working `FastFlow` code out of the shell pattern expressions *and* of some properly defined “business logic code”. The business logic code generating streams should be implemented according to the templates shown in Fig. 9.6. This is needed to correctly parse the types used for the different tasks without actually requiring a complete parse of the full C++ code of the business logic code. The business logic code components may be read into the shell by importing the file. After the import, the names of the components will be available as standard shell identifiers. The association with the actual business logic code is kept in the environment in such a way suitable `FastFlow` code can be generated, if required. The `gencode` verb only works properly with properly imported business logic code.

Indeed, to simplify the parallel pattern application programmer task we introduced two new annotations `typein` and `typeout` that can be used to associate the input and output (C++) data types to specific sequential portions of code. The annotations may be used to produce some correct but partial “template” code that the programmer may complete with the business logic code portions.

Code generation uses all the modern C++ features as needed and as suggested by the `FastFlow` programming praxis, but it is currently being reviewed to generate better quality code and, especially, to exploit the new pattern interface provided from `FastFlow` version 2 on. Code generation works correctly for pattern expression following the two tier schema (topmost stream parallelism, nested data parallelism). The new `twotier` optimization rule may be used to make any pattern expression compliant with this implicit requirement.

Implementation and expandability. Standard implementation techniques have been used as much as possible in the implementation of `ff-RPL`. DSL language is managed through

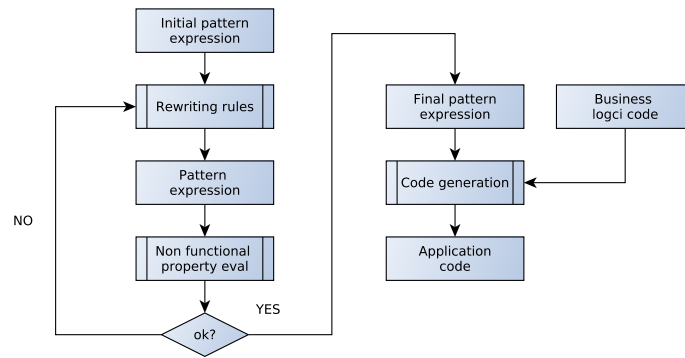


Figure 9.7 User centric ff-RPL workflow

standard lexer/parser tools to produce the internal representation needed to implement the ff-RPL commands. Standard OO patterns have been used to implement visitors and optimizers, in such a way they may be provided independently of the actual abstract pattern expression syntax tree traversing. All the implementation has been designed in such a way new features may be seamlessly implemented. In particular: *i*) new annotations may be added, modelling different non functional features, simply defining a new name, adding proper clauses to the lexer and parser classes and defining proper `visit` methods in the visitor patterns used to recursively manage annotations in pattern expressions, *ii*) new visitors may be defined, mostly adding proper `visit` methods to be used by the Visitor engine traversing the patter expression tree, *iii*) introducing new optimizer and rewrite rules, using a properly defined set of methods supporting access to abstract syntax tree sub-trees. The code of ff-RPL is publicly available on github at <https://github.com/Murray1991/rplsh>.

9.5 ff-RPL USAGE

We show here a small but significant set of sample usages of the ff-RPL shell, that ideally should assess and validate the ff-RPL design and implementation choices. First of all, we illustrate the “Mathematica like” usage of ff-RPL, that is what an application programmer may investigate with a minimal but necessary knowledge on refactoring rules and non functional property evaluators. Then we will discuss the workflow typical of automatic tuning of structured parallel applications and we will outline how this process may be seamlessly supported using the new facilities provided by ff-RPL.

Application programmer driven solution space exploration. The application programmer may explore the solution space exploiting the rewriting rules and the optimization rules available, by providing an initial, annotated pattern expression and subsequently iteratively applying refactorers and optimizers up to the point the non functional measures of interest, computed by ff-RPL through visitors, are the ones he/she was looking for. Fig. 9.7 shows the abstract workflow and Fig. 9.8 shows a sample shell session that illustrates this use case.

The programmer starts importing some business logic code and providing proper annotations with the monitored business logic code latency. Then he/she uses the available sequential components to build a simple pipeline and consults the performance models to get the expected service time. To reduce the service time, the farm introduction refactoring rule is used to farm out the second “compute” stage. The programmer then investigates an alternative pattern expression, by taking away the farm and farming out instead the couple of business logic code stages as a whole. In particular, the `farmopt` rule is used

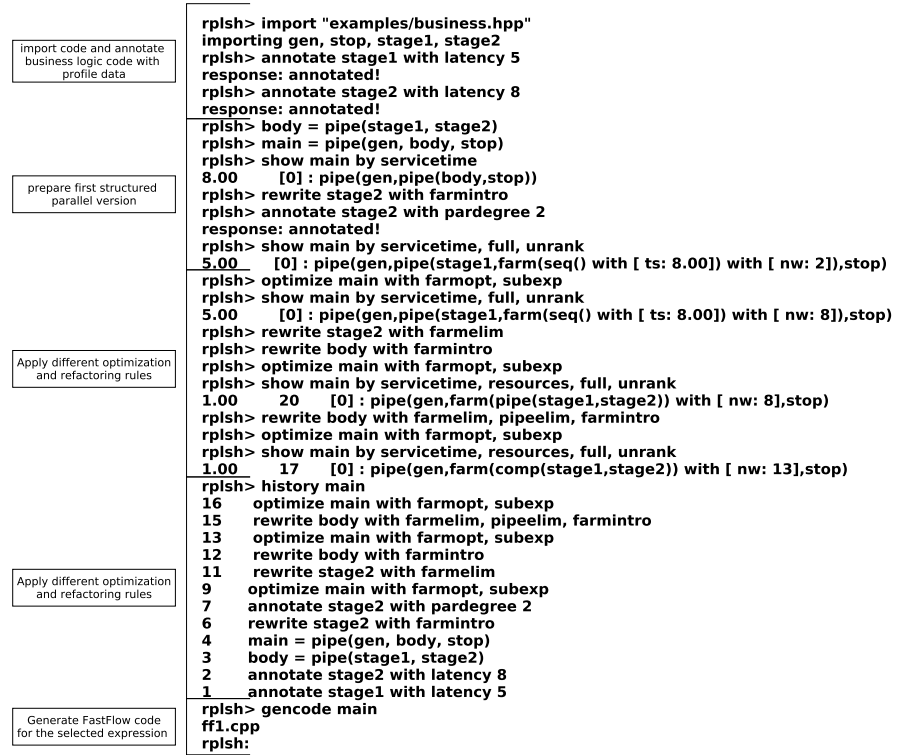


Figure 9.8 Sample ff-RPL sessions (application programmer driven space exploration)

to compute “optimal” number of workers in the farm. Eventually, the programmer got an expression such that the service time is 1 unit of time (the same of the default emitter/-collector modules of the farm) using 17 parallel computing resources. The output lines of the `history` verb represent the actions to be taken to turn the original code from the “business.hpp” source file into a pattern based parallel code implementing the final pattern expression candidate, i.e. `pipe(gen, farm(comp(stage1, stage2)), stop)`. These lines (in reverse order and leaving out the “annotate with latency” ones, that ideally only describe the business logic code performance behavior) show to the user a safe way that he/she can follow in order to transform its sequential application into the desired parallel version. Actually, with the imported business logic code the application programmer may generate the full application code implementing the pattern expression identified in the shell by simply issuing the `gencode` verb with the pattern expression as a parameter (last ff-RPL command in Fig. 9.8).

Automatic optimization workflow. In most cases, the application programmer has not enough knowledge to play with the ff-RPL shell commands up to the point some kind of “optimal” pattern expression is individuated. He/she will have in general *a*) enough know-how to be able to identify a general, possibly very simple pattern expression modelling the dependencies in the application and *b*) a rough idea of the non functional concern of interest (performance, resources used, etc.) to drive the refactoring/optimization process.

As an example, an application programmer may reasonably figure out that an application may be divided in a given number of independent stages, and whether or not some stages may be internally data parallel. This means a simple pattern expression with pipelines and maps may be easily derived as the “initial” pattern expression to submit to the optimization process. Concerning the non functional concerns, the application programmer may easily

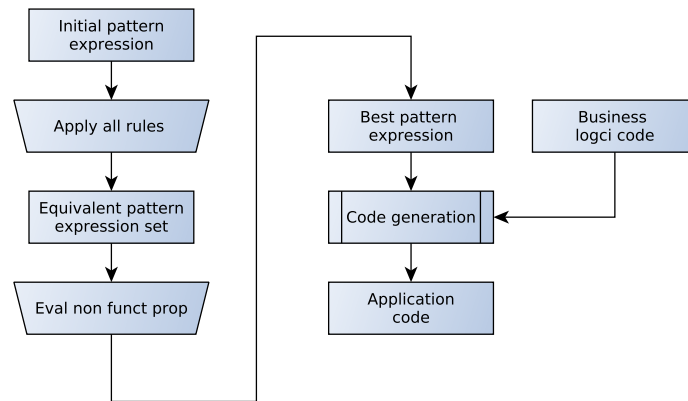


Figure 9.9 Automatic optimization/tuning

```

### Generate equivalent rules, then clean and optimize
rewrite main with allrules, allrules
rewrite main with farmfarmelim, mapmapelim, rec
optimize main with farmopt, maxresources
show main by servicetime, resources +10

```

Figure 9.10 Simple stream parallel expression service time optimization script

say what has to be optimized. Currently we support latency, service time and resources used as the possible targets for optimizations.

Under this assumptions, the workflow to be implemented using ff-RPL will be the one listed in Fig. 9.9. In this case, the second and fourth step in the workflow may be implemented as well defined set of ff-RPL commands. In particular:

- the *apply all rules* step may be built of the commands `rewrite expression` with `allrules` followed by a number of `optimize expression` with `OptRule` applying the different optimization rules in the proper order. The result of this stage is a list of pattern expressions all functionally equivalent to the original one with different non functional features.
- the *eval non func prop* step simply consists in two commands: *i)* a `optimize` with `maxresources` command, reducing the number of resources needed to the one actually available, in and *ii)* the command `show expression by NFProp`, where the `NFProp` is the non functional property evaluator of interest. The command provides an ordered list of pattern expressions whose first item(s) is (are) those optimized with respect to the non functional feature of interest.

ff-RPL provides a way to read and interpret a set of commands from a file (`load` verb). Leveraging on this feature, we may easily provide a command file such as the one in Fig. 9.10 that applies all the steps needed to optimize a pattern expression named “main”. With such a file available, the application programmer is in charge only of the first step in the flowchart of Fig. 9.9, namely to define the initial, “simple” pattern expression and then he/she may use the file to get an optimized version of this initial pattern expression:

```

rplsh> s1 = seq(10)
rplsh> s2 = seq(20)
rplsh> s3 = seq(30)
rplsh> main = pipe(s1,s2,s3)
rplsh> load "examples/stream_servicetime_opt_rules"
rplsh> show main by servicetime,resources +4
4.285714 16 [4] : farm(comp(s1,comp(s2,s3))) with [ nw: 14]

```

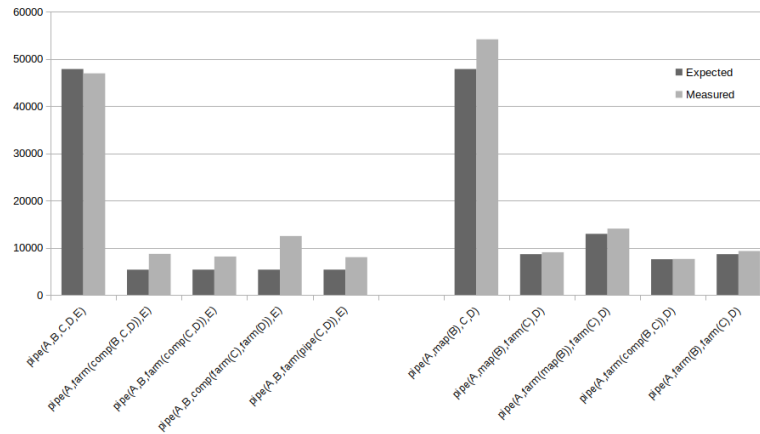



Figure 9.11 Predicted/Estimated vs Measured completion times (Y axis represents completion times in msecs, normalized)

```

4.571429 16 [16] : comp(farm(s1) with [ nw: 10],
                      comp(farm(s2) with [ nw: 14],
                          farm(s3) with [ nw: 14]))
4.571429 16 [47] : comp(farm(s1) with [ nw: 10],
                      farm(comp(s2,s3)) with [ nw: 14])
5.000000 16 [7] : pipe(farm(s1) with [ nw: 2],
                      farm(comp(s2,s3)) with [ nw: 10])
rplsh> gencode main[4]

```

Metric usage. ff-RPL embeds in the pattern “performance” models all the knowledge related to the target hardware. At the moment being, singleton abstract performance models are provided relative to shared memory multi core architecture assuming UMA memory accesses. ff-RPL may be easily extended with different models such that user may experiment different performance estimates simply using the model relative to the target architecture of interest. Is it worth pointing out that the shell does not provide any suggestions concerning the “best” pattern expressions to be used. Rather, ff-RPL provides all the pattern expressions in the pattern refactorings required by the user/programmer. The only facility provided to evaluate alternatives is the possibility to “show” pattern expression refactoring/optimization results sorted by the non functional feature of interest (e.g. `show prog by resources`, `servicetime` sorts pattern expressions in `prog` first relatively to the resources used and second relatively to the service time provided).

9.6 VALIDATION

In order to validate the general ff-RPL design, we performed different experiments aimed at: validating the performance models used in design space exploration, validating the code generator prototype and validating the refactoring and optimization rules used.

Performance model validation. To run the performance tests we used four different small applications, whose parallelisation aspects are known. As mentioned, the performance models included in ff-RPL only provide abstract modelling useful to compare different alternative pattern expressions rather than to provide actual expected performance (completion time or service time) figures. Performance experiments have been conducted on different architectures, including a machine hosting 2 AMD Magny Cours CPUs, each with 12 cores, no

hyper-threading, an Intel dual E5-2695 v2 Xeon with 48 cores, 2 way hyper-threading and a Xeon PHI KNL with 64 cores, 4 way hyperthreading. The applications we used include:

- a video filtering application, processing 1920×1080 pixel frames with three filters eventually producing a greyscale “Sobel” video, just exploiting stream parallelism,
- an application computing 1K 1000×1000 pixel images representing different views of the Mandelbrot set with 1K colors exploiting both stream and data parallelism,
- a synthetic “dot product” kernel, exploiting a map reduce kernel with sum and product operators lasting 5 and 15ms, respectively, to emulate more complex operators⁶⁷. The kernel processed a stream of vector pairs, therefore both stream and data parallelism has been exploited, and
- a classical map reduce word count kernel.

Fig. 9.11 shows some of the results achieved while comparing the predicted times with the times actually measured on our multi core architectures. The left part is relative to stream parallel video filtering application, the right part is relative to the stream+data parallel Mandelbrot kernel. The X axis legend shows the different refactorings considered with the capital letters representing the different sequential business code portions. In the left block, the fourth plots are relative to a patterns with a `comp(farm(...), farm(...))` component that did not performed as expected (see Sec. 9.6 below). Apart from this, we can observe that, despite the differences between the estimated and measured completion times, the relative ordering is basically respected. This assesses the quality of the suggestions obtained using the ff-RPL when exploring a patterned application design space implicitly defined by the available refactoring rules.

Code generation validation. For the applications listed in Sec. 9.6 we provided properly wrapped business logic code and we verified the ff-RPL generated code was correct. It is worth pointing out that at the moment being **FastFlow** does not provide a `comp` pattern. We therefore provided a trivial implementation of the pattern that actually performs perfectly on sequential components (e.g. in a `comp(seq, seq)`) but demonstrates poor performance on non sequential components (e.g. in a `comp(farm(seq), seq)`). This is the reason why in some experiments involving stream parallel patterns we measured sensibly different completion times than the ones computed with the performance models in ff-RPL.

Fig. 9.12 shows part of the business logic code used to implement the video filtering application, just to give an idea of the kind of effort required to the application programmer to reuse the existing code within the wrapper templates already shown in Fig. 9.6.

Rule implementation validation. Each one of the rules supported in the `rewrite` and `optimize` verb implementation have been carefully tested to *a)* check that the result expressions (after refactoring) were correct independently of the nesting context and *b)* that the code generated before and after the application of the rules correctly computed the same results⁶⁸.

9.7 RELATED WORK AND CONCLUSIONS

To the best of our knowledge, there is no other tool providing ways to refactor and optimize parallel pattern expressions. None of the existing pattern based parallel program-

⁶⁷delays added through active waiting

⁶⁸this is a sort of cross validation of correctness of refactoring rules *and* of prototype code generation, actually

```

class vsource : public source<Mat> {
public:
    vsource() : vsource("sample.mp4") {}

    vsource(const std::string& filename) : capture(filename) {
        if (!capture.isOpened()) {
            std::cerr << "error: no video open" << std::endl;
            exit(EXIT_FAILURE);
        }
    }

    bool has_next() {          // are there more frames to read?
        frame = new Mat();
        return capture.read(*frame);
    }

    Mat* next() {              // to be used after has_next !!
        return frame;
    }

protected:
    VideoCapture capture;      // class for video capturing
    Mat *frame;                // frame pointer for video frames
};

// sobel filter
class sobel : public seq_wrapper<Mat, Mat> {
public:
    Mat compute(Mat& in_frame) {
        Mat out_frame;
        Sobel(in_frame, out_frame, -1, 1, 0, 3 );
        return out_frame;
    }
};

```

Figure 9.12 Business logic code for the video filtering application (partial, to be linked with OpenCV)

ming frameworks supports programmers in the design and tuning of parallel applications at this abstraction level. Rather, most of these frameworks provide ways to automatically monitor performances and some of them provide ways to refactor actual parallel code (www.paraformance.com/). In this paper, we discussed the last version of our ff-RPL, an interactive tool that supports the programmer in the design of structured parallel applications. The tool offers a DSL based approach to write, evaluate, refactor and optimize parallel patterns that may help the application designer to find the “best” parallel structure for its application and assist programmers to develop rapid prototypes of their application. The tool is completely written in C++ and re-implements (with a new command syntax and more functionalities) a previous *proof-of-concept* prototype written in OCaml presented in [25, 33]. The version discussed in this papers adds new verbs to the DSL, such that the development of new patterned parallel applications or the tuning of existing ones is simplified. The work-flows to design new applications and to refine existing ones have been clearly defined, along with the usage of the new verbs and attributes included in this improved version of ff-RPL.

The experiment section assesses usability, correctness and efficiency of ff-RPL. In particular, our application examples show that parallel performance models actually provide realistic approximations of the actual performances achieved with the generated code. “Realistic approximations” means the predicted/measured times actually differ, but it is possible to see that to similar predicted times correspond similar measured times and, typically, the relative ordering of the predicted/measured performances corresponding to different pattern expressions is preserved. For this reason we consider the overall validation of the ff-RPL satisfactory.

We are currently working to include (and provide a methodology to include) different non functional performance models. In particular, we are working to include alternative time and energy models (completion time and energy consumed) based on machine learning techniques in the evaluators available in ff-RPL. The aim is *a)* to include new a more sophisticated models estimating completion time also for those cases where the actual parallel resources are less than the exposed parallelism (excess parallelism, hyper-threading exploitation), *b)* to include models estimating energy consumption (which are not taken into account at the moment being) and *c)* to show how completely different models (machine learning based vs. analytical ones) may be seamlessly integrated into ff-RPL.

CHAPTER 10

ADAPTIVITY

Most of times, parallel programs execution needs to take into account with variations of the execution environment. These variations may be:

Endogenous that is depending on the parallel application features. As an example, a parallel application may be such that computations phases have different “weights” and therefore, in case they are executed on a constant number of processing resources, in some phases the available resources are underutilized and in other phases they are over utilized. This obviously leads to poor application performance, in general.

Exogenous that is depending on the target architecture features and/or behaviour, which represent causes somehow “external” to the parallel application. This is the case of non exclusive target architecture access, for instance. If another (parallel) application is run on the very same resources our application is currently using, the load share may lead to a poor performance of our application. Or, in case there is a kind of “hot spot” in Operating System activities, the same kind of impact on the running parallel application may be observed, even in case of exclusive target architecture access⁶⁹.

Therefore, dynamicity in the conditions related to parallel application execution generates two distinct kind of problems:

- *resource under utilization*, which is bad as unused resources may be dedicated to more useful tasks instead, and
- *resource over utilization*, which is bad as more resources could greatly improve the overall application performance, if available.

⁶⁹this is the case of operating system activities scheduled at fixed times, such as backup activities, for instance.

In turn, these two kind of problems have different effects on the execution of parallel applications:

- load unbalancing may be verified,
- performance is unsatisfactory,
- efficiency is poor.

Therefore it is natural to think to the implementation of some kind of *adaptivity process* that could dynamically adapt the application execution to the varying features of the execution environment or to the varying needs of the application itself.

E.G.▷ Adaptation has been included in some applications, not using skeletons, actually. As an example *multi-grid* applications, i.e. those applications that analyze phenomena that happen to have different effects in different space portions usually adapt the computation to space portions features. Let suppose you want to compute the fluid dynamics of airflow on an airplane wing. Simulation must be more accurate in the space portions where more complex dynamics take place, such as on the front wing borders. Less accurate simulation is enough on flat wing surface space portions, instead. Therefore, if we want to tackle the problem with a kind of data parallel approach and we divide the wing space in equal portions, the portions simulating airflow on the wing border will over stress the computing resources they have got assigned while (comparatively) the portions simulating airflow on wing flat surface will under utilize the (same dimension) resources assigned. This require some kind of adaptation. In multi grid applications such as the one just mentioned this may consist in

Static adaptation The partition of the overall simulation space takes into account the features of the computation and splits more deeply those areas subject to more intensive computations, in such a way eventually all the partitions generated achieve a good load balancing.

Dynamic adaptation The initial partitioning is uniform (a number of partitions with similar dimensions) but then, at run time, in case a partition turns out to be “too computationally intensive”, it may be dynamically split in such a way the resulting partitions use both the resources previously assigned to the original partition and further resources dynamically recruited.

Now, when unstructured parallel programming models are used, the adaptation is actually programmed by intermingling adaptation code with the application code (functional and related to parallelism exploitation). This requires notable extra-knowledge to the application programmer and also contributes to poor maintainability of the resulting code. — ■

The structure of general purpose adaptation process suitable to manage dynamicity in parallel applications have been studied by different authors. A general adaptation schema has been proposed in [3] which is represented in Fig. 9.1. In this case:

- The adaptation schema is clearly organized in two steps:
 - DS Decision step: the behaviour of the execution of the application is monitored and analyzed. Possibly, decisions are taken concerning the adaptation of the application, in case it does not perform as expected.
 - ES Execution step: a plan is devised to implement the decisions taken in the decide step and this plan is eventually executed in such a way the decision could be implemented.

In turn, the decision step may be logically divided into *monitoring* and *analyze* steps and the execution step may be logically divide into *plan* and *execute* steps, with obvious “semantics” associated to each one of these phases.

- The adaptation schema is also logically divided in two layers:

GL Generic layer. This encapsulates all the *generic*—i.e. non specific of the particular application at hand—decisions and actions.

DSL Domain specific layer. This encapsulates all the details more specifically related to the particular application domain.

- Eventually, the whole adaptation process has some parts that are application specific and some that are implementation specific. In particular:
 - The monitor, plan and execute phases are implementation specific. They rely on the mechanisms provided to perceive application behaviour (called *sensors*) and on those provided to actuate decisions (called *actuators*).
 - The analyze phase is completely application dependent and *de facto* it embodies all the details specific of the application at hand.

Let us analyze more in detail the two steps of the adaptation process.

Decision step. In the first part of the decision step the current behaviour of the application execution is monitored. The necessary monitoring activity is usually planned at compile time by inserting proper monitor “probes” into the application parallel code. This activity is usually referred to by the term *code instrumenting*. Code instrumenting can be organized such that it is automatically handled by the compiling tools or performed under user (the programmer, in this case) responsibility. In the former case, the compiling tools must be exposed with the parallel structure of the application to be able to place the probes in the more appropriate places within the application source code. In general, the monitoring phase of the decision step is greatly impacted by the number and kind of probes available on the target architecture.

E.G.▷ Most of modern architectures (CPUs) provide *hardware counters* that can be used to measure clock cycles spent in a computation, as well as cache hits/misses and other measures of interest to determine the performance of an application. In case these hardware counters are not available, then more approximate measures can be obtained through operating system probes. The quality of the monitoring results is consequently more approximated. — ■

In the second part of the decision step the current behaviour of the application—as exposed by the monitoring phase—is analysed and decisions are possibly taken concerning adaptation actions to be performed. This phase must be supported by some kind of abstract performance modeling of the application at hand. The performance figures computed using the abstract models should be compared to the performance figures actually gathered through monitoring. In case the two measures do not match, adaptation is required.

Once established that adaptation is required, the analysis phases must be able to figure out which *abstract adaptation actions* may be taken to handle the mismatch in between observed and predicted performances. Possible abstract actions include increase/decrease the parallelism degree and load re balancing, as an example, but also more complex actions such as complete restructuring of the parallel application, that is changing the parallelism exploitation pattern—or the pattern composition—used to implement the application.

The analysis phase may be implemented with one of two possible “general behaviours” in mind:

- It can be implemented as a *reactive* process. The analysis phase reacts to the monitored events not matching the expected application behaviour and decides to take *corrective* actions impacting future application behaviour.
- It can be implemented as a *proactive* process. The analyze phase proactively takes decisions that try to anticipate possible malfunctioning of the application. It digs

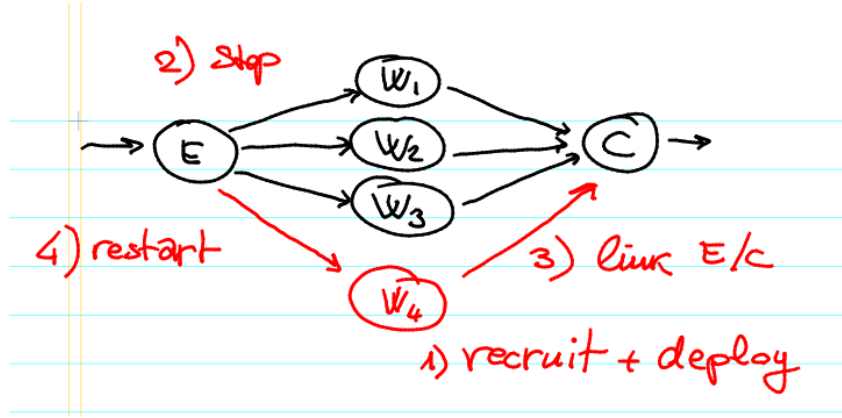


Figure 10.1 Adding worker plan

monitored events and performances to look for symptoms of future malfunctioning and reacts to the symptoms—rather than to actual malfunctioning—in such a way the appearance of malfunctioning is prevented.

Execution step. The execution step is actually built out of two distinct activities. The first activity is aimed at devising a suitable implementation plan for the decisions taken in the decision step. The second one is aimed at actually implementing the decisions taken on the running application according to the plan just devised.

During the plan devising phase, the ongoing computation may proceed without any kind of interference. As a matter of fact, apart from the monitoring phase of the decision step and the commit phase of the execution step, the current computation is not impacted by the adaptation process. Also, during the devising phase the parallel structure of the application—exposed through the skeleton tree actually used in the application—is exploited to devise the precise points of intervention on the running application to be used to implement the adaptation decision taken. In particular, solutions requiring a minimum downtime of the running application will be taken into account.

E.G.▷ When implementing the decision to increase the parallelism degree of a farm, we both the following implementation plans are feasible⁷⁰:

1. stop farm template execution, then recruit a new processing resource, then deploy and start worker code onto the recruited resource, then link the worker to the emitter and collector processes, restart farm execution.
2. recruit a new processing resource, then deploy and start worker code onto the recruited resource, then stop template execution, then link the worker to the emitter and collector processes, restart farm execution (see Fig. 10.1).

Clearly, the second plan requires much smaller downtime than the first one as during all the process of resource recruiting and of remote code deployment, the farm template may continue its normal operations. ■

At the end the plan devising phase, we may assume an ordered list of actions $\langle a_1; \dots; a_k \rangle$ is devised that will implement the decisions taken in the decision phase. Each action a_i is defined in terms of the *actuators* provided by the implementation of the skeletons affected by the adaptation decision. This means that in case of template based skeleton frameworks

⁷⁰we assume the farm is implemented through an emitter-workers-collector template such as the one discussed in Sec. 7.2.1

the templates implementing skeletons should provide actuators to implement any reasonable adaptation action. In case of a macro data flow based framework, the implications are more relative to the possibility of change *on-the-fly* the template used to implement the distributed macro data flow interpreter, instead.

If we take a “template first” viewpoint, instead, the actuators provided by the templates *determine* the kind of dynamic⁷¹ adaptation decision the system can actually implement. It is clear, in fact, that if the system does provide, as an example, mechanisms to stop the execution of our template and to modify the mechanisms or the configuration used during template execution, it will be impossible to actuate any kind of decision taken during the adaptation process.

The second phase of the execution step simply scheduled and applies actions $\langle a_1; \dots; a_k \rangle$ onto the current running application. At the moment being we are only considering consecutive actions, that is we assume that the sequence $a_i; a_{i+1}$ has to be executed as

first execute action a_i and then execute action a_{i+1} .

Nothing prevents to adopt a more structured action plan language. As an example, we can assume that actions are defined by a grammar such as:

```

Action ::=
    <simple action>
  |   Action then Action
  |   Action par Action

```

where we interpret the second clause as “do first action then do the second one” and the third one as “do the first and the second action in parallel”.

Also, we should consider the possibility that each action corresponds to a finite sequence of “actuator mechanisms calls”, as the actuator mechanisms provided by the system may range from low level ones—simple, RISC style mechanisms—to high level ones—corresponding to full actions or event to entire parts of actions plans.

E.G.▷ To illustrate how the adaptation process discussed in this section works, we consider a simple data parallel computation and we discuss all the aspects related to adaptation in detail.

The application we consider is such that it can be expressed using a single data parallel skeleton. Computation at the workers is sequential. The expected performance will be roughly proportional to the number of workers used in the implementation of the data parallel skeleton⁷².

During the monitoring phase we must gather information concerning the *progress* of our application. If the data parallel skeleton actually worked on a stream of data parallel tasks, it would be probably enough to gather data relative to the service time of the data parallel skeleton. This needs only one kind of sensor, namely the one reporting the inter-departure time of results from data parallel template “collector” process, that is from the process in charge of delivering the results onto the output stream. However, if no stream is present⁷³, we must be able to monitor the progress in the computation of a single data parallel task, the only one submitted for computation. In this case, all worker processes must be instrumented in such a way they report the percentage completed of the task(s) they received. This is a little bit more complex to be implemented, as a sensor. First of all, each worker must know a way to devise the percentage of work already performed. This is clearly an application dependent issue. Therefore only the programmer may probably be able to write this sensor. In some simple cases, the sensor may be determined from the skeleton and from the template structure. As an example, in embarrassingly parallel computations on arrays, each sub-item y_{ij} of the result is computed applying some kind of f to the corresponding sub item x_{ij} of the

⁷¹and run time

⁷²we assume to use a template based skeleton framework

⁷³as in a pure data parallel computation

input data structure. The implementation template will probably assign a range of indexes $\langle [i_{min}, i_{max}], [j_{min}, j_{max}] \rangle$ to each one of the workers. Therefore each one of the workers will execute a

```
for (i=imin; i<imax; i++) { for (j=jmin; j<jmax; j++)
```

and as a consequence at each iteration it will be able to give the percentage of the iterations already performed.

During the analyze phase we reason about the progress of our data parallel computation. We may observe different situations that may possibly be corrected with a corrective action:

- workers computed sensibly different amounts of the assigned work. This denotes *load imbalance* than in turn will eventually produce poor performance. Therefore the computations assigned to the workers that have currently computed much less than the other workers may be split and partially re-distributed to new workers, recruited to the purpose. Alternatively, the computation assigned to these workers may be split and the “excess” portions may be delegated to the workers currently performing better than expected, if any.

It is worth pointing out that

- * load imbalance may be due to extra load charged on the processing resources used by the under performing workers, but also
- * load imbalance may be due to intrinsically heavier work loads assigned to the workers. Equal size partitioning of sub-items to workers is not the optimal choice in case of sub-items requiring sensibly different times to be computed, as an example.
- workers computed fairly equal amounts of the assigned work, but this amount does not correspond to the expected amount. In this case, we can assume that the target architecture eventually is “slower” than expected and therefore we can assume to re-distribute work among a larger number of workers, with extra workers recruited on-the-fly to the computation.
- workers computed fairly equal amounts of the assigned work, but this amount sensibly exceeds the expected amount. In this case, we can assume that the target architecture eventually is “faster” than expected and therefore we can assume to re-distribute work among a smaller number of workers, dismissing some of the workers currently used.

During the planning phase, several alternative decisions can be made. As an example:

- when working on an embarrassingly parallel data parallel skeleton, the plan should include the actions:
 - * stop worker with excess load, split load still to be computed in two parts, then *in parallel*
 - re-assign the worker the first part, restart the worker *and*
 - stop a worker with low load left to compute, re-assign the second partition to the worker and restart it.
- when working on a stencil data parallel skeletons, the plan should include the actions:
 - * wait for a barrier to be executed⁷⁴
 - * then proceed as in the case of the embarrassingly parallel skeleton, that is stop the worker with excessive load, split load ... etc.

⁷⁴this is the mechanisms usually employed to synchronize workers before starting a new iteration, in such a way all the workers start with a consisted “neighborhood to collect stencils

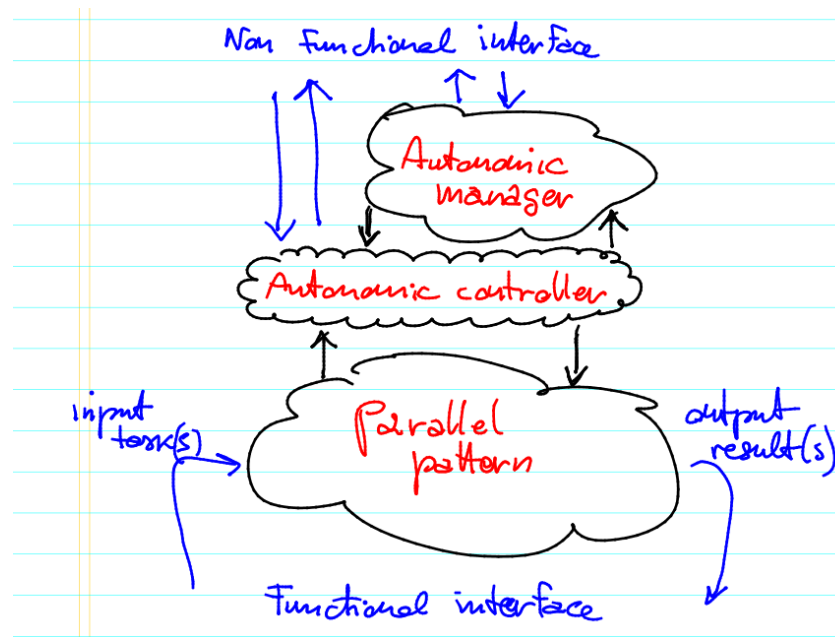


Figure 10.2 A behavioural skeleton (abstract view)

10.1 BEHAVIOURAL SKELETONS

Behavioural skeletons have been introduced in early 2000s with the aim of supporting autonomic management of non functional features related to skeleton implementation. In particular, the first non functional property taken into account was performance.

A behavioural skeleton is the result of the co-design of a parallelism exploitation pattern and of an autonomic manager, taking care of some non functional feature related to the parallelism exploitation pattern implementation.

The concept behind behavioural skeleton is simple. By exposing the parallel computation structure to the autonomic manager, the manager may quite easily implement the adaptation schema discussed in Sec. 10. The complete separation of concerns between pattern implementation—taking care of the details related to parallelism exploitation—and manager—taking care of non functional features—allows very effective adaptation policies to be implemented. Eventually, the separation of concerns allows very clean and effective interfaces to be implemented exposing parallel pattern implementation (monitoring/triggering interface) and the parallel pattern mechanisms suitable to intervene on pattern implementation (actuator interface) to the autonomic manager.

Also, the complete separation of concerns between pattern implementation and pattern non functional management allows management policy reuse across different skeletons with similar implementations.

E.G.▷ As an example, once a proper performance management policy has been developed in an autonomic performance manager controlling a task farm emitter-workers-collector implementation, the very same policy could probably be reused in the implementation of an autonomic performance manager controlling an embarrassingly parallel skeleton, that will probably run a very similar scatter-workers-gather implementation. ■

A behavioural skeleton (see Fig. 10.2) is made out of separate and cooperating entities, both active and passive, namely:

Parallel pattern (active+passive) This is actually the parallel pattern implementation template, that is the part actually computing—in parallel—the function exposed to the user/programmer by the behavioural skeleton.

Autonomic manager (active+passive) This is the manager taking care of non functional features related to the pattern implementation. It implements the adaptation policies as outlined in Sec. 10.

Autonomic controller (passive) This component of the behavioural skeleton embodies the interfaces needed to operate on the parallel pattern execution, both the monitoring/triggering and the actuator ones.

Active parts of the behavioural skeleton are implemented with their own control flow (e.g. as threads, processes or components), while the passive parts are implemented in such a way their services may be started from an external, independent control flow. In particular:

- the autonomic controller is passive in that it only provides services modeled by the monitoring, triggering and actuator interfaces. Monitoring and actuator interfaces may be invoked from the behavioural skeleton manager active part. Triggering interface is usually invoked by the manager active part to register proper callbacks to be invoked when a given event happens in the parallel pattern execution.
- the parallel pattern passive part provides the interfaces needed to operate the pattern, that is to submit task(s) to be computed and to retrieve the corresponding computed result(s). These interfaces are known as the behavioural skeleton *functional interfaces*.
- the autonomic manager passive part provides instead the *non functional* interfaces, that is those interfaces that may be used from outside to monitor behavioural skeleton execution (that is to retrieve measures and parameters relative to the behavioural skeleton execution progress) and/or to intervene on the behavioural skeleton manager policies (that is to re-define the management policies of the manager).

The functional and non functional interfaces of behavioural skeletons may be implemented in two different styles: in RPC/RMI style or in a more conventional⁷⁵ send/receive data flow channel style.

- In the former case, computing services offered by the behavioural skeleton may be invoked as we invoke a remote procedure or method. The invocation may be synchronous or asynchronous. If synchronous, the invocation is blocking and we'll eventually get the result of the service as result of the remote procedure/method invocation. If asynchronous, instead, we will immediately get an handle (usually a *future*) that represents the result that will be eventually computed by the behavioural skeleton. Once the result is actually needed the handle (future) *reification* may be invoked. This invocation is synchronous and will only terminate when the handle (future) is actually substituted by the actual computed value.
- In the latter case, instead, behavioural skeletons will implement ports (kind of communication channels) and services may be invoked sending data items to proper ports and receiving results from other ports.

The very first behavioural skeletons have been designed and implemented within the reference implementation of the GCM, the CoreGRID Grid Component Model. In that case:

⁷⁵at least from the parallel processing viewpoint

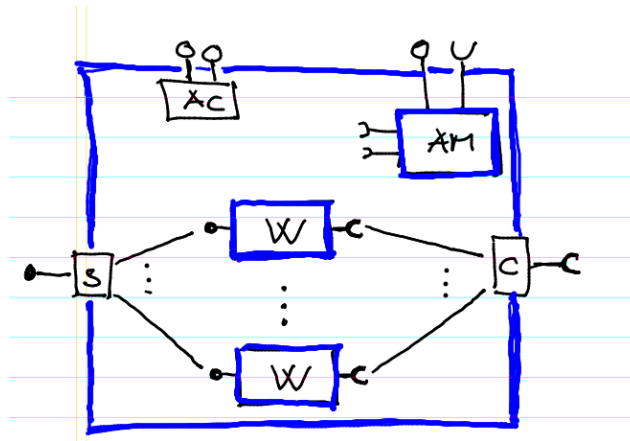


Figure 10.3 Functional replication behavioural skeleton

- each behavioural skeleton was provided as a compound component
- the autonomic manager (AM) and the parallel pattern were components
- the autonomic controller (AC) was part of the interface of the parallel pattern component
- the behavioural skeleton non functional interface was the interface of exposed by the autonomic manager component
- the behavioural skeleton functional interface was the interface exposed by the parallel pattern component
- autonomic managers was only taking care of performance of the parallel patterns
- pipeline and functional replication behavioural skeletons were implemented. The functional replication behavioural skeleton could be specialized to implement both stream parallel and data parallel embarrassingly parallel computations.

10.1.1 Functional replication behavioural skeleton in GCM

Fig. 10.3 shows the structure of the functional replication behavioural skeleton. Functional replication behavioural skeleton (FRbs) operates as follows:

- the functional server port accepts input stream data items and schedules them for execution to the FRbs internal worker components.
 - If the FRbs is configured as a stream parallel behavioural skeleton, each input data item is scheduled one of the worker components in the FRbs. The scheduling policy may be customized by the programmer instantiating the FRbs. The default scheduling policy is “on demand”: the task is scheduled to the first worker component asking for a new task to compute.
 - If the FRbs is configure as a data parallel behavioural skeleton, each input task is split into a set of partitions (one for each internal worker component) and this partition set is scattered to the set of internal worker components.
- the worker components implement a loop. The loop body starts asking for a new task to be computed, then proceeds computing the task and sending the results to the

client port delivering the FRbs results. Possibly double or triple buffering techniques are used to hide communication latencies.

- the functional client port receives results from the internal worker components and delivers them to the FRbs output stream.
 - In case the FRbs is configured as a stream parallel behavioural skeleton, each item received from an internal worker component is delivered to the FRbs output stream, possibly after a reordering phase aimed at maintaining the input/output ordering of tasks/results.
 - In case the FRbs is configured as a data parallel behavioural skeleton, the client port waits—in a sort of barrier—a result item from each one of the internal worker components, then it rebuilds the global result and this result is eventually delivered to the FRbs output stream
- the autonomic controller AC implements the monitoring and actuator interfaces of the pattern. Monitoring interface provides methods to read the amount of tasks already computed (completed) by the FRbs, the current parallelism degree, the service time delivered by and the inter arrival time measured at the FRbs. Actuator interface basically provides two services: one to increase and one to decrease the FRbs parallelism degree.
- each FRbs is given a *performance contract* either by the user or by the upper level behavioural skeleton in the behavioural skeleton nesting. The performance contract establishes, in the form of a Service time, the non functional behaviour expected by the user and therefore the goal to achieve through autonomic management policies in the autonomic manager of the FRbs
- the autonomic manager includes a JBoss business rule engine⁷⁶. Such engine processes a set of *pre-condition* $\rightarrow$ *action* rules. The set of rules is processed in steps:
 1. first, all the pre-conditions are evaluated. Pre-conditions, in our case, include formulas on the values provided by the autonomic controller monitoring interface. As an example, a pre-condition may be

$$AC.currentServiceTime() \leq X \text{ AND } AC.parDegree == Y$$
 2. then the rules with a true pre-condition are considered. The Rete algorithm is used to pick up one of these rules as the candidate rule
 3. the candidate rule is executed, that is action part is executed. The action part of the rule is made of a sequence of FRbs autonomic controller actuator interface calls.

The autonomic manager of the GCM behavioural skeletons works in quantum deadlines. Each behavioural skeleton is associated with a quantum time. In a loop, a timer is started with this quantum time. When the timer elapses, the JBoss rule engine is run, and the loop is started again.

- Sample policies implemented in the FRbs manager include:
 - augmenting the parallelism degree in case the inter arrival time is smaller than the current service time

⁷⁶The JBoss rule engine has been first used in a SCA based implementation of behavioural skeletons. The very first behavioural skeletons in GCM used plain Java code to implement the autonomic manager (policies)

- lowering the parallelism degree in case the current service time is much better than the user specified performance contract
- avoiding taking an increase/decrease parallelism degree decision in case a decrease/increase decision as just be taken. This avoid fast fluctuation of the FRBs around a given parallelism degree value.

This kind of behavioural skeleton has been demonstrated effective in different situations, including:

Computation hot spots In case a computation traverses an “hot spot”, that is a situation where the currently available resources are not sufficient to exploit all the parallelism present, new resources are recruited and allocated to the computation (e.g. as new workers) in such a way the hot spot effects are reduced. The recruited resources are automatically released when the computation needs lower again after the end of the hot spot.

Target architecture overload In case a computation is executed on a target architecture whose nodes become overloaded by other applications, the autonomic manager succeeds recruiting more resources to the computation in such a way these new resources supply the computing power missing from the overloaded resources. As before, the recruited resources are automatically released when the target architecture nodes are anymore overloaded.

Initial parallelism degree setup Long running applications may be started with an arbitrary initial parallelism degree. The autonomic manager will recognize whether or not this parallelism degree supports the user supplied performance contract and intervene on the parallelism degree consequently. Eventually, the application will reach a state such that the measured service time matches the user performance contract.

Some further points are worth to be mentioned, concerning GCM behavioural skeleton implementation:

- the autonomic management policies implemented are *best effort*. In case the user asks for a contract that could not be achieved due to limitation of the target architecture or of the parallelization chosen for the application at hand, it will stop to the closest performance that can be achieved with the parallel pattern and management mechanisms available
- the autonomic management is actually *reactive*, in that it only reacts to situations, rather than trying to anticipate problems.

10.1.2 Hierarchical management

Up to now, we only considered original behavioural skeletons, as defined within CoreGRID and GridCOMP EU funded research projects. These behavioural skeletons were implemented in such a way that they could be functionally composed (e.g. a pipeline stage could be implemented with a FRBs) but no coordination among managers of different behavioural skeletons was supported.

Later on, the possibility to implement *hierarchical management* in behavioural skeleton nesting has been explored. The idea was to implement policies suitable to implement performance (the first non functional concern considered) as a whole in the overall nesting of behavioural skeletons used to implement the user application. This means:

- to have a single user defined performance contract. Formerly, each behavioural skeleton needed its own performance contract, not related to the contracts provided to the other behavioural skeletons appearing in the application.

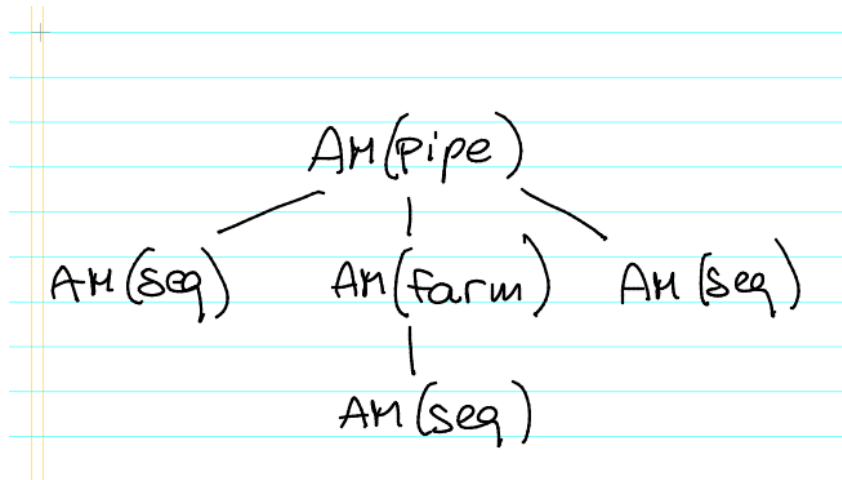


Figure 10.4 Sample manager tree

- to be able to derive from the unique, application related user provided contract contracts and policies suitable to manage all the behavioural skeletons of the application
- to be able to coordinate manager policies according to the hierarchical composition of managers deriving from the hierarchical composition of behavioural skeletons in the user application.

E.G.▷ Consider the manager tree of Fig. 10.4. This tree corresponds to the managers appearing in a behavioural skeleton tree (nesting) such as:

$$\text{pipe}(\text{seq}(C1), \text{pipe}(\text{farm}(\text{seq}(C2)), \text{seq}(C3)))$$

When designing coordinated, hierarchical autonomic management of performance in this behavioural skeleton composition we wish:

- to be able to supply a single performance contract related to the overall application. E.g. a single contract stating that the application must be implemented with a service time of at most t_s .
- to be able to derive, from the single performance contract, one “local” performance contract for each one of the managers in the tree. These local contract are the ones that, once satisfied, ensure the achievement of the overall, application performance contract.
- in case of failure of one of the managers, i.e. in case the manager does not succeed ensuring its local performance contract, the manager should inform its “parent manager” of the failure, in such a way more global adaptation actions can be taken, possibly involving issuing a new local contract for this manager.

■

We will now discuss the different problems related to hierarchical, coordinated non functional feature management in behavioural skeletons.

10.1.2.1 Contract propagation The first problem is contract propagation. How can we derive local contracts for skeletons used as parameters of a parent skeleton, once the contract relative to the parent skeleton is known? In general, the process depends on both the type of the skeletons involved and in the type of non functional concern managed.

The specification of the contract propagation rules is therefore a task of the manager designer that uses knowledge and expertise from the parallel pattern designer.

The main rule to implement while designing contract propagation can be stated as follows:

for any skeleton $S(\Delta_1, \dots, \Delta_k)$, given a contract C_S k contracts $C_{S_1}, \dots, C_{S_k}$ have to be derived. Each new contract C_{S_i} will be a function of C_S , $\text{type}(S)$ and $\text{type}(\Delta_i)$ and it will be such that, if skeleton (manager of skeleton) Δ_i ensures C_{S_i} , for any $i \in \{1, k\}$ then C_S is automatically ensured.

We assume, of course, that the user provided performance contract is actually the topmost skeleton initial contract.

E.G.▷ To exemplify the process, we consider again performance contracts in the form of required service times and a (behavioural) skeleton framework supporting pipelines and farms, that is, stream parallel only skeletons.

The contract propagation rule is defined by induction on the skeleton type as follows:

Pipeline In a pipeline with service time contract t_s ⁷⁷ the local contracts of all the pipeline stages are the same as the pipeline contract. This follows from the performance model of pipeline service time stating that the service time of a pipeline is the maximum among the service times of the pipeline stages.

Farm In a farm with service time contract t_s the service time of the workers is set to $t_s \times n_w$, with n_w being the number of workers in the farm. This follows from the performance model of the farm stating that the service time of the farm is the maximum among the emitter service time, the collector service time and the worker service time divided by the number of workers, under the assumption the emitter and the collector are not bottlenecks in the farm implementation.

Seq A sequential portion of code wrapped in a sequential skeleton inherits the service time contract of its parent skeleton (as it happens for pipeline stages).

■

It is clear that although this rules must be eventually implemented by the manager designer, they definitely require all the knowledge available from the managed parallel pattern designer. The result of application of this contract propagation rule to the behavioural skeleton composition of Fig. 10.4 is the one illustrated in Fig. 10.5.

E.G.▷ Consider now the same skeleton framework, but assume managers must take care of computation security. In particular, assume that security in this contexts means ensuring confidentiality and integrity of data and or code processed and used by the application. In this case, the contract propagation rule may be simply stated as follows:

- the contract of the parent skeleton is inherited by nested skeletons.

This follows from the features of the particular security non functional concern considered. It is a binary property: either it holds or it does not hold. Therefore if the user asks the application to be secure, all the skeleton used must ensure “local” security. — ■

10.1.2.2 Manager coordination The second relevant problem related to coordinated and hierarchical management of non functional features in a behavioural skeleton composition is even more important. It consists in the design of the coordinated management policies to be run among the different managers involved in the behavioural skeleton composition.

In this case we must devise proper policies to handle impossibility for a behavioural skeleton manager to ensure its local contract. This is the only case of interest as when all the managers succeed ensuring their local contract, the global contract is ensured “by construction”, that is by the procedure used to derive the local contracts.

⁷⁷that is required to guarantee at a service time $T_S(\text{pipe}) = t_s$

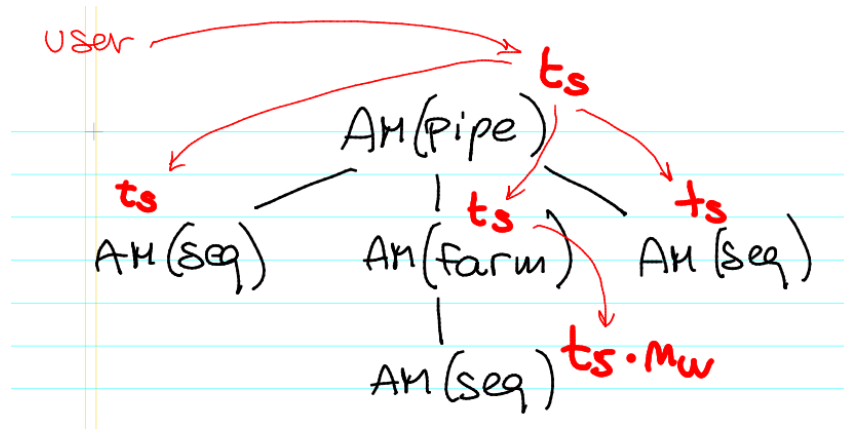


Figure 10.5 Sample performance contract propagation

Once again, the handling of the manager fault in ensuring the local contract must be properly set up by the autonomic manager designer and requires specific knowledge relative to parallel pattern and to the particular non functional feature managed. The general policy to handle local failures can be stated as follows:

A manager failing to ensure the local contract keeps maintaining its current configuration and reports the local contract violation to the parent manager. “Keep the current configuration” means that no more adaptation decisions are taken up to the moment a new contract will be received from the parent manager. This “wait” state is called manager passive mode.

A manager receiving from a “son” manager a contract violation message will re-formulate its own management policy in such a way the sub-manager local contract violation could be accommodated. In case the re-formulation succeeds, the manager will deliver new performance contracts to all of its “son” managers. In case it does not succeed, the violation will be recursively reported to parent manager.

In case a contract violation is eventually reported by the top level manager, this is simply exposed to the user that may decide whether to continue in the execution of the “sub-performing” application or to stop it.

E.G.▷ To show how this policy may be implemented, we take again into account the behavioural skeleton composition of Fig. 10.4 and the same stream parallel behavioural skeleton framework considered when discussing contract propagation. Let us assume the manager of the first sequential skeleton (the first pipeline stage) may vary the rate producing result items onto the output stream, and let us assume an initial configuration of the program (that is an initial parameter n_w , the number of the workers in the second stage farm).

In case the processing elements that run the farm workers become overloaded because of some external program run concurrently with our skeleton application, the farm would eventually verify a service time no more ensuring the performance contract received.

The farm manager could apply the policy whose effect final effect is recruiting new resources and allocating more farm workers on these resources. If these resources may be actually recruited, the farm manager may locally re-ensure its contract. In case no new resources may be recruited⁷⁸, the performance manager of the farm will block any further adaptation process and communicate the local contract violation to the parent manager, that is to the manager of the pipeline skeleton.

The pipeline skeleton will therefore be made aware of a situation such that the first and the last stages perform better than the second one. As this is not useful at all, as the performance

⁷⁸e.g. in case there are no additional resources available

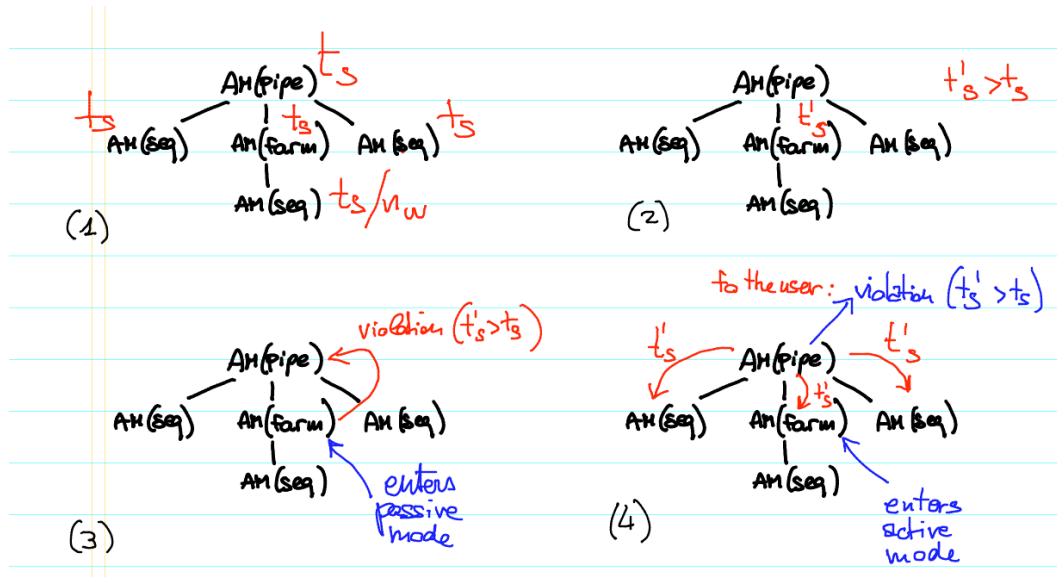


Figure 10.6 Hierarchical management of a contract violation

model of the pipeline states the pipeline service time is the maximum of the service times of its stages, the pipeline manager may resend to all its stages a new performance contract stating the service time should be the one currently provided by the second stage farm.

As a consequence, the first stage could decrease its delivery rate. Being a sequential stage, this does not actually save any processing element resource as it will happen if the first stage was a farm, as an example⁷⁹. However, it may impact power consumption of the node or even this may allow to move the node on a less performant—and therefore less “expensive”—processing node⁸⁰.

In case the farm manager could not cooperate with its parent manager, instead, the violation of the contract would be confined to the farm and therefore no resource optimization could have been achieved.

It is worth pointing out that the violation eventually observed by the pipeline manager must be reported to the application user. The violation may be reported as a *non functional exception*. As for any exception, it can be “trapped” and managed by the user or explicitly ignored. ■

The example just discussed only details what can be done by the manager hierarchy in presence of a violation of a contract that cannot be solved. A much more interesting case is the one discussed below, concerning a situation when a local contract violation may actually be solved by the hierarchy of managers.

E.G.▷ We consider the same situation discussed in the last example. The user asked for a service time $T_S = t_s$. After a while, all the three stages satisfy their “local” contract that happens to be the same as the user supplied one, due to pipeline semantics.

Some overload on the node where the first stage is allocated makes the first stage unable to match its local contract anymore.

⁷⁹in case it was a farm, the decrease in the service time allowed by the new contract could lead to the release of resources used for one or more workers

⁸⁰the situation is shown in Fig.10.6

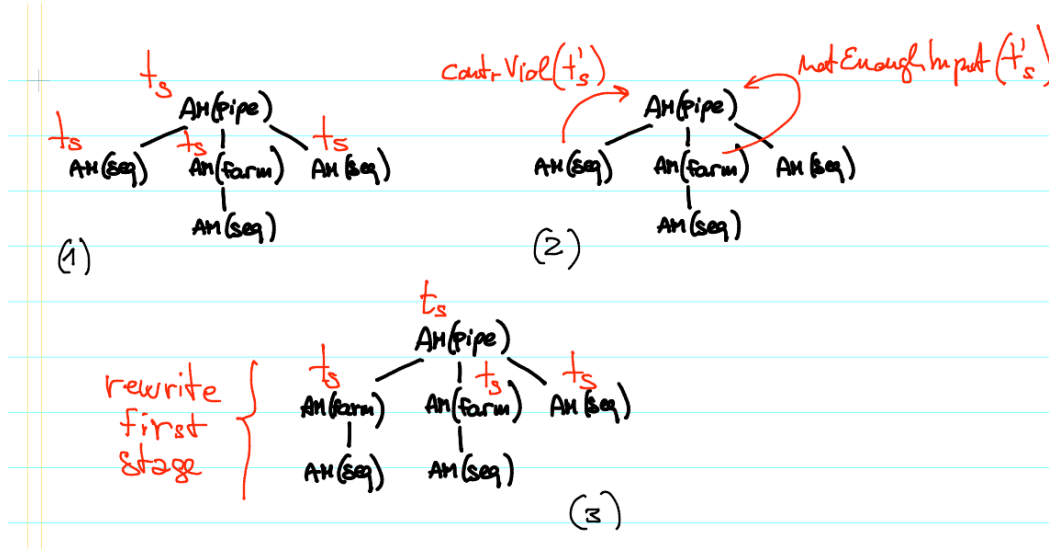


Figure 10.7 Skeleton tree rewriting in hierarchical management

The first stage autonomic manager enters the passive mode and raises a violation exception to the pipeline manager. This happens as there is no way for a sequential code to improve its performance⁸¹.

The violation of local contract by the first stage impacts also the second stage farm. In this case the farm will not be able to deliver the service time t_s required by its local contract as the inter arrival time of the tasks to be computed will be at most t'_s which is larger than t_s and no matter the number of workers included in the farm the farm service time will always be bound by this t'_s .

Now the pipe manager will get the two violations from first and second stage. The first one will be a simple `ContractViolation( $t'_s$ )` while the second one should be something different expressing the fact the violation is due to the behaviour of the modules external to the farm, such as a `NotEnoughInput( $t'_s$ )`. The pipe manager may therefore decide to improve the service time of the first stage by transforming this `seq(C1)` stage into a `farm(seq(C1))` stage. This eventually will lead to the full ensuring of the local contract at the first stage and, consequently, at the second stage also. This situation is depicted in Fig. 10.7. ■

10.1.3 Multi concern management

When considering behavioural skeletons and autonomic management of non functional features in parallel applications we should take into account that normally more than a single non functional concern is of interest relatively to parallel applications.

Notable non functional concerns to be taken into account, along with the main management goals are listed in Table 10.1. The question we must answer now is “can we design manager policies that allow more than a single non functional concern to be managed by the same manager hierarchy?” This would constitute a big improvement of the results discussed in the previous Section and relative to the autonomic management of a single non functional concern coordinated through a hierarchy of managers.

⁸¹we are taking the black-box approach here, that is we assume that no intervention is allowed on the sequential code wrapped in skeleton parameters.

Non functional concern	Main goals in management
Performance	Best performance independently of the resources used
Security	Code and data integrity and confidentiality
Fault tolerance	Tolerate limited amount of faults
Power management	Lowest power consumption

Table 10.1 Non functional concerns of interest

However, if we take again into account separation of concerns, another question comes immediately in mind: how may we force security and performance experts to work together to implement a single manager (hierarchy) dealing with both security *and* performance issues? It would be much better if complementary approach is followed: performance experts design performance policies and security experts design security policies. The two sets of policies are implemented in two independent manager hierarchies. The two hierarchies are somehow coordinated in such a way no conflicting “adaptation” decisions are taken.

This will of course a better solution:

- experts will only be concerned by their own field of interest issues, and therefore they will probably come out with excellent (the best possible?) policies.
- independent managers may be built each monitoring (triggered by) non functional concern specific values (events) and operating on the parallel pattern by non functional concern specific actions. As the monitor/trigger and actuator interface are independent, we expect a much simpler implementation.

However, completely independent manager hierarchies are not a good solution, nor coordinating independently developed manager is a simple task.

E.G.▷ To illustrate the concept, assume two managers have been associated to each behavioural skeleton pattern, one taking care of performance and one taking care of security. The performance manger may decide a new resource is to be acquired to allocate a new worker that hopefully will improve some functional replication behavioural skeleton performance. The resource will be looked for, recruited, the worker allocated and eventually linked to the current computation. The computation happens to be the very same also controlled by the security managers. In case the new resource has been recruited from some “secure” domain (e.g. a cluster under the complete control of the user running the application, security requirements may be satisfied at no further cost. No “alien” users are supported by the cluster and therefore no (new) risks arise related to security of user application code and data. However, in case recruited resource belongs to some untrusted domain, or even in case it needs to be reached through some non secure interconnection link, then these nodes should not be used, in the perspective of the security manager of, if used, the communications to and from these nodes should be performed in some secure way. This means that in this case the decision taken by the performance manager is *in contrast* with the policies of the security manager. ■

E.G.▷ Another case illustrating the issues related to concurrent autonomic management of different non functional concerns is the following one. Assume two manager take care of performance and power management issues in a parallel computation. The power management will constantly look for more power efficient processing elements to allocate parallel computation activities. Notably, power efficient processing elements provide worst absolute performance than power hungry processing elements. If you execute the same code on a Xeon or on an Atom, you will probably eventually consume less power when using the Atom, but you also

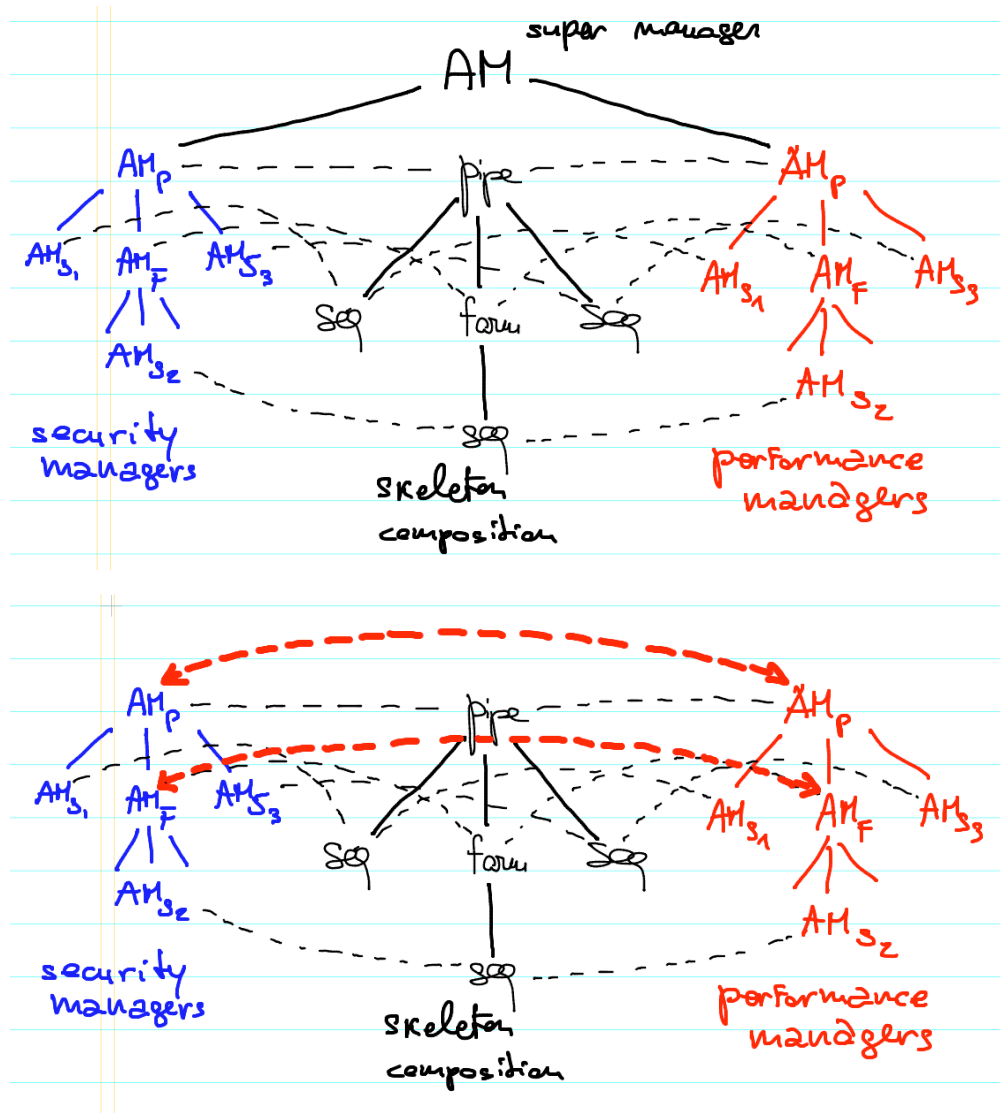


Figure 10.8 Alternative manager orchestration alternatives

spent much more time waiting for your application to complete. Therefore the performance manager will constantly look for more powerful processing elements to allocate the parallel computation activities. If the two managers are run concurrently on the same computation this will result in one manager destroying the other manager decisions. In turn this represents a complete waste of resources (time and power).

In order to try to *orchestrate* different managers in such a way some globally “good” behaviour is achieved aiming at reaching the goals of *all* the managers involved, we can take two different ways:

W1 we use a *super manager*, that is a manager coordinating the decisions of the multiple manager hierarchies used in our application. The situation is the one depicted in Fig. 10.8 (upper part). In this case, the super manager should coordinate adaptation actions of the two manager hierarchies. This means that each manager hierarchy should “ask permission” to the super manager to commit any adaptation action

planned. In turn, the super manager may ask consent to allow the commitment of a given planned action to the other manager hierarchy.

W2 we don't use a super manager, but we make managers instances relative to the same parallel pattern to coordinate each other (*layered coordination*). The situation is the one depicted in Fig. 10.8 (lower part). In this case, a manager deciding to take some adaptation action should "ask permission" to the other(s) manager(s) taking care of the same parallel pattern before committing the action execution.

In both cases, therefore, the key point consists in the *establishment of some mutual agreement protocol* in order to avoid that adaptation actions are taken that impair management policies of some of the involved managers (manager hierarchies).

10.1.4 Mutual agreement protocol

The mutual agreement protocol among a set $\mathcal{M}_1, \dots, \mathcal{M}_m$ of managers should be designed such that:

- no adaptation actions are taken by any $\mathcal{M}_j$ that may impair the policies of another manager $\mathcal{M}_i$
- in case, the manager $\mathcal{M}_i$ whose policies may be impaired by a decision taken by manager $\mathcal{M}_j$ may grant $\mathcal{M}_j$ the permission to execute the planned action provided it is modified to accomplish $\mathcal{M}_i$ policies too.

In case we adopt a layered coordination approach, a simple mutual agreement protocol may therefore be designed as follows:

Decision step Manager $\mathcal{M}_i$ analyzes the current behaviour of the managed parallel pattern and decides an action A should be executed. This decision is taken on the exclusive basis of the policies implemented by manager $\mathcal{M}_i$.

Consensus "ask" step Manager $\mathcal{M}_i$ communicates the decision taken to other managers $\mathcal{M}_j$ ($j \neq i$) taking care of different non functional concerns for the same parallel pattern. The decision should be communicated in such a way the other managers could appreciate its effects. As the only common knowledge between managers is represented by the parallel structure of the entities participating to the parallel pattern computation, the decision may be suitably represented by the *changes in the current parallel activities graph*. Therefore the communication from manager $\mathcal{M}_i$ to the other managers should include information such as

$$\langle A, G, G' \rangle$$

where G represents current parallel activities graph and G' represents the graph eventually resulting from the implementation of action A .

Consensus "answer" step Each manager $\mathcal{M}_j$ receiving the message from manager $\mathcal{M}_i$ analyzes the effects of decision A according to its own management policies and sends an "answer" message to $\mathcal{M}_i$. The message may be one of three different messages:

1. *OK*

This means that decision A does not anyway impact the management policies of $\mathcal{M}_j$. As far as manager $\mathcal{M}_j$ is concerned, action A may be safely committed by manager $\mathcal{M}_i$.

2. *NOK*

This means that decision A contradicts some management policy of $\mathcal{M}_j$. As far as manager $\mathcal{M}_j$ is concerned, action A *should not* be taken by manager $\mathcal{M}_i$.

3. *provide(P)*

This is the most interesting case, actually. As far as manager $\mathcal{M}_j$ is concerned, action A *may be taken* by manager $\mathcal{M}_i$ provided that the action is taken in such a way property P —of interest of manager $\mathcal{M}_j$ —is granted.

Commit step Manager $\mathcal{M}_i$ gathers all the answers from the other managers and then:

- in case the answers are all of type *OK*, it commits the action, that is it goes on executing the planned action
- in case at least one answer of type *NOK* has been received, the action is aborted. Possibly, the policies that led to the decision of taking action A are given a lower priority, in such a way the very same decision is not chosen again in the immediate future
- in case some *provide(P)* answers has been received, $\mathcal{M}_i$ tries to modify action A in such a way property P is granted. This produces action A' , as a result. If an A' satisfying property P (or the collection of $P_1, \dots, P_k$ relative to k *provide* messages received) may be computed, it is eventually committed, otherwise the decision is aborted and the relative policy priorities lowered, as in the *NOK* case.

In case we adopted a super manager approach, instead, a similar protocol may have been implemented in the super manager. In that case, each local manager deciding to implement adaptation action A will communicate the action plan to its parent manager. Eventually, root manager will communicate the decision to the super manager. The super manager will implement the mutual agreement protocol as described above, querying the other top level managers and collecting answers, and eventually will communicate the decisions to the querying top level manager. This, in turn, will propagate the decision to the manager originally raising the request for agreement on action A .

E.G.▷ In order to better understand how this agreement protocol works, consider the following example. Two manager hierarchies take care of the same behavioural skeleton composition implementing a given parallel application. One manager hierarchy takes care of performance and one of security. Performance contracts are such that:

- a service time of at least t_s is required
- both code and data confidentiality and integrity is required

Let us suppose a farm performance manager decides it is under performing with respect to the contract received and therefore a new worker may be added. Therefore it enters the recruiting process. Suppose resource RW_{new} is found. If

$$G = (N, A)$$

with

$$N = \langle E, W_1, \dots, W_{nw}, C \rangle$$

and

$$A = \langle \langle E, W_1 \rangle, \langle E, W_2 \rangle, \dots, \langle E, W_{nw} \rangle, \langle W_1, C \rangle, \langle W_2, C \rangle, \dots, \langle W_{nw}, C \rangle \rangle$$

was the current parallel activities graph, than the graph resulting from the actual implementation of the planned action will be

$$G' = (A', N')$$

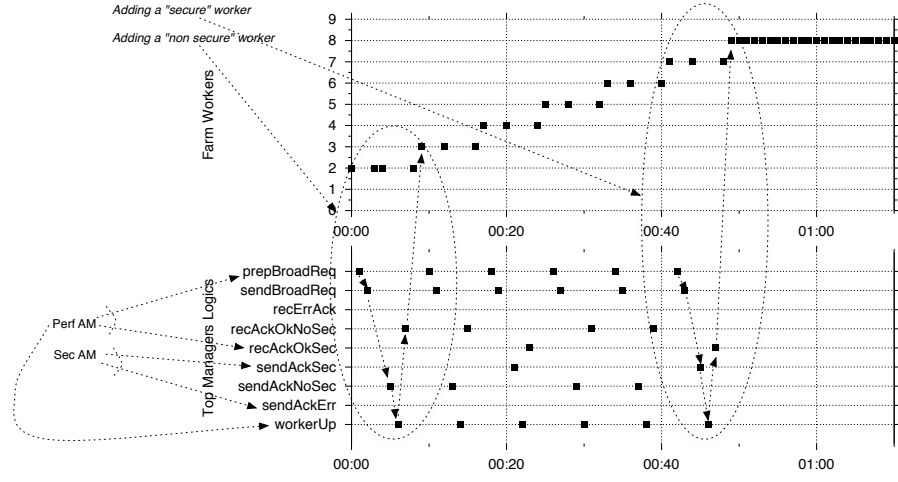


Figure 10.9 Sample trace of mutual agreement protocol operation

with

$$N' = N \cup \{RW_{new}\}$$

and

$$A' = A \cup \{\langle E, RW_{new} \rangle, \langle RW_{new}, C \rangle\}$$

At this point the farm performance manager may send to all the other farm managers the request

$$\langle addWorker, G, G' \rangle$$

Now three cases may be given:

1. the security manager, according to its policies and node security property knowledge decides it is sage to add the new resource RW_{new} , and therefore answers a *OK* message to the farm performance manager, or
2. it decides the new resource RW_{new} is “nasty” for same reason and therefore it answers a *NOK* message to the farm performance manager, or eventually
3. it decides the new resource may be used, provided secure data and code communications are used. In this case it answers a *provide(secureCodeData)* message to the farm performance manager.

Depending on the kind of answer received, the farm performance manager:

1. commits the action, adding RW_{new} to the farm
2. aborts the action and manages to avoid in the near future to try to recruit again resource RW_{new}
3. manages to modify the action A in such a way *secureCodeData* is ensured. Now action A is the high level action *add worker*. The corresponding plan could be something as

$$A = \langle deploy, start, link \rangle$$

where the *deploy* action deploys the worker RTS code to resource NR_{new} , the *start* action starts a worker on that resource and eventually, the *link* action links the new

worker to collector and emitter in such a way it can be used for the farm computation. Ensuring *secureCodeData* property, in this context, means different actions, including at least:

- deploy code that eventually uses secure communication to interact with the emitter and collector processes (e.g. SSL sockets rather than plain TCP/IP ones)
- use secure mechanisms to deploy the code to the remote resource

■

The mutual agreement protocol just discussed is a possible solution to the coordination of multiple managers in behavioural skeletons. It is relatively simple, and it has been demonstrated to be working with stream parallel behavioural skeleton applications with two managers taking care of performance and security. Fig. 10.9 is relative to the execution of a farm application whose performance contract was to have a constant parallelism degree equal to 8. The application was started with a parallelism degree equal to 2. The farm manager therefore started to plan adding more workers. Some of the resources recruited were judged “secure” by the security manager, and therefore they are added without taking any particular action to ensure security in communications, other were judged “not so secure” and therefore a *provide(secureCodeData)* was answered (*recAckOkSec* in the graph) such that the farm manager used secure worker set up (*sendSecAck* in the graph) to add the worker to the farm.

However, the protocol has also some flaws. First of all, the *provide(P)* messages violate somehow the separation of concerns principle. Our performance manager is forced to know about *secure* deployment and communications, which is not in general something he should know of, being a *performance* manager.

Second, in case more than a single *provide(P)* answer is collected the scheduling of the modifications of the planned action may actually impair the feasibility of the modification itself. More in detail, if P_1 and P_2 must be ensured ensuring first P_1 and then P_2 may have different implications and feasibility with respect to ensuring first P_2 and then trying to ensure P_1 on the resulting set of modified actions.

Last but not least, this protocol *de facto* gives priority to negative actions. The *NOK* messages always lead to the abortion of the planned action. This ensures the achievement of the policies of the “forbidding” manager—the one sending the *NOK* message—but may be it impairs the achievement of the management policies of the “forbidden” manager—the one receiving the *NOK* message—which could be in the situation of not having any more alternative actions to take to achieve its goals.

However, this represents the only real agreement protocol studied up to now.

10.1.5 Alternative multi concern management

Actually, multiple concern management has also been studied from a completely different perspective, in parallel programming contexts not using skeletons at all. The approach followed in that case can be summarized as follows.

We consider a number of functionally equivalent application implementations $A_1, \dots, A_a$. Therefore A_i computes the same results of A_j but the results are computed differently as far as non functional behaviours are concerned.

Then we consider the non functional behaviour of interest $\mathcal{NF}_1, \dots, \mathcal{NF}_{nf}$ and we assume a function $f_i(p_1, \dots, p_p)$ estimating “how much” $\mathcal{NF}_i$ is being achieved when application A_k is executed with a given set of parameters $p_1, \dots, p_p$ (application or target architecture related) is also available. We also assume that a linear combination of f_i exists

$$F(p_1, \dots, p_p) = \sum_{i=1, a} a_i f_i(p_1, \dots, p_p)$$

that evaluates the collective “goodness” our set of implementations, that is a weighted sum of the estimated of the goodness of the single non functional concerns achievements on certain application and target architecture parameters.

Eventually, at each adaptation iteration, we evaluate F for each A_j considering the current execution parameters $p_1, \dots, p_p$ and we chose to adopt for the next step the application whose F gives the “best” value.

E.G.▷ In this case, taking into account our sample

`pipe(seq(C1), pipe(farm(seq(C2), seq(C3))))`

application and performance and security non functional concerns, we will have:

- $nf = 2$ and with $\mathcal{NF}_1 = performance$ and $\mathcal{NF}_2 = security$
- the set of equivalent applications including i) different versions of the original application differing in the parallelism degrees relative to the `farm`, ii) different versions of the original application modifies in such a way `seq(C1)` is rewritten as `farm(seq(C1))` differing in the $\langle nw_1, nw_2 \rangle$ parallelism degrees considered for the first and second stage farms, iii) different versions of this last version of the application with `seq(C3)` rewritten as `farm(seq(C3))` differing in the parallelism degrees $\langle nw_1, nw_3, nw_3 \rangle$, iv) different versions of a `farm(pipe(seq(C1), pipe(seq(C2), seq(C3))))` differing in the external farm parallelism degree, v) ...
- function f_1 evaluating performance of a given application. The function evaluates the application with respect to its parallel activities graph mapped on real resources. The function returns, as an example, the service time of the applications.
- function f_2 evaluating security of a given application. The function evaluates the application with respect to its parallel activities graph mapped on real resources. The function returns 1 in case of achievement of the security in the current application, or 0 otherwise.
- two weight parameters a_1 and a_2 . The choice of these parameters is obviously critic. We have to consider that smaller service times are the better ones and that security is a “binary” valued function. As an example, we can pick $a_1 = 1$ and $a_2 = -k$ being k a value larger than the maximum service time possible.
- eventually, we can look for the smaller F among all the applications evaluated on the currently available parameters. In case A_j is the application delivering the best F and A_i is the current one, we should manage to run an adaptation step rewriting A_i as A_j . This may simply mean that we have to increase/decrease the parallelism degree of A_i , if A_i and A_j share the same skeleton tree but differ in the parallelism degree. It can also require a complete rewriting of the application (e.g. to change the application parallel patten from a pipe of farms to a farm of pipes). In this case the adaptation phase will obviously take longer and possibly require an application checkpoint-stop-transform-restart cycle.

■

It is evident that:

- the critical step in this case is the design of F : the choice of proper a_i as well as the choice of the way to evaluate “best” results (minimum, maximum)
- solutions like the one discussed above to handle “binary” concerns may be unfeasible when multiple binary concerns are taken into account, and in general deciding weights is again a critic issue
- at each adaptation step we must consider all the alternative possibilities. This space is huge. When adopting the mutual agreement protocol discussed in Sec. 10.1.4 instead, only the currently proposed application rewriting is to be considered and evaluated by the other managers.

All this points make this alternative multi concern management strategy slightly less effective than the one discussed in Sec. 10.1.4.

CHAPTER 11

SKELETON FRAMEWORK EXTENDABILITY

One of the major criticisms made to algorithmic skeleton frameworks was related to their *poor or absent* extendibility. Traditional algorithmic skeleton frameworks, in fact, do not allow users to intervene on the skeleton set neither to add new skeleton nor even to slightly modify the existing skeletons.

As a result, two kind of situations may arise:

- either the programmer succeeds modeling his/her application with (a composition of) the existing skeletons, or
- there is no way to write the application using the skeleton framework.

This is very bad, actually. The set of skeletons implemented in an algorithmic skeleton programming framework model most common and used parallelism exploitation patterns. However it is quite frequent that in order to achieve the very last bit of performance, programmers usually “fine tune” these patterns in such a way the computation at hand can be perfectly a very efficiently modeled. A programmer that could not use its favorite, super-optimized parallelism exploitation pattern would therefore dislike algorithmic skeleton frameworks.

Cole recognized this problem as one of the most important aspect impairing diffusion of algorithmic skeletons in the parallel programming community. In his “manifesto” paper [19], in fact, he lists four principles that must be ensured in order to allow widespread diffusion of the algorithmic skeleton concept (see Sec. 6.1). One of these principles exactly states that we must

***Integrate ad-hoc parallelism.** Many parallel applications are not obviously expressible as instances of skeletons. Some have phases which require the use of less structured interaction primitives. For example, Cannon’s well-known matrix multiplication algorithm invokes an initial step in which matrices are skewed across processes in a manner which is not efficiently expressible in many skeletal systems. Other applications have*

conceptually layered parallelism, in which skeletal behaviour at one layer controls the invocation of operations involving ad-hoc parallelism within. It is unrealistic to assume that skeletons can provide all the parallelism we need. We must construct our systems to allow the integration of skeletal and ad-hoc parallelism in a well-defined way.

Now in order to “construct our systems to allow the integration of skeletal and ad-hoc parallelism in a well-defined way” we may follow different ways. Each different approach must taken into account the way skeletons are provided to the user/programmer that is both the kind of skeletons provided *and* the implementation model chosen to support these skeletons.

We distinguish two alternative approaches, that are described in the following sections:

1. skeleton set extension in template based frameworks
2. skeleton set extension providing controlled user access to intermediate implementation layer

The two approaches basically differ in the level of access granted to the user/programmer to the internals of the skeleton framework.

11.1 SKELETON SET EXTENSION IN TEMPLATE BASED FRAMEWORKS

When implementing a template based skeleton framework we basically have to implement a set of distinct tools:

- a compiler (or a library), parsing source code (implementing skeleton declaration calls) and building the abstract skeleton tree
- a template library (see Sec. 7.2.1)
- a compiler assigning templates to the skeletons in the skeleton tree and devising proper parameters to instantiate all the templates assigned (e.g. parallelism degree)
- a tool compiling the annotated skeleton tree and deploying the resulting object code onto the target architecture for execution
- a run time system to support execution on the target architecture.

In order to support skeleton set expandability we should guarantee the programmer with:

- the possibility to include new skeletons in the skeleton set
- the possibility to provide implementation templates for the new skeletons

Both points are quite complex to provide. The former requires full access to parsing facilities of the compiler or, in case of implementation of the skeleton framework through a library, the possibility to enter brand new entries in the skeleton library. In both cases, full access to internal skeleton tree data structures must be granted. The latter requires the possibility to access all the mechanisms of the target abstract machine used to implement the skeletons. If this target machine is C++ with POSIX threads and TCP/IP sockets, then the programmer should use these mechanisms to provide the new template(s) corresponding to the new skeleton he wants to add to the system. Moreover, as the new template should be nestable in other templates, full compliance to some general template composition model should be ensured by the new template. All in all, this is a quite huge effort to add a new skeleton into the system.

As far as we know, no algorithmic skeleton framework provides support to extension of the skeleton—and as a consequence, of the template—set as discussed above. Indeed, most

```

Skeleton inc1 = new Inc();
Dest d = new Dest(0, 2, Mdfi.NoGraphId);
Dest[] dests = new Dest[1];
dests[0] = d;
Mdfi i1 = new Mdfi(manager, 1, inc1, 1, 1, dests);
Skeleton sq1 = new Square();
Dest d1 = new Dest(0, Mdfi.NoInstrId, Mdfi.NoGraphId);
Dest[] dests1 = new Dest[1];
dests1[0] = d1;
Mdfi i2 = new Mdfi(manager, 2, sq1, 1, 1, dests1);
MdfGraph graph = new MdfGraph();
graph.addInstruction(i1);
graph.addInstruction(i2);
ParCompute userDefMDFg = new ParCompute(graph);

```

Figure 11.1 Custom/user-defined skeleton declaration.

algorithmic skeleton frameworks are template based and open source. Therefore nothing prohibits to dig the skeleton framework source code and to add new skeletons and templates. But the no framework provides support for this activity and actually internals of the framework implementation are in most cases completely undocumented. This means the programmer wishing to extend the skeleton set must first reverse engineer the framework implementation. This is probably much more than the work required to write a brand new, non skeleton based implementation of the particular parallelism exploitation pattern needed.

11.2 SKELETON SET EXTENSION THROUGH INTERMEDIATE IMPLEMENTATION LAYER ACCESS

An alternative way of providing support for skeleton set extension may be designed when non template based implementations are used. If a macro data flow based implementation is used we can provide the user/programmer with a controlled access to the intermediate implementation layer—the macro data flow graph language. We must expose the user/programmer with the definition of “well formed MDFG”, i.e. of a MDFG that may be used in a skeleton nesting. As discussed in Sec. 7.2.2 a macro data flow graph representing a skeleton is basically any MDFG with a single input token and a single output token. Then the user/programmer should be given a “language” suitable to express these well formed MDFG and a way to name the new MDFG modelling the particular parallelism exploitation pattern needed. The Muskel skeleton library [7] provides such a possibility, and we will described how this is supported in detail in the next section. For the moment being, we want to point out that the process providing skeleton set expandability is much simpler than the one discussed with template based implementations. All what the user needs to know is how to “describe” a well formed MDFG and how to name it in standard skeleton compositions. Then the MDF distributed interpreter takes care of executing the user supplied MDFG with the same efficiency achieved in the execution of MDFGs deriving from the compilation of standard, predefined skeletons.

11.3 USER-DEFINED SKELETONS IN `muskel`

In order to introduce completely new parallelism exploitation patterns, `muskel` provides programmers with mechanisms that can be used to design arbitrary macro data flow graphs. A macro data flow graph can be defined creating some `Mdfi` (macro data flow instruction) objects and connecting them in a `MdfGraph` object.

For example, the code in Fig. 11.1 is the one needed to program a data flow graph with two instructions. The first computes the `incl` compute method on its input token and delivers the result to the second instruction. The second computes the `sql` compute method on its input token and delivers the result to a generic “next” instruction (this is modelled by giving the destination token tag a `Mdfi.NoInstrId` tag). The `Dest` type in the code represents the destination of output tokens as triples containing the graph identifier, the instruction identifier and the destination input token targeted in this instruction. Macro data flow instructions are built by specifying the manager they refer to, their identifier, the code executed (must be a `Skeleton` object) the number of input and output tokens and a vector with a destination for each of the output tokens.

We do not present all the details of arbitrary macro data flow graph construction here (a complete description is provided with the `muskel` documentation). The example is just to give the flavor of the tools provided in the `muskel` environment. Bear in mind that the simple macro data flow graph of Fig. 11.1 is actually the same macro data flow graph obtained by compiling a primitive `muskel` skeleton call such as:

```
Skeleton main = new Pipeline(new Inc(), new Sq())
```

More complex user-defined macro data flow graphs may include instructions delivering tokens to an arbitrary number of other instructions, as well as instructions gathering input tokens from several distinct other instructions. In general, the mechanisms of `muskel` permit the definition of any kind of graph with macro data flow instructions computing sequential (side effect free) code wrapped in a `Skeleton` class. Any parallel algorithm that can be modeled with a data flow graph can therefore be expressed in `muskel`⁸². Non deterministic MDFi are not yet supported (e.g. one that merges input tokens from two distinct sources) although the firing mechanism in the interpreter can be easily adapted to support this kind of macro data flow instructions. Therefore, new skeletons added through the macro data flow mechanism always model pure functions. skeletons in `muskel` programs gives the programmer the possibility to develop the same parallel programs he could develop using more classic programming environments (e.g. MPI), but those explicitly using data

`MdfGraph` objects are used to create new `ParCompute` objects. `ParCompute` objects can be used in any place where a `Skeleton` object is used. Therefore, user-defined parallelism exploitation patterns can be used as pipeline stages or as farm workers, for instance. The only limitation on the graphs that can be used in a `ParCompute` object consists in requiring that the graph has a unique input token and a unique output token.

When executing programs with user-defined parallelism exploitation patterns the process of compiling skeleton code to macro data flow graphs is slightly modified. When an original `muskel` skeleton is compiled, the standard MDF compilation (see Sec. 7.2.2) process is applied. When a user-defined skeleton is compiled, the associated macro data flow graph is directly taken from the `ParCompute` instance variables where the graph supplied by the user is maintained. Such a graph is linked to the rest of the graph according to the rules appropriate to the skeleton where the user-defined skeleton appears.

To show how the whole process works, let us suppose we want to pre-process each input task in such a way that for each task t_i a new task

$$t'_i = h_1(f_1(t_i), g_2(g_1(f_1(t_i))))$$

is produced. This computation cannot be programmed using the stream parallel skeletons currently provided by `muskel`. In particular, current pre-defined skeletons in `muskel` allow only processing of one input to produce one output, and therefore there is no way to implement the graph described here. In this case we wish to process the intermediate

⁸²Note, however, that common, well know parallel application skeletons are already modelled by pre-defined `muskel` Skeletons.

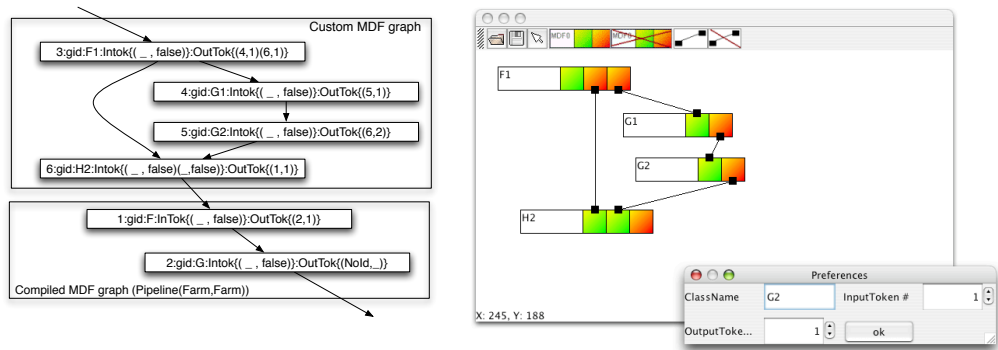


Figure 11.2 Mixed sample macro data flow graph (left): the upper part comes from a user-defined macro data flow graph (it cannot be derived using primitive `muskel` skeletons) and the lower part is actually coming from a three stage pipeline with two sequential stages (the second and the third one) and a parallel first stage (the user-defined one). GUI tool designing the upper graph (right).

results through a two-stage pipeline to produce the final result. To do this the programmer can set up a new graph using code similar to the one shown in Fig. 11.1 and then use that new `ParCompute` object as the first stage of a two-stage pipeline whose second stage happens to be the post processing two-stage pipeline. When compiling the whole program, the outer pipeline is compiled first. As the first stage is a user-defined skeleton, its macro data flow graph is directly taken from the user-supplied one. The second stage is compiled according to the standard (recursive) procedure (such as the one described in Sec. 7.2.2) and eventually the (unique) last instruction of the first graph is modified in such a way that it sends its only output token to the first instruction in the second stage graph. The resulting graph is outlined in Fig. 11.2 (left).

Making good use of the mechanisms allowing definition of new data flow graphs, the programmer can arrange to express computations with arbitrary mixes of user-defined data flow graphs and graphs coming from the compilation of structured, stream parallel skeleton computations. The execution of the resulting data flow graph is supported by the `muskel` distributed data flow interpreter in the same way as the execution of any other data flow graph derived from the compilation of a skeleton program. At the moment the `muskel` prototype allows user-defined skeletons to be used as parameters of primitive `muskel` skeletons, but not vice versa. We are currently working to extend `muskel` to allow the latter. There is no conceptually difficult step behind. In order to allow primitive `muskel` skeleton usage as code to be executed in an instruction of a user-defined macro data flow graph it is sufficient to compile “on the fly” the primitive skeleton and include in the user-defined macro data flow graph the result (i.e. the macro data flow graph) of this compilation.

While the facility to include user-defined skeletons provides substantial flexibility, we recognize that the current way of expressing new macro data flow graphs is error prone and not very practical. Therefore we have designed a graphic tool that allows users to design their macro data flow graphs and then compile them to actual Java code as required by `muskel` and shown above. Fig. 11.2 (right) shows the interface presented to the user. In this case, the user is defining the upper part of the graph in the left part of the same Figure. It is worth pointing out that all that is needed in this case is to connect output and input token boxes appropriately, and to configure each MDFi with the name of the sequential `Skeleton` used. The smaller window on the right lower corner is the one used to configure each node in the graph (that is, each MDFi). This GUI tool produces an XML representation of the graph. Then, another Java tool produces the correct `muskel` code implementing the macro data flow graph as a `muskel` `ParCompute` skeleton. As a result,

```

public class Map2 extends ParCompute {

    public Map2(Skeleton f, Manager manager) {
        super(null);
        // first build the empty graph
        program = new MdfGraph();
        // build the emitter instruction
        Dest [] dds1 = new Dest[2];
        dds1[0]=new Dest(0,2);
        dds1[1]=new Dest(0,3);
        Mdfi emitter =
            new Mdfi(manager, 1,
                new MapEmitter(2), 1, 2, dds1);
        // add it to the graph
        program.addInstruction(emitter);
        // build first half map Skeleton node
        Dest [] dds2 = new Dest[1];
        dds2[0] = new Dest(0,4);
        Mdfi if1 = new Mdfi(manager,2, f, 1, 1, dds2);
        program.addInstruction(if1);
        // build second half map Skeleton node
        Dest [] dds3 = new Dest[1];
        dds3[0] = new Dest(1,4);
        Mdfi if2 = new Mdfi(manager,3, f, 1, 1, dds3);
        program.addInstruction(if2);
        Dest[] ddslast = new Dest[1];
        ddslast[0] = new Dest(0,Mdfi.NoInstrId);
        Mdfi collector = new Mdfi(manager,4,new
            MapCollector(), 2, 1, ddslast);
        program.addInstruction(collector);
        return;
    }
    ...
}

```

```

public class SampleMap {
    public static void main(String[] args) {
        Manager manager =
            new Manager();
        Skeleton worker = new Fdouble();
        Skeleton main =
            new Map2(worker,manager);

        InputManager inManager =
            new DoubleVectIM(10,4);
        OutputManager outManager =
            new DoubleVectOM();

        ParDegree contract =
            new ParDegree(10);
        manager.setInputManager(inManager);
        manager.setOutputManager(outManager);
        manager.setContract(contract);
        manager.setProgram(main);

        manager.compute();
    }
}

```

Figure 11.3 Introducing a new, user-defined skeleton: a map working on vectors and with a fixed, user-defined parallelism degree.

users are allowed to extend, if required, the skeleton set by just interacting with the GUI tool and “compiling” the graphic MDF graph to `muskel` code by clicking on one of the buttons in the top toolbar.

As a final example, consider the code of Fig. 11.3. This code outlines how a new `Map2` skeleton, performing in parallel the same computation on all the portions of an input vector, can be defined and used. It is worth pointing out how user-defined skeletons, once properly debugged and fine-tuned, can simply be incorporated in the `muskel` skeleton library and used seamlessly, as the primitive `muskel` ones, but for the fact that (as shown in the code) the constructor needs the manager as a parameter. This is needed so as to be able to link together the macro data flow graphs generated by the compiler and those supplied by the user. data flow graph creation to the moment the graph needs to be instantiated after the arrival of a new task to compute, as at that time all the information necessary to perform graph “conjunction” is available. It is worth noting that skeletons such as a general form of `Map` are usually provided in the fixed skeleton set of any skeleton system and users usually do not need to implement them. However, as `muskel` is an experimental skeleton system, we concentrate the implementation efforts on features such as the autonomic managers, portability, security and expandability rather than providing a complete skeleton set. As a consequence, `muskel` has no predefined map skeleton and the example of user defined skeleton just presented suitably illustrates the methodology used to expand the “temporary” restricted skeleton set of the current version of `muskel` depending on the user needs. The `Map2` code shown here implements a “fixed parallelism degree” *map*, that is the number of “workers” used to compute in parallel the skeleton does not depend on the size of the input data. It is representative of a more general `Map` skeleton taking a parameter specifying the number of workers to be used. However, in order to support the implementation of

a map skeleton with the number of workers defined as a function of the input data, some kind of support for the dynamic generation of macro data flow graphs is needed, which is not present in the current `muskel` prototype. provide users a more handy way to build the user-defined data flow graphs. With the tool, graphs can be built on the screen and all the work needed to arrange `Dest` code in the instructions, which is definitely the more cumbersome and error prone stuff related to graph designing, can simply be generated with a few mouse clicks connecting with arrows source and destination tokens.

APPENDIX A

MEASURES

When dealing with parallel computations we are in general interested in achieving “the best performance” possible. This expectation may be read in several different and complementary ways:

- First of all, we are interested in knowing which kind of performance we can achieve when using a given parallelism exploitation pattern—or skeleton composition—*before* actually starting writing application code. In case expected performance of our pattern doesn’t match our requirements we can simply avoid to go on implementing a useless application.
- In case several different¹ implementations are possible, we are interesting in knowing which one will provide the better performances. Alternatively, we may be interested in knowing which ones will eventually provide performance figures matching our performance requirements, in such a way we can go on implementing the “simplest”—e.g. the less expensive in terms of coding effort—one.
- Last but not least, we may be interested in knowing how much overhead is added by our implementation, that is how much the measured performance will eventually differ from the theoretic one.

In all cases, we need to have some kind of *performance model* of our application available. The performance model measures the performance of an application on a given target machine in function of a full range of parameters depending either on the application features (parallelism exploitation patterns used, time spent in the execution of the sequential portions of code, dimensions of the data types involved, etc.) or on the target machine features

¹different as far as parallelism exploitation is concerned, but functionally equivalent

(communication times, processing power of the different processing elements, topology of the interconnection network, features of the (shared) memory subsystem, etc.).

In other words, we need a function $\mathcal{P}$ mapping values of parameters $p_1, \dots, p_n$ to values of some performance indicator. More precisely we are interested in a set of functions $\mathcal{P}_1, \dots, \mathcal{P}_h$ each modeling a different performance indicator (e.g. service time, latency, completion time, etc.).

Once these functions have been defined, we can use these performance model functions to

- *predict* performance of an application,
- *compare* alternative implementations of an application, or
- *evaluate* the overheads of an implementation of an application.

Of course, the evaluation of the overheads in the implementation of an application also requires some methods to *measure* the performance values achieved on the target architecture (see Sec. A.9).

A.1 MODELING PERFORMANCE

We start taking into account which kind of performance indicators are useful. When dealing with performance of parallel applications we are in general interested in two distinct kind of measures:

- those measuring the absolute (wall clock) time spent in the execution of a given (part of) parallel application, and
- those measuring the throughput of the application, that is the rate achieved in the delivering of the results.

The first kind of measures include measures such as

Latency (L) the time spent between the moment a given activity receives input data and the moment the same activity delivers the output data corresponding to the input.

Completion time (T_C) the overall latency of an application computed on a given input data set, that is the time spent from application start to application completion.

The second kind of measures include measures such as

Service time (T_S) the time intercurring between the delivery of two successive output items (or alternatively, the time between the acceptance of two consecutive input items), and

Bandwidth ($\mathcal{B}$) the inverse of the service time.

These are the basic performance measures of interest in parallel/distributed computing. Applications with a very small service time (a high bandwidth) have a good throughput but not necessarily small latencies. If we are interested in completing a computation within a deadline, as an example, we will be interested in the application completion time rather than in its service time.

However, when modeling performance of parallel applications we are also interested in some derived² performance measures such as:

²i.e. non primitive, expressed in terms of other base measures

Speedup ($s(n)$) the ratio between the *best known* sequential execution time (on the target architecture at hand) and the parallel execution time. Speedup is a function of n the parallelism degree of the parallel execution.

$$s(n) = \frac{T_{seq}}{T_{par}(n)}$$

(here we denote with T_{seq} the best sequential execution time and with $T_{par}(n)$ the parallel execution time with parallelism degree equal to n).

Scalability ($scalab(n)$) the ratio between the parallel execution time with parallelism degree equal to 1 and the parallel execution time with parallelism degree equal to n .

$$scalab(n) = \frac{T_{par}(1)}{T_{par}(n)}$$

Both these measures may be computed w.r.t. service time or latency/completion time, that is the T_x in the formulas may be consistently instantiated by service times or by completion times, provided we do not mix the two.

Another “derived” measure useful to model performance of parallel computations is efficiency:

Efficiency (ϵ) the ratio between the ideal execution time and actual execution time.

$$\epsilon(n) = \frac{T_{id}(n)}{T_{par}(n)}$$

As the ideal execution time may be expressed as the sequential execution time divided by parallelism degree³

$$T_{id}(n) = \frac{T_{seq}}{n}$$

then we can express alternatively efficiency as follows:

$$\epsilon(n) = \frac{T_{seq}}{nT_{par}(n)} = \frac{s(n)}{n}$$

A.1.1 “Semantics” associated with performance measures

Each performance measure has an associated “semantics”.

L , T_C the latency and the completion time represent “wall clock time” spent in computing a given parallel activity. We are interested in minimizing latency when we want to complete a computation as soon as possible.

T_S , $\mathcal{B}$ service time represents (average) intervals of time incurring between the delivery of two consecutive results (acceptance of two consecutive tasks). We are interested in minimizing service time when we want to have results output as frequently as possible but we don’t care of latency in the computation of a single result. As bandwidth is defined as the inverse of the service time, we are interested in bandwidth maximization in the very same cases.

$s(n)$ Speedup gives a measure of how good is our parallelization with respect to the “best” sequential computation. Speedup asymptote is given by $f(n) = n$. This because, if

³this is the best we can do with n parallel “executors”, actually: evenly divide the overall work among the parallel executors, without adding any kind of additional overhead.

$s(n) > n$ then the portions of the algorithm executed in parallel on the n processing elements will take a time $t_p < \frac{T_{seq}}{n}$ and therefore we could have performed sequentially the n computations taking t_p on a single processing element, taking an overall time $t = nt_p < n \frac{T_{seq}}{n} < T_{seq}$ which is in contradiction with the assumption T_{seq} was the “best” sequential time. Therefore we will never expect to have speedups larger than the parallelism degree. Actually, there are several reasons that make possible the achievement of *super linear speedups*⁴. In parallel computing we will be constantly looking for higher speedups, of course.

scalab(n) Scalability looks slightly different than speedup. It measures how efficient is the parallel implementation in achieving better performances on larger parallelism degrees. As for the speedup, the asymptote is $f(n) = n$ for the very same reasons. However, scalability does not provide any comparison with the “best” sequential computation. In fact, we can have optimal scalability and still spend more time than the time spent computing sequentially the same program for any reasonable value of n .

$\epsilon(n)$ Efficiency measures the ability of the parallel application in making a good usage of the available resources. Being the speedup bound by $f(n) = n$ the efficiency is bound by $f(n) = 1$. It must be clear that if we have two parallel implementations I_1 and I_2 of the same application with efficiencies ϵ_1 and ϵ_2 , provided that $\epsilon_1 > \epsilon_2$ we can’t conclude that implementation I_1 is “better” than implementation I_2 . This is for sure true if “better” refers to efficiency. But if we are interested in completion time, as an example, it may be the case that the more efficient implementation also gives a larger completion time. We should not forget that a sequential implementation has $\epsilon(n) = 1$. In fact in this case $n = 1$ and therefore

$$\epsilon(1) = \frac{T_{seq}}{nT_{seq}} = \frac{T_{seq}}{1T_{seq}} = 1$$

Of course, if ϵ_1 and ϵ_2 refer to the same parallelism degree n , then $\epsilon_1 > \epsilon_2$ actually means that I_1 is “better” than implementation I_2 . In fact, in this case we have

$$\begin{aligned} \epsilon_1 &> \epsilon_2 \\ \frac{T_{seq}}{n \times T'_{par}(n)} &> \frac{T_{seq}}{n \times T''_{par}(n)} \\ \frac{1}{T'_{par}(n)} &> \frac{1}{T''_{par}(n)} \\ T'_{par}(n) &< T''_{par}(n) \end{aligned}$$

A.2 DIFFERENT KIND OF MODELS

When using performance models with skeleton based parallel programming frameworks, we are obviously interested in deriving *usable* performance models of the skeletons provided by the framework. These performance models will be hopefully *compositional* in such a way the performance of a given application expressed in terms of a skeleton nesting can be

⁴as an example, this may be due to the fact that input/processed data does not fit cache sizes in sequential execution while it fits in each one of the parallel activities in which the sequential program is eventually split. The gain due to the cache hits substituting the main memory accesses could be larger than the overhead necessary to set up the parallel activities and to distribute then the data partitions.

derived by properly composing the performance models of the skeletons appearing in the nesting.

The skeleton performance models, in general, are strictly dependent on the implementation of the skeletons on the target architecture. In turn, the implementation of a skeleton will probably have itself a performance model.

Therefore we can refine our concept of “skeleton performance model” by distinguishing between several different performance models:

Skeleton performance models These are *abstract* models of the theoretic peak performance we can get out of a skeleton. They are abstract in that they do not depend on the implementation of the skeleton but rather depend only on the parallel features of the parallelism exploitation pattern modeled by the skeletons.

E.G.▷ A performance model of the pipeline service time states that the service time of the pipeline is the maximum of the service times of the pipeline stages, as it is known that the slowest pipeline stage sooner or later⁵ becomes the one behaving as the computation bottleneck.

If we consider a task farm, instead, the service time of the farm is given by the service time of the workers (T_w) in the farm divided by the number of workers (n_w), as hopefully n_w results will be output by the workers every T_w units of time. ————— ■

These performance models only take into account “abstract” measures, such as the service time of a stage or of a worker. They do not take into account overheads introduced by communications between pipeline stages, for instance. In other terms, they do take into account actual implementation of the skeletons. They model the theoretical performance provided by the skeleton.

Template performance models These are much more concrete models that express *the performance of a template implementing a skeleton on a given target architecture*. As we are considering implementation on a particular target architecture these performance models assume to know details (parameters, actually) modeling typical mechanisms of the target architecture, such as latency and bandwidth of communications, of accesses to memory, etc.

E.G.▷ The template performance model of a pipeline on a complete interconnection target architecture (e.g. a cluster of workstations with an Ethernet interconnection network) takes into account the communication overhead in addition to the other parameters taken into account by the pipeline skeleton performance model. Therefore, while stating that the service time of the template is the maximum of the service times of the pipeline template stages, the service time of each stage is given⁶ by the latency of the stage *plus* the time spent to complete the communication needed to receive the input data *plus* the time spent to complete the communications needed to deliver results to next pipeline stage. Now let us assume that our pipeline has two sequential stages only with latencies L_1 and L_2 respectively. And let us assume that communication of input data to stage one takes t_{c1} , of output from stage one to stage two t_{c2} and of final results t_{c3} . Then the *skeleton performance model* for service times will state that⁷

$$T_{S_{pipe}}^{sk}(2) = \max\{T_{S_1}, T_{S_2}\} = \max\{L_1, L_2\}$$

whereas the template performance model for the linear chain process template will state that

$$T_{S_{pipe}}^{templ}(2) = \max\{(T_{S_1} + t_{c1} + t_{c2}), (T_{S_2} + t_{c2} + t_{c3})\} > T_{S_{pipe}}^{sk}(2)$$

⁵depending on the amount of buffering among the different pipeline stages

⁶we assume to have sequential stages only

⁷service time of a sequential stage is equal to its latency

and therefore efficiency of the template is smaller than the efficiency (theoretic) of the skeleton.

Now, in case the target architecture is such that communication and computation times may be overlapped⁸ we will have

$$T_{S_{pipe}}^{templ}(2) = T_{S_{pipe}}^{sk}(2) = \max\{L_1, L_2\}$$

as the communication costs are negligible⁹, in this case.

Last but not least, consider what happens in case the template is used in a context where there is no chance to have two distinct processing elements available to support the execution of the two stages. In this case, the template concurrent activities may be “collapsed” on a single processing element and the consequent template performance model will state that:

$$T_{S_{pipe}}^{templ}(1) = T_{S_1} + T_{S_2}$$

or, in case communications are exposed due to impossibility to overlap communications with computation

$$T_{S_{pipe}}^{templ}(1) = T_{S_1} + T_{S_2} + t_{c_1} + t_{c_3}$$

■

In addition to the performance models related to parallelism exploitation—skeleton and template models—performance models are also needed to model target architectures. Therefore we introduce:

Architecture performance models These are the models exposing *the time spent in the different mechanisms implemented by the target architecture* to the template performance models. Typical architecture performance models are those modeling inter-processing element communication or relative power of processing elements.

- E.G.▷ Communication costs may be modeled as a linear combination of setup time and message transfer time. With quite significant approximations, communication cost of a message having length l on an distributed architecture with no overlap among communication and computation, may be modeled as:

$$T_{comm}(l) = T_{setup} + \frac{l}{B}$$

being B the bandwidth of the communication links.

The same mechanisms, when considering many/multi cores with communications implemented in memory may be modeled as

$$T_{comm}(l) = T_{synchr}$$

if and only if the communication fits a single producer single consumer schema. In this case, one thread writes the message in main (shared) memory, and another thread reads the message. The producer and consumer threads must synchronize in such a way consumer does not consume “empty” messages and producer does not send messages in buffers that still hold the previously sent message. However, if the communication schema fits a different schema, the model changes. As an example, when a single produce multiple consumer schema is used, then additional synchronization cost has to be added to take into account to model contention among consumers to read the message. In general, these additional synchronization costs are proportional to both the length of the message and to the number of consumers involved. ——— ■

⁸this happens when interconnection network is implemented through dedicated “communication processors”, as an example. In that case, the communication time spent on the processor is negligible as the only cost paid is the offloading (to the communication processor) of the message descriptor.

⁹masked by the computation times

A.3 ALTERNATIVE APPROACHES

When developing any performance model we are faced to the problem of deciding the *level of accuracy* of the models. Depending on the number of the parameters taken into account, as well as on the final “form” of the performance models looked for, different results may be achieved. In particular, different approximation levels may be achieved, that is different accuracies may be achieved in the predicted results.

Most of the skeleton based programming frameworks developed so far considered performance models sooner or later. Some of them took into account performance models “*ab initio*”, either to support template assignment or to compare achieved performances with the theoretic ones. Others only introduce performance models after an initial development that did not take them into account. Often, models are introduced and used to understand what’s going wrong with templates that do not sport the expected behaviour.

In most cases, anyway, one of two distinct approaches is followed in the design of the performance models:

Approximate model approach In this approach, most of the peculiar features of the skeleton, template or architecture¹⁰ are “summarized” in global parameters or neglected at all. The result consists in quite simple models (performance formulas) than may indeed capture all the details actually impacting the performance modeling process.

Detailed model approach This alternative approach, instead, tries to model any detail of the skeleton, template or architecture as accurately as possible. The result is usually a quite complex formula computing the performance value out of a large number of parameters.

Both approaches have been followed in the past. The former promises to provide simpler performance modeling and therefore to enhance the possibilities to exploit these models in different contexts. The latter promises to be much more accurate at the expense of models much more “difficult” to compute¹¹.

Unfortunately, very often the detailed model approach explodes both in terms of the parameters considered and in terms of the complexity of the final models. In addition, the detailed model approach requires much more detailed verification and validation activities than the approximate model approach. Eventually, the approximate model approach is the one currently being followed in the waste majority of cases when performance models are considered and used.

E.G.▷ Let us consider a classical farm template consisting of an *emitter/collector* process and of a string of n_w worker processes. The emitter/collector process gathers input tasks from the input stream and schedules these tasks for execution on one of the available workers. Workers, in turn, receive tasks to be computed, compute them and send back to the emitter/collector process the results. The emitter/collector process—concurrently to its task scheduling activity—collects results, possibly re-order results in such a way the input task order is respected in the output result sequence, and eventually delivers such re-ordered results onto the farm output stream. This template is often referred to as *master-worker pattern* although—in our perspective—this is actually a template suitable to implement both the task farm and map/data parallel skeletons.

In order to model service time of the master/worker template following the “approximate” model approach we ignore as much details as possible while observing that:

¹⁰depending on the model considered

¹¹although this “difficulty” could appear negligible with respect to the complexity of the “optimization” process of parallel programs

- the emitter/collector is actually the result of merging of task scheduling and result collection activities
- the string of worker processes dispatch an overall service time of $\frac{T_w}{n_w}$, being T_w the service time of the single worker.

The three activities—task scheduling, task computation and results gathering and dispatching—happen to be executed in pipeline. Therefore the service time of the master worker will be the maximum of the service times of the three stages. However, first and third stages are executed on the same processing element (the one hosting the emitter/collector concurrent activity). Therefore the service time eventually will be modeled (and approximated!) by the formula

$$T_S(n) = \max\left\{\frac{T_w}{n_w}, (T_e + T_c)\right\}$$

possibly complicated by taking into account communication times.

If we follow a “detailed model” approach, instead, we should start considering that the emitter/collector spends a time composed by 2 task communication times (one to receive a task from the input stream and one to deliver it to one of the workers), plus a scheduling time (to decide which worker should be given the task), plus 2 result communication times (one to receive the result from a worker and the other one to deliver the result to the output stream). In turn, these times will be modeled by using a kind of “setup + transmission time” expression. Then we should consider that also service time of the worker includes a task and a result communication time. Eventually we should evaluate how these expressions relate to the overall functioning of the master worker template, that is to reason about matching to be guaranteed between task scheduling, computation and result gathering times in order to avoid bottlenecks in the master/worker computation. Possibly a much more complicated formula will be obtained with a structure similar to the one listed above relative to the “approximated” approach. ■

A.4 USING PERFORMANCE MODELS

Let us make the following assumptions:

- for each skeleton Sk_i provided by our programming framework, we have available a set of skeleton performance models

$$\mathcal{M}^{Sk_i} = \langle T_S^{Sk_i}(n), T_C^{Sk_i}(n), s^{Sk_i}(n), \epsilon^{Sk_i}(n), \dots \rangle$$

- for each template Tpl_i suitable to implement skeleton Sk_i we have a similar set of template performance models

$$\mathcal{M}^{Tpl_i} = \langle T_S^{Tpl_i}(n), T_C^{Tpl_i}(n), s^{Tpl_i}(n), \epsilon^{Tpl_i}(n), \dots \rangle$$

- for each measure of interest M_i and for each target architecture targeted A_j we have a model

$$M_i^j = f_i^j(p_1, \dots, p_k)$$

where p_k are the base parameters modeling the target architecture (e.g. communication setup time, communication bandwidth, etc.).

We already stated in the introduction section of this chapter that these performance models will be eventually used to predict performance of applications, to compare different implementations of the same application or to evaluate how much an achieved performance is close to the ideal one.

In the the following sections, we discuss how these activities exploit performance models at compile time, at run time, or post-run time.

A.4.1 Compile time usage

At compile time different usages can be made of the performance models:

- in case of template based skeleton framework implementation performance models are used to *compare alternative* template-to-skeleton assignment, as explained in Sec. 7.2.1
- again, with template based implementations performance models may be used to *predict* the impact of accelerator usage (e.g. the impact of using a GPU to implement a map skeleton). This could be also considered a variant of the “compare alternatives” usage, as *de facto* this corresponds to evaluate whether the performance achieved when using the accelerator turns out to be better than the one achieved when the accelerator is not actually used.
- when using a MDF implementation, performance models may be used to decide which kind of tasks are worth to be delegated for concurrent execution to a concurrent MDF interpreter instance. As a matter of fact, tasks that require a computation time¹² smaller than the time required to communicate the task to the remote interpreter and to retrieve the result back from the remote interpreter are not worth to be scheduled for concurrent execution.
- again, when using a MDF based implementation, the performance models may be used to fine tune the template actually used to implement the distribute MDF interpreter.

However, the main and more important usage of performance models consists in automatically deriving, without any kind of application programmer support, unspecified non functional parameters relative to the execution of the skeleton/template at hand.

We stated that some non functional parameters are requested, in some skeleton based programming frameworks, that could be derived automatically¹³. The optimal parallel degree of a template is one of such parameters.

In case we are implementing a parallel program modeled by a single skeleton, the optimal parallelism degree may be obtained as follows:

- s1 We start considering the number of processing elements available P . We identify this as the maximum parallelism degree of our application.
- s2 We evaluate the performance model of interest $\mathcal{P}(p)$ for the all parallelism degrees $p \in [0, P]$.
- s3 We assign the template the parallelism degree $p_i \mid \forall j \neq i \mathcal{P}(p_j) \leq \mathcal{P}(p_i)$

Although the process is linear in the amount of processing elements/resources available, the whole computation will not take a significant time, provided the formula expressing $\mathcal{P}$ is reasonably simple *and* P is not huge.

Alternatively, we can “solve” the performance model in such a way the exact values of the parameters maximizing the performance with the given resource constrains are found.

In case our application is modeled using a skeleton nesting, a process such as the one described in Sec. 7.2.1 may be used instead. In this case, performance models are used

- in the first phase (the bottom up assignment of “optimal” parallelism degree) to compute the upper bound (if any) to the theoretic parallelism degree of the skeleton-s/templates used, and

¹²on the local node, that is on the processing element in charge of scheduling the task to a remote interpreter

¹³we mentioned the parallelism degree, as an example

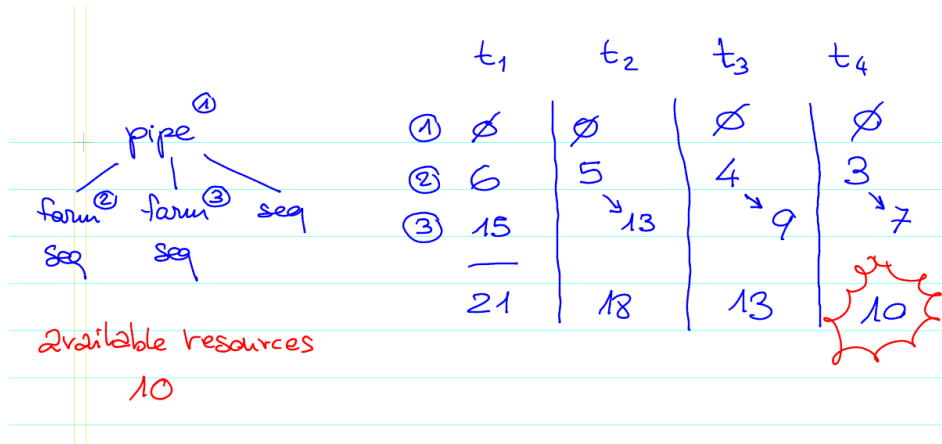


Figure A.1 Computing resource assignment in a skeleton tree. Initially the two farms are given 5 and 15 processing elements. Then we visit the skeleton tree bottom up and we start taking away one processing element from the first farm. According to the pipeline performance model this allows to take away 2 resources from the second farm. The process is repeated until the number of required resources happens to be smaller than the number of the available resources.

- in the second phase (the top down iterative process taking away resources from the assignments in such a way balancing is preserved) to estimate the performance of the “reduced” skeleton/templates, that is, of the skeletons/templates from which resources have been removed in the tentative of coming to a parallelism degree matching the resources actually available.

This process is exemplified in Fig. A.1.

A.4.2 Run time usage

At run time, performance models are mainly used to verify the current behaviour of the application with respect to the expected behaviour. This means that:

- we measure actual performance achieved by the application on the target machine. This means we insert probes¹⁴ into the application code and through the probes we gather the measures needed to assess the current application performance¹⁵.
- we measure the target architecture parameters needed to instantiate the (template) performance model of our application
- we evaluate the expected performance model of the application and we measure the distance with respect to the performance values actually measured.

Two possibilities are open, once the actual and the expected performance measures have been got.

- a former, simpler possibility is to report the application programmer of the differences in between the measured and predicted performance values. No other action is taken

¹⁴methods used to fetch performance values. This may be implemented completely in software (e.g. by invoking proper operating system calls) or with some degree of hardware support (e.g. fetching values of proper “hardware counters” available on most modern processors).

¹⁵it is known that probe insertion perturbs the computation, therefore we must choose probes in such a way a minimal intrusion is enforced and, therefore, a minimal perturbation of actual application performance is achieved.

to correct possible mismatches between measured and predicted performance in this case. Possibly, the application programmer may use the reported information to improve¹⁶ the application.

- a second, more complex possibility consists in analyzing the differences in between the measured and predicted performances and in operating—without programmer intervention, actually—to restructure the parallel application in such a way the measured performance is made closer to the predicted one. We will further discuss this possibility in Sec. 10.1. At the moment being it is sufficient to mention that application restructuring may involve rewriting the high level application code using different skeletons or different skeleton compositions, as well as re-computing the template assignment to skeletons, in such a way more efficient templates are picked up instead of the inefficient ones. It is worth pointing out that in this latter case, the choice of the “better” templates to be used in spite of the currently used ones could not rely on evaluation of the associated performance models. These models are the very same that led to the current template choice. The discrepancies in between observed and predicted performance should be due to inaccuracies in the performance model or in temporary conditions arising on the target architecture which are not considered in the model. Therefore some kind of “artificial intelligence” or heuristic reasoning should be applied to get rid of the faulty situation.

A.4.3 Post-run time usage

The last way of exploiting performance models is the newest one in the algorithmic skeleton scenario. It has been first introduced by Mario Leyton in his Calcium skeleton framework built on top of the Java ProActive middle-ware [14].

In this case, performance models are used as follows (Leyton referred to this process as *codeblame*):

- during application computation, current performance figures are collected on a *per skeleton* basis.
- after the end of the application computation, predicted performance figures are computed (again, *per skeleton*).
- the predicted values are compared to the measured ones and differences are individuated.
- in case of sensible differences, an analysis is performed on the sub-terms constituting the performance models in such a way a precise characterization of the causes of skeleton (or template) under performing are individuated.
- eventually, such causes are exposed to the application programmer as “performance summary” of the application execution, along with more general data such as overall completion and service time.

The programmer, looking at the causes of the inefficiencies found in the execution of the current version of the application, may modify the application in such a way these inefficiencies are eventually removed (Fig. A.2 shows the kind of output presented to the user by the Calcium framework).

¹⁶to fine tune the application varying some of the parameters used to instantiate the skeletons used, or even, in case of large differences, to completely restructure—in terms of the skeletons or templates used—the whole application.

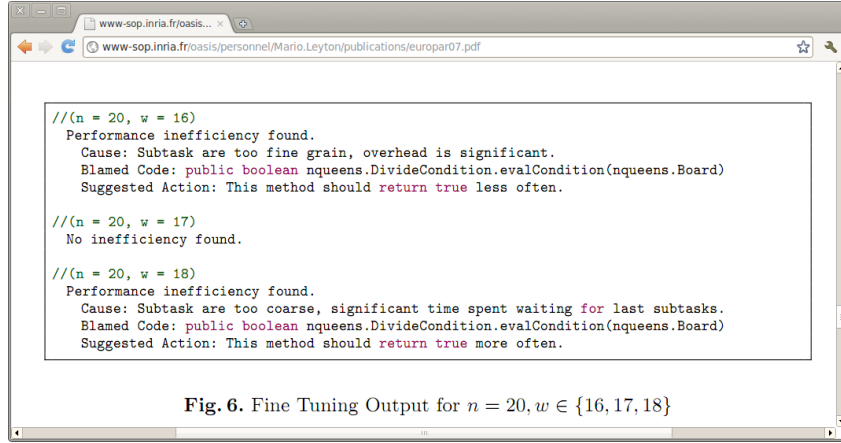


Fig. 6. Fine Tuning Output for $n = 20, w \in \{16, 17, 18\}$

Figure A.2 CodeBlame figure from M. Leyton paper [14] The output is relative to three different runs (with parameters outlined in the comment lines) of a program solving the N-queens problem.

E.G.▷ Consider an application that can be implemented using data parallel skeletons, and suppose the proper skeleton to use is a map. Suppose also that the map may be applied at different items of a complex input data structure: as an example, a map may be applied either at the level of singletons belonging to a matrix or at the level of the rows of the same matrix. Now suppose the programmer chooses to exploit parallelism in the computation of the single element of the matrix. It may be the case that the grain of these computations is too fine and therefore the predicted performance happens to be far from the observed one. The skeleton framework may analyze the map model and perceive the inefficiency is given by the time needed to send data to remote processes (or to remote MDF interpreter instances) higher than the time spent computing the single matrix item. It can therefore report to the programmer a statement such as:

Map skeleton Sk_i : fine grained worker.
Please use coarser grain workers.

As a consequence, the application programmer has a clear hint on how to intervene to improve application efficiency. ■

A.5 AMDAHL LAW

A well know law, the Amdhal law, states that the amount of non parallelizable work in an application determines the maximum speedup we may achieve by parallelizing that application.

Assume you have an application where f percent of the total work is inherently sequential—it could not be parallelized. And assume that the whole application needs T_S units of time to be completed in sequential. Therefore fT_S units of time will be spent in the application *serial fraction* and $(1 - f)T_S$ units in its non serial fraction. The best thing we can imagine is that using n processing elements the $(1 - f)T_S$ execution time can scaled down to

$$\frac{(1 - f)T_S}{n}$$

Therefore the speedup we can achieve with our application is

$$sp(n) = \frac{T_s}{fT_S + (1 - f)\frac{T_S}{n}}$$

If we assume to have infinite processing resource available, the term $(1 - f)\frac{T_s}{n}$ clearly goes to 0 and therefore the asymptotic speedup may be written as:

$$\lim_{n \rightarrow \infty} sp(n) = \frac{T_s}{fT_s} = \frac{1}{f} \quad (\text{A.1})$$

Equation A.1 is the Amdhal law. It has important consequences. In particular, it states that any application having $x\%$ of serial fraction could not achieve a speedup larger than $1/x$.

E.G.▷ If you have an application running in 10 hours with a 10% serial fraction, you will never succeed running this application in less than 1 hour ($\lim_{n \rightarrow \infty} sp(n) = \frac{1}{0.10} = 10$, $\frac{10 \text{ hours}}{10} = 1 \text{ hour}$). ■

Amdhal law establish a completely theoretic limit to the speedup we can achieve through the parallelization of the activities needed to solve a given problem. However, there are many factors that impair the possibility of achieving the limits established by Amdhal law. In particular, *overheads* incur in the implementation of parallel applications. Overhead incur in the execution of any non functional code of parallel applications. An overhead is to be intended as a time loss with respect to the computation of the parallel application (functional) results. As an example, setting up the parallel activities, either threads or local/remote processes, is an activity that introduces an overhead. If we consider the time needed to compute the solution of our problem, this does not comprehend the time needed to set up parallel activities. However, if we do not setup parallel activities we'll not be able to achieve any kind of speedup. Therefore overheads incur, they are somehow necessary to implement parallelism, but we have to manage to minimize them as much as possible. Another example of overhead is the one related to communications. If we set up a number of concurrent activities as processes on different processing elements and some data has to be distributed to the processes before they start computing, then the time needed to distributed the data through communication is plain "communication overhead".

Looking again at the Amdhal law, we can modify it in such a way overheads are taken into account. In particular we can image the time spent to compute in parallel the non serial fraction of the application is $\frac{T_s}{n} + T_{ov}$ instead of being simply $\frac{T_s}{n}$. Unfortunately, overheads (T_{ov}) depend on the parallelism degree. Communication time depends on the number of entities participating in the communication or the time spent to start concurrent/parallel activities depends on the number of activities actually involved, as an example. Therefore Amdhal law should be rewritten as

$$\lim_{n \rightarrow \infty} sp(n) = \lim_{n \rightarrow \infty} \frac{T_s}{fT_s + (1 - f)(\frac{T_s}{n} + T_{ov}(n))} = \frac{1}{f + (1 - f)\frac{T_{ov}(n)}{T_s}} \quad (\text{A.2})$$

In case of negligible overhead this clearly reduces to Eq. A.1, but if the fraction of overhead with respect to the total sequential time is significant, the asymptote speedup is even smaller than one expected taking into account only the weight of the serial fraction. It is clear therefore that the less overhead we introduce in our parallel application with non functional code the better performance we can achieve.

A.6 GUSTAFSSON

Some year later after the formulation of the Amdahl law, two scientist observed that at Sandia Labs (US) several applications were demonstrating speedups far beyond what expected taking into account Amdahl's law.

Gustafson[], with the help of E. Barsis formulated a different speedup law, taking into account the possibility to increase the problem size while looking for opportunities to exploit larger numbers of processing elements.

In the original paper, Gustafson observes that, being s the serial and p the parallel time spent on a parallel system with N computing elements to compute a given application, the speedup may be given as:

$$\text{speedup}(n) = \frac{s + p \times N}{s + p} = \frac{(s + p) + (N - 1)p}{s + p} = 1 + \frac{(N - 1)p}{s + p}$$

Looking at the actual results (speedups) achieved using parallel machines, that were sensibly away from Amdahl law predictions, Gustafsson formulated speedup with another law, still considering serial fraction, but in a slightly different perspective w.r.t. Andahl's law. Gustafsson evaluates the execution time with n processors, considering the *scaled* serial fraction as

$$f = \frac{t_s}{t_s + t_p(n)}$$

and the non serial fraction as

$$(1 - f) = \frac{t_p(n)}{t_s + t_p(n)}$$

From the definitions it follows that:

$$T(n) = f + (1 - f) = 1$$

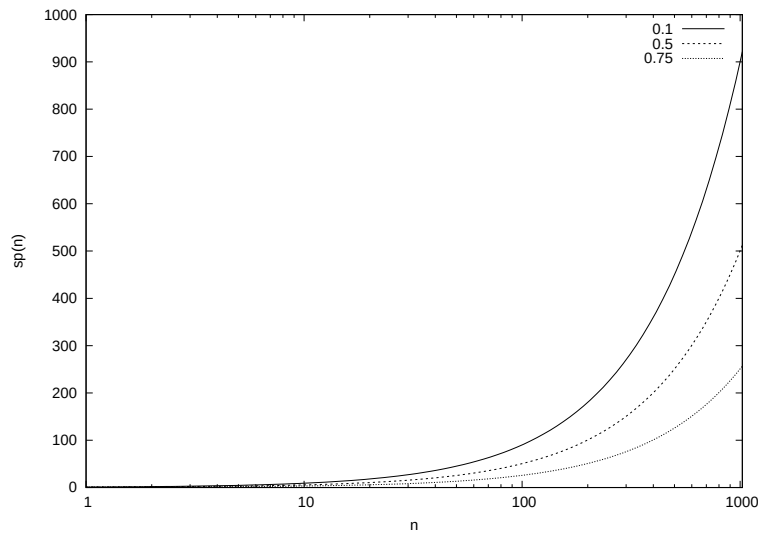
and then the sequential time as

$$T(1) = f + (1 - f) \times n$$

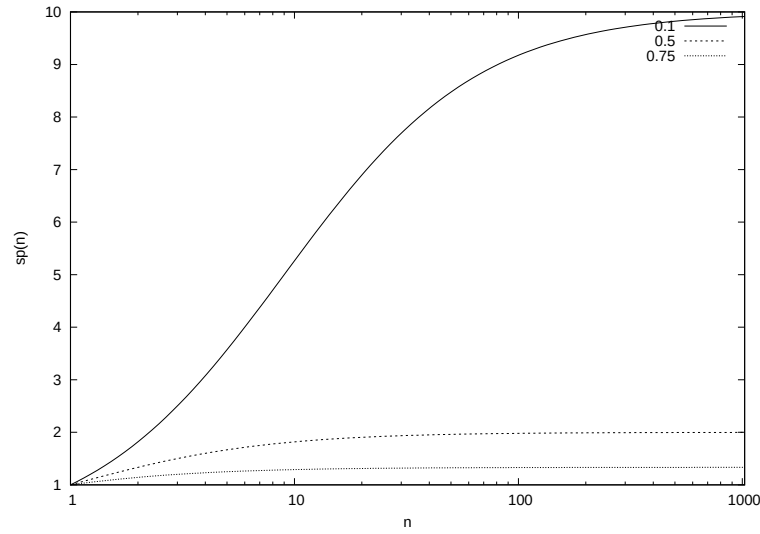
With these values, the speedup can be obtained as

$$sp(n) = \frac{T(1)}{T(n)} = \frac{f + (1 - f) \times n}{1} = f + n - n \times f = n - (n - 1) \times f$$

With this modelling of the speedup, Gustafsson was able to explain the speedups close to 1000 measured on a parallel machine of his time sporting 1024 processing elements. In fact, considering different scaled serial fractions, the kind of speedups can be plotted as follows:



while considering the same serial fraction values, Amdahl law would give:



What's wrong here? Despite Gustafsson succeeds modelling the experienced speedups, both Amdahl and Gustafsson look reasonable formulas to express speedup. However, the serial fraction used in Gustafsson is not the same one of Amdahl, and this is why the results are so different.

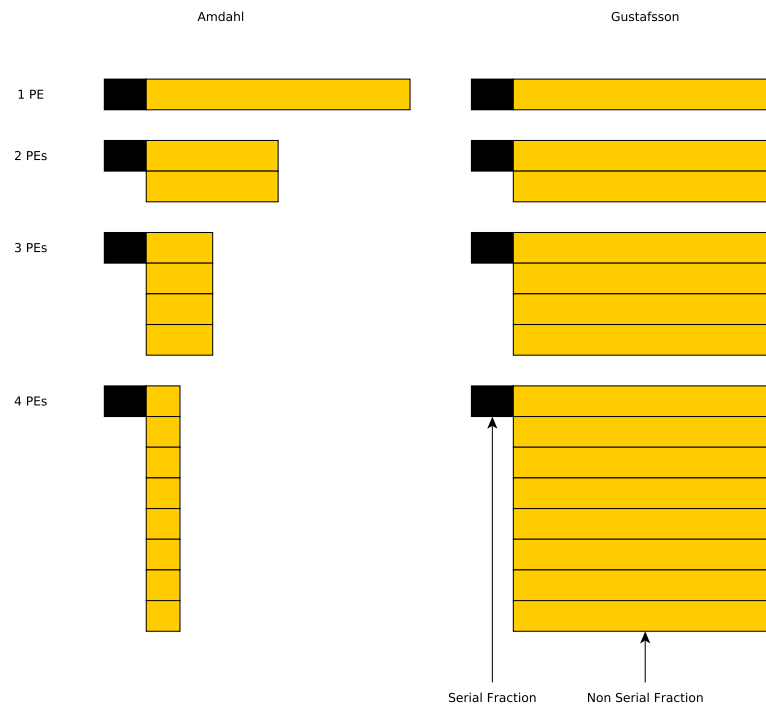
Let suppose for Gustafsson law we consider $f = 0.5$ and a parallel time of 1000s. Then speedup on 10 processors should be $10 - (10 - 1) * 0.5 = 5.5$ according to Gustafsson, while according to Amdahl we need to compute sequential time and then compute the speedup according to Eq. A.2. Sequential time can be computed as follows: 50% of 1000s is serial. That is sequential time can be computed as 500s (serial fraction) plus the rest of the time multiplied by the processing elements used, that is $500s + 10 \times 500s = 5500s$. With this numbers, the serial fraction according to Amdahl will be $\frac{500s}{5500s} = 0.090909$ determining an asymptotic speedup of about 11 rather than the 2 predicted when using the Gustafsson f for the Amdahl f .

We can actually formally compute Amdahl serial fraction f_a from Gustafsson scaled serial fraction f_g as

$$f_a = \frac{1}{1 + \frac{(1-f_g) \times P}{f_g}}$$

However, Gustafsson law models realistically those computations where the amount of computation actually does not stay the same for all parallelism degrees. In other words, his notion of *scaled* serial fraction perfectly fits those computations where data increases in size with the amount of processing elements available. As the amount of overhead needed to set up and orchestrate the relative parallel activities does not increase as the data size, this means that with higher and higher parallelism degrees the improvement coming from parallel execution of larger chunks of computations overcomes the loss due to the (slightly) increasing overheads needed to set up and orchestrate the parallel computations.

This can be probably better understood representing graphically the process of augmenting the parallelisable work while succeeding keeping constant the serial fraction of the application. In the following picture, the black parts correspond to the serial fraction that remains constant in front of the execution of larger and larger non serial computations assigned to different processing elements:



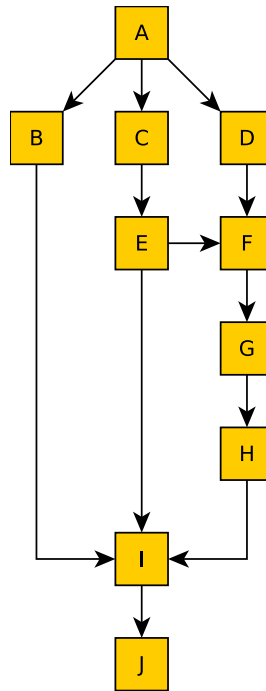
A.7 WORK SPAN MODEL

An alternative view at what happens in a parallel application consists in distinguishing the kind of activities that lead to the final application result and, in particular, to analyze the dependencies in between the different activities that represent the application components.

In a parallel application we can usually distinguish several components (sequential portions of the application) that contribute to the final result of the application along with a graph of *dependencies* among these components. Given two components of our application, say A and B , we say there is a dependency between the two components, denoted as $A \rightarrow B$, if the component A produces some data that is needed to compute component B .

Components and dependencies may be used to represent the application graph (C, D) where the nodes C represent components and the directed arcs A represent dependencies.

As an example, the following graph represents an application with 10 components, A to J , with some dependencies among them (e.g. $B \rightarrow I$ or $E \rightarrow F$).



The work span model defines as *work* the total amount of work (e.g. time) to be spent executing the whole application. In our case, assuming for the moment that each component C_i spent a single unit of time t to compute once all the input dependencies (arcs $C_j \rightarrow C_i$, any j) have been satisfied—that is the corresponding components C_j have produced the data needed by C_i —the work will be $10t$, as the computation needs to execute the whole set of 10 components to compute the result which is the output of component J .

The *span* instead, is the total amount of work needed to complete the *longest* chain of components from the input component (the one with no input dependencies) to the output component (the one with no dependencies on other components). The “longest” chain has to be intended with respect to the target measure taken into account. As we usually intend to reduce the execution time, the “longest” chain is the chain of components from input to output components that requires the longest time to compute. In our example:

- the input component is component A
- the output component is component J
- the chains to consider are
 - A, B, I, J
 - A, C, I, J
 - A, C, E, F, G, H, I, J
 - A, D, F, G, H, I, J
- considering unit time costs t for all the components, the “longest” chain is clearly the third one A, C, E, F, G, H, I, J
- the cost (execution time) of the longest chain is $8t$

Now in order to execute the application in parallel, we need to spend at least the time needed for the span. No matter how many parallel processing elements we have, the execution of two components C_1 and C_2 such that $C_1 \rightarrow C_2$ is a dependency in the graph requires first the time needed to compute C_1 and *then* the time needed to compute C_2 as

the latter computation cannot start before the former has produced the data creating the dependency. In the work span model, the time spent computing the span chain is called T_∞ . On the other side, the time needed to compute the whole applicaiton with just a single processing element is clearly the time spend to compute the *work*, denoted as T_1 . The work span model simply observes that the maximum speedup we may achieve is nothing but

$$sp(\infty) = \frac{T_1}{T_\infty}$$

By simply observing that obviously

$$T_\infty \geq \frac{T_1}{P}$$

it follows as usual that

$$sp(\infty) = \frac{T_1}{T_\infty} \leq \frac{T_1}{\frac{T_1}{P}} = P$$

as expected.

It worth pointing out that the work span model is much more pessimistic w.r.t. Amdahl law, in that it further limits possibilities for parallel execution beyond serial fraction. In the sample graph above the work span model predicts a maximu speedup of $\frac{10t}{8t} = 1.25$, independently of the number of parallel processing elements we have available.

The *Brent's lemma* gives a lower bound to the speedup we can achieve. The time spent using P processing elements (T_P) can be expressed as

$$T_P \leq \frac{T_1 - T_\infty}{P} + T_\infty$$

as the span time T_∞ has to be spent anyway and the rest of the computation (taking time $T_1 - T_\infty$) could be at the best computed in equal parts distributed on the P processing element available, and therefore:

$$\frac{T_1}{P} \leq T_P \leq \frac{T_1 - T_\infty}{P} + T_\infty$$

A.8 SKELETON ADVANTAGE

When modeling performance of a parallel computation a number of different factors have to be taken into account. Such factors depend on the kind of concurrent graph activities involved, in general. In case we are considering the implementation of the parallel application rather than the application in abstract terms, the factors depend also on the target architecture features (both hardware and software). Therefore performance modeling turns out to be a quite difficult task, possibly more difficult than the development of the parallel application itself in the general case.

However, modeling algorithmic performance in skeleton programming frameworks is somehow simplified by different factors:

- First of all, the *exact parallel structure* of the parallel computation is completely exposed by the algorithmic skeletons used. No side, “hidden” interaction exist impacting the performance modeling process. This is not usually the case in classical parallel programming frameworks where programmers work directly at the level of the base mechanisms in order to develop a parallel application.
- Second, when using hierarchical composition of skeletons, the performance modeling of the application may be incrementally build from the performance models of the

base skeletons. Composition of performance modeling is achieved through proper development of modeling of a set of base measure—such as service time or latency—for each one of the skeletons provided to the final user/the application programmer. These base measures are the ones used in the definition of skeleton performance model as well and therefore the skeleton models recursively compose.

- Last but not least, when dealing with *implementations* rather than abstract parallelism exploitation patterns, the architecture modeling needed to support template performance modeling is somehow simplified as we can reason in terms of abstract components of the templates rather than of collections of very precise (base) mechanisms eventually used to implement the template on the target architecture at hand.

A.9 MONITORING APPLICATION BEHAVIOUR

Monitoring application behaviour requires two kind of activities:

- measuring times relative to the execution of the application as a whole. This is needed to be able to compute the actual completion time of the application as well as the service time achieved in delivering the results, just in case the application processes streams of tasks to produce streams of results.
- continuous measuring of the times relative to execution of different application parts. This is needed to be able to check the correct¹⁷ execution of the application.

The former monitoring activity is simple and does not need actually too much modifications of the application code. Completion time and service time may both be measured using tools external to the application. In Linux/Unix the approximated wall computation time of an application may be taken by launching the application as a parameters of a `time` command rather than directly at the shell prompt. Once the completion time and the number of items computed is known, it is easy to derive the service time.

E.G.▷ In order to know the time spent in compiling the PDF of this book from its latex code, we can use the command shown below. The timed execution of `pdflatex` command with the output redirected to the null device is requested. The results of the computation are discarded, and at the end a summary of real, user and system time required to complete the command is show. “Real” time is the wall clock time spent to compute the program, as perceived by the shell process executing the command. “User” time is the one spent executing user code and eventually “sys” time is the one spent executing syscalls on the behalf of the “user” code. _____ ■

```
marcod@macuntu:$ time pdflatex main.tex > /dev/null

real 0m1.203s
user 0m1.156s
sys 0m0.044s
marcod@macuntu:$
```

More precise time measures may be obtained reading time or clock cycles at the very beginning of the application code, then at the very end and eventually computing the differences among the measured values. Different system or library calls are available to the purpose in the different operating systems. In Linux `clock` and `gettimeofday` may be used as well as `omp_get_wtime` or `MPI_Wtime` in case OpenMP or MPI are used.

¹⁷with respect to performance non functional concerns

In modern C++, the class `std::chrono` may be used, that usually provides μ s precision timers. As an example, the section of code of interest may be “bracketed” with a couple of calls to `std::chrono` that read timers before and after code execution and then further methods of the library may be called to compute and print the time difference observed:

```
using namespace std::chrono;
auto start = high_resolution_clock::now();
// section of code to measure
// ...
auto elapsed = high_resolution_clock::now() - start;
auto usec = duration_cast<microseconds>(elapsed).count();
std::cout << "Spent " << usec << " usecs" << std::endl;
```

Leveraging the OO programming techniques, RAII pattern may be used to define objects that measure time on creation and at the destroy time and report time spent in a block of code. The following class implements such a pattern:

```
class utimer {
    system_clock::time_point start;
    system_clock::time_point stop;
    std::string message;
    using usecs = microseconds;
    using msecs = milliseconds;

private:
    long * us_elapsed;

public:

    utimer(const std::string m) :
        message(m), us_elapsed((long *)NULL) {
        start = system_clock::now();
    }

    utimer(const std::string m, long * us) :
        message(m), us_elapsed(us) {
        start = system_clock::now();
    }

    ~utimer() {
        stop =
            system_clock::now();
        uration<double> elapsed =
            stop - start;
        auto musec =
            duration_cast<microseconds>(elapsed).count();

        std::cout << message << " computed in " << musec <<
            " usec " << std::endl;
        if(us_elapsed != NULL)
            (*us_elapsed) = musec;
    }
};
```


such that the time spent in the execution of a given section of code can be measured as follows:

```
{
    utimer newTimer("mysec"); // constructor: read time

    // code to be profiled here
    // ...
} // destructor : read time and print diff
```

The latter activity (continuous monitoring) is a little more complicated. It requires the usage of the same library calls possibly used to for the former monitoring activity. However, care has to be taken not to introduce significant impact in the original computation due to the monitoring mechanisms used and to the monitoring event frequency used.

This problem is known as the *intrusion problem*. The more probes you insert into the application code, the less accurate measures you take. Inserting a probe usually requires to perform some extra system call, which is quite expensive, in terms of time spent in the call, or library call, which is less expensive than a system call but may be anyway expensive. Moreover, the data read by the probes have to be stored somewhere, in order to be usable at some point during the execution of the application or after the current application execution. The memory footprint of the probe data is usually small, but it could be anyway a problem at least for those applications using huge amounts of virtual memory for their own “functional” data.

Care has to be taken also in reporting probe measures on the user terminal *while* the parallel application is running. If you insert a `printf` to show some probe measure, the resulting `write` system call is practically executed asynchronously with respect to the application code, and therefore it may look like it will not introduce any delay but the one relative to call to the `write`. However, the `printf` results will eventually be reported on the user terminal through some kind of network transport protocol. The resulting traffic will be supported by the same network supporting the parallel application communications and synchronizations. And this eventually *perturbs* the application behaviour.

When reasoning about parallel applications executed on multi cores, rather than on distributed architectures such as COW/NOWs, similar intrusion problems must be taken into account, related to thread—rather than process—scheduling and interaction.

In any case, parallel application probing is even more complicated than sequential application probing because the changes in relative timings of the concurrent activities due to probes may be such that some deadlocks or some bottlenecks completely disappear from the application run. When the probes are removed to run the application at its own maximum speed, they may come back again with definitely unpleasant effects¹⁸.

Structured parallel programming helps in the implementation of application monitoring in distinct ways:

- The placement of probes is clearly suggested by the skeletons used in the application. In a pipeline, we can easily identify probe placement before and after the computation of a single task on each one of the stages involved. These probes may accumulate *average* service time for each one of the stages. Then pipeline termination procedure may be used to gather the stage service times all together in such a way the pipeline service time may be eventually computed, or the stage service times may be attached to the partially computed tasks flowing through the pipeline in case the pipeline overall service time has to be computed *during* the pipeline computation. In this case the

¹⁸by the way Murphy law states this will happen when you'll be demonstrating the application to the final client paying for the application development :-)

computation of the pipeline overall service time may be made on the last pipeline stage.

- The implementation of the skeletons may be structured in such a way primitive measures are automatically gathered (may be depending on some skeleton configuration/-compilation flag) and returned upon the invocation of proper skeleton library calls. This is not a difficult task for the system programmer designing skeleton implementation, actually, as he/she has the complete knowledge related to the implementation and therefore he/she may figure out the best probing strategies to gather these base performance measures. This contributes to the separation of concerns between system and application programmers enforced by the structured parallel programming approach, actually.
- Due to the clear parallel structure of the skeletons, the perturbation effect of the probes may be estimated/modeled—at least in an approximate way—in such a way the measures returned by the probes actually estimate the performance of the application *without* the probes. Again, consider a pipeline. If we measure the service time by inserting a couple of probes reading time before a task computation is started and after it has been completed, we can add to the measured computation time an estimate of the communication overhead and return this value as the stage latency, which is not including—*de facto*—the overhead relative to the time measure probes.

A.10 PERFORMANCE MODEL DESIGN

Performance models represent a critical resource associated to skeletons and templates. Different approaches may be followed in order to design performance models that will be discussed in the following sections.

A.10.1 Analytical performance models

Traditionally, skeleton and template performance models have been developed following an “analytic” approach. This means that

- first performance modeling of the base mechanisms is designed such that we have analytical formulas expressing all the base performances
- then the performance model of the skeleton (template) is designed as a proper composition of instances of the base mechanisms performance models.

In this case, no general theory has been used to support performance model development but some quite usual mathematical background.

E.G.▷ The pipeline skeleton performance model may be developed, according to the analytical performance modeling approach, as follows:

- we first consider communications costs, and we conclude the cost of the communication of an item of type α and “sizeof” $sizeof(\alpha)$ among stages i and j is

$$T_{comm}(i, j) = T_{setup} + \frac{sizeof(\alpha)}{\mathcal{B}}$$

being $\mathcal{B}$ the inter stage bandwidth.

- then we consider costs relative to the execution of the two stages (latencies) as L_1 and L_2 .
- eventually we conclude the slowest stage will determine the pipeline service time:

$$T_s(pipe(S_1, S_2, \dots, S_m)) =$$

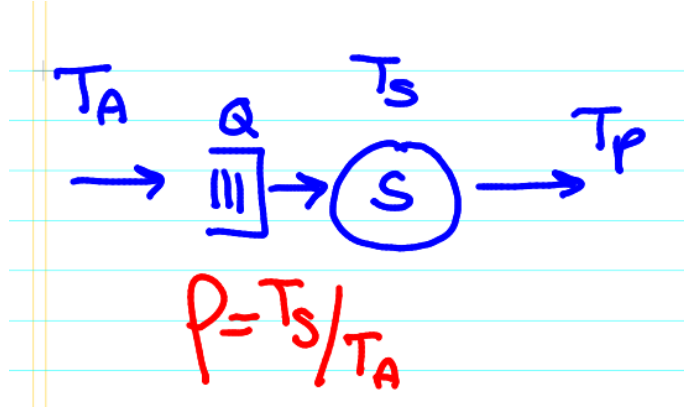


Figure A.3 Server modeling

$$\max\{(L_0 + T_{comm}(i, i + 1)),$$

$$\forall i \in [1, m - 1] : (L_i + T_{comm}(i - 1, i) + T_{comm}(i, i + 1)),$$

$$(L_m + T_{comm}(m - 1, m))\}$$

Using the same approach, but ignoring communication overheads, we can design the performance model of a farm skeleton as follows:

- we first consider the n_w workers may compute n_w tasks in a time equal to T_w (the time spent computing a single task)
- then we consider the entity providing the tasks will have its own service time (t_e), as well as the entity gathering the task computation results (t_c).
- we look at the farm implementation as a three stage pipeline and therefore

$$T_s(\text{farm}(W)) = \max\{t_e, t_c, \frac{T_w}{n_w}\}$$

In case we want to “solve” this model in such a way the optimal number of workers is computed, we can reason as follows:

- the service time is given by a maximum.
- the only term of the maximum depending on n_w is the $\frac{T_w}{n_w}$ term
- therefore we will have the minimum service time when this term will not be the maximum
- and herefore

$$n_w^{opt} = n \in \mathcal{N} : \frac{T_w}{n_w} = \max\{t_e, t_c\}$$

that is

$$n_w^{opt} = \lceil \frac{T_w}{\max\{t_e, t_c\}} \rceil$$

■

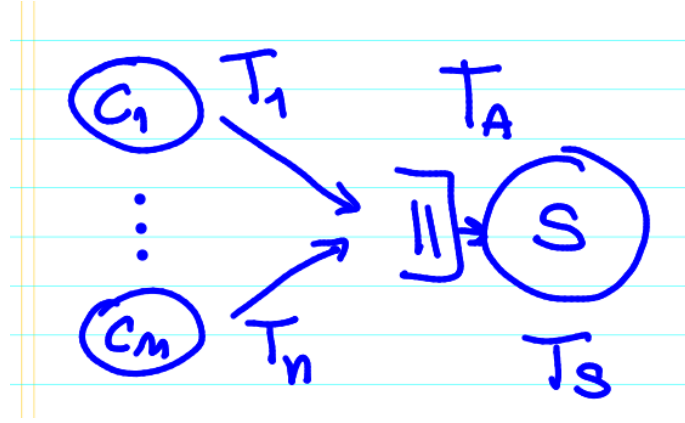


Figure A.4 Multiple clients for the same server

A.10.2 Queue theory

We can use some results from queue theory that basically model server systems in order to design skeleton (template) performance models. Queue theory is a quite complex field that has been extensively used in the past to model several distinct systems. Here we just mention a couple of results relative to M/M/1 queues¹⁹ that may be easily re-used to model performance of skeletons and, more effectively, implementation templates. The interested reader may refer to existing publications and books. In particular, our students may refer to the notes for SPA course by M. Vanneschi available on line [51].

We introduce queue systems, such as the one represented in Fig. A.3. A *server* S has a service time equal to T_S . It accepts server requests through an input queue. The inter arrival time of service requests is T_A . The *utilization factor* of the input queue Q is defined as

$$\rho = \frac{T_S}{T_A}$$

The inter departure time T_P of results relative to service requests from the server S is given by

$$T_P = \begin{cases} T_A & \text{iff } \rho \leq 1 \\ T_S & \text{iff } \rho > 1 \end{cases}$$

Two results are particularly useful, due to the kind of parallel patterns we are considering in this book.

1. Consider a system made of a server a several clients, such as the one depicted in Fig. A.4. Suppose that each client C_i directs a service request to the server S each T_i . Then the inter arrival time to server S will be

$$T_A = \frac{1}{\sum_{i=1}^n \frac{1}{T_i}}$$

Intuitively, each client contributes sending a request every $\frac{1}{T_i}$ units of time. The frequencies of the different clients accumulate. Time is the inverse of the frequency.

¹⁹queue systems differ depending on the features of servers and clients. Usually a queue system is named as a triple “A/B/C”. A represents the kind of inter arrival time distribution, B represents the type of service time for the server(s) and C the number of servers (refer to Fig. A.3). In our case, we are interested in results relative to queues M/M/1 and M/M/g where the M stands for “Markovian, that is stochastic, with no memory.”

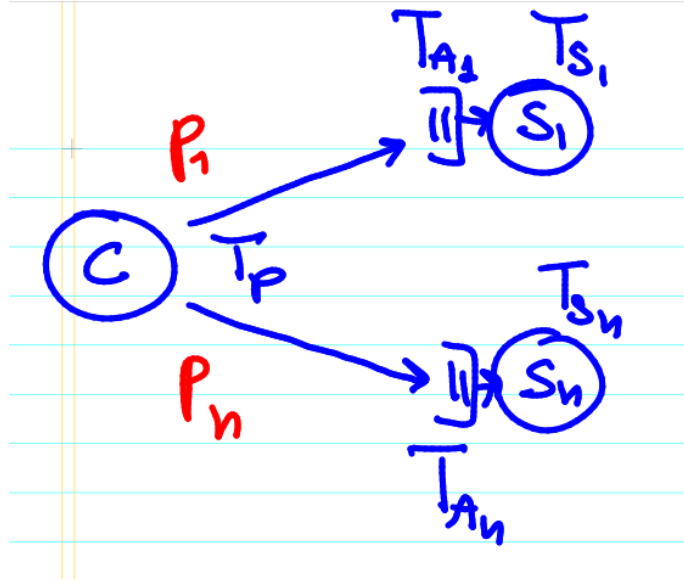


Figure A.5 One client for the multiple servers

- Consider a system made of one client interacting with some servers, as depicted in Fig. A.5. The client sends a service request towards server S_i with probability p_i . The inter departure time of service requests from client C is T_C . Then the inter arrival time of requests to server S is

$$T_A = \frac{T_P}{p_i}$$

With these results, we can model several different aspects related to the performance of a skeleton.

E.G.▷ The emitter-collector-worker implementation of a task farm (or equivalently, of a data parallel map) may be modeled as a client (the emitter) directing task computation requests to n_w servers (the workers) and by a set of clients (the workers) directing “result” requests to a single server, the collector.

Consider the first part of the system. The emitter has an inter arrival time T_A^e . Being sequential, its inter departure time is $T_P^e = T_A^e$. Assuming a round robin scheduling of tasks to workers, the probability to direct a request to a given worker will be given by

$$p_i = \frac{1}{n_w}$$

Due to the second result listed above, the inter arrival time to generic worker will be therefore

$$T_A = \frac{T_A^e}{\frac{1}{n_w}} = n_w T_A^e$$

The utilization factor of the input queue of the generic worker will therefore be

$$\rho = \frac{T_w}{n_w T_A^e}$$

In order to have the worker delivering its service time T_w , the utilization factor should be smaller than 1. This means:

$$\rho = \frac{T_w}{n_w T_A^e} < 1$$

and therefore:

$$T_w < n_w T_A^e$$

$$n_w > \frac{T_w}{T_A^e}$$

which corresponds to the usual result we already mentioned several times relative to the optimal number of workers in a task farm. _____ ■

REFERENCES

1. E. Agerbo and A. Cornils. How to preserve the benefits of Design Patterns. In *Proceedings of OOPSLA '98*, pages 134–143, 1998.
2. Enrique Alba, Gabriel Luque, Jose Garcia-Nieto, Guillermo Ordonez, and Guillermo Leguizamón. MALLBA a software library to design efficient optimisation algorithms. *Intl. Journal of Innovative Computing and Applications*, 1(1):74–85, 2007.
3. Marco Aldinucci, Francoise André, Jérémy Buisson, Sonia Campa, Massimo Coppola, Marco Danelutto, and Corrado Zoccolo. Parallel program/component adaptivity management. In G. R. Joubert, W. E. Nagel, F. J. Peters, O. Plata, P. Tirado, and E. Zapata, editors, *Parallel Computing: Current & Future Issues of High-End Computing (Proc. of PARCO 2005, Malaga, Spain)*, volume 33 of *NIC*, pages 89–96, Germany, December 2005. John von Neumann Institute for Computing.
4. Marco Aldinucci, Massimo Coppola, and Marco Danelutto. Rewriting skeleton programs: How to evaluate the data-parallel stream-parallel tradeoff. In *Proc. of International Workshop on Constructive Methods for Parallel Programming. Technical Report MIP-9805. University of Passau. Passau*, 1998.
5. Marco Aldinucci and Marco Danelutto. Stream parallel skeleton optimization. In *Proc. of PDCS: Intl. Conference on Parallel and Distributed Computing and Systems*, pages 955–962, Cambridge, Massachusetts, USA, November 1999. IASTED, ACTA press.
6. Marco Aldinucci and Marco Danelutto. Stream parallel skeleton optimization. In *Proc. of PDCS: Intl. Conference on Parallel and Distributed Computing and Systems*, pages 955–962, 1999.
7. Marco Aldinucci, Marco Danelutto, and Patrizio Dazzi. Muskel: an expandable skeleton environment. *Scalable Computing: Practice and Experience*, 8(4):325–341, December 2007.
8. Marco Aldinucci, Marco Danelutto, and Peter Kilpatrick. Skeletons for multi/many-core systems. In *Parallel Computing: From Multicores and GPU's to Petascale (Proc. of PARCO 2009, Lyon, France)*, volume 19 of *Advances in Parallel Computing*, pages 265–272, Lyon, France, September 2009. IOS press.
9. Krste Asanovic, Rastislav Bodik, James Demmel, Tony Keaveny, Kurt Keutzer, John Kubiatowicz, Nelson Morgan, David Patterson, Koushik Sen, John Wawrzynek, David Wessel, and

- Katherine Yelick. A view of the parallel computing landscape. *Commun. ACM*, 52(10):56–67, 2009.
10. P. Au, J. Darlington, M. Ghanem, Y. Guo, H.W. To, and J. Yang. Co-ordinating heterogeneous parallel computation. In L. Bouge, P. Fraigniaud, A. Mignotte, and Y. Robert, editors, *Proc. of Euro-Par 1996*, pages 601–614. Springer-Verlag, 1996.
 11. B. Bacci, M. Danelutto, S. Orlando, S. Pelagatti, and M. Vanneschi. P³L: A Structured High level programming language and its structured support. *Concurrency Practice and Experience*, 7(3):225–255, May 1995.
 12. A. Benoit, M. Cole, S. Gilmore, and J. Hillston. Flexible skeletal programming with eSkel. In J. C. Cunha and P. D. Medeiros, editors, *Proc. of 11th Euro-Par 2005 Parallel Processing*, volume 3648 of *LNCS*, pages 761–770, Lisboa, Portugal, August 2005. Springer.
 13. Denis Caromel and Mario Leyton. Fine tuning algorithmic skeletons. In Anne-Marie Ker-marrec, Luc Bougé, and Thierry Priol, editors, *Euro-Par*, volume 4641 of *Lecture Notes in Computer Science*, pages 72–81. Springer, 2007.
 14. Denis Caromel and Mario Leyton. Fine tuning algorithmic skeletons. In *13th International Euro-par Conference: Parallel Processing*, volume 4641 of *Lecture Notes in Computer Science*, pages 72–81, Rennes, France, 2007. Springer-Verlag.
 15. C. Chambers, B. Handerson, and J. Vlissides. A Debate on Language and Tool Support for Design Patterns. In *Proceedings of POPL 2000*.
 16. S. Ciarpaglini, M. Danelutto, L. Folchi, C. Manconi, and S. Pelagatti. ANACLETO: a template-based P3L compiler. In *Proceedings of the PCW'97*, 1997. Camberra, Australia.
 17. Philipp Ciechanowicz and Herbert Kuchen. Enhancing Muesli's Data Parallel Skeletons for Multi-Core Computer Architectures. In *Proc. of the 10th Intl. Conference on High Performance Computing and Communications (HPCC)*, pages 108–113, Los Alamitos, CA, USA, September 2010. IEEE.
 18. Murray Cole. *Algorithmic Skeletons: Structured Management of Parallel Computations*. Research Monographs in Par. and Distrib. Computing. Pitman, 1989.
 19. Murray Cole. Bringing skeletons out of the closet: A pragmatic manifesto for skeletal parallel programming. *Parallel Computing*, 30(3):389–406, 2004.
 20. Murray Cole. Bringing skeletons out of the closet: A pragmatic manifesto for skeletal parallel programming. *Parallel Comput.*, 30(3):389–406, March 2004.
 21. Rémi Coudarcher, Jocelyn Sérot, and Jean-Pierre Dérutin. Implementation of a skeleton-based parallel programming environment supporting arbitrary nesting. In *Proceedings of the 6th International Workshop on High-Level Parallel Programming Models and Supportive Environments*, HIPS '01, pages 71–85, London, UK, 2001. Springer-Verlag.
 22. M. Danelutto, R. Di Meglio, S. Orlando, S. Pelagatti, and M. Vanneschi. A methodology for the development and support of massively parallel programs. *Future Generation Computer Systems*, 8(1–3):205–220, July 1992.
 23. Marco Danelutto. Efficient support for skeletons on workstation clusters. *Parallel Processing Letters*, 11(1):41–56, 2001.
 24. Marco Danelutto and Massimo Torquati. *Structured Parallel Programming with “core” Fast-Flow*, pages 29–75. Springer International Publishing, Cham, 2015.
 25. Marco Danelutto, Massimo Torquati, and Peter Kilpatrick. A dsl based toolchain for design space exploration in structured parallel programming. *Procedia Computer Science*, 80:1519 – 1530, 2016. International Conference on Computational Science 2016, ICCS 2016, 6-8 June 2016, San Diego, California, USA.
 26. E. Gamma, R. Helm, R. Johnson, and J. Vissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison Wesley, 1994.
 27. Leonardo Gazzarri and Marco Danelutto. A tool to support FastFlow program design. In *Proceedings of ParCo 2017*. IOS Press, 2017. *RePara 2017 MiniSymposium: the 3rd International Workshop on Reengineering for Parallelism in Heterogeneous Parallel Platforms*, to appear.

28. Horacio González-Vélez and Mario Leyton. A survey of algorithmic skeleton frameworks: High-level structured parallel programming enablers. *Softw. Pract. Exper.*, 40(12):1135–1160, November 2010.
29. D. Goswami, A. Singh, and B. R. Preiss. Using Object-Oriented Techniques for Realizing Parallel Architectural Skeletons. In *Proceedings of the ISCOPE'99 conference*, pages 130–141. Springer Verlag, 1999. LNCS No. 1732.
30. N. Gunn. Using the Concurrency and Coordination Runtime, 2010. available at <http://www.infoq.com/articles/Using-CCR;jsessionid=9B1FE895F8A97410B7E1BEC40D18B33A>.
31. Infiniband web page, 2011. <http://www.infinibandta.org/>.
32. Intel. Intel Threading Building Blocks Design Patterns, 2010. available at http://threadingbuildingblocks.org/uploads/81/91/Latest%20Open%20Source%20Documentation/Design_Patterns.pdf.
33. Vladimir Janjic, Christopher Brown, K. Mackenzie, Kevin Hammond, Marco Danelutto, Marco Aldinucci, and J Daniel Garcia. RPL: A domain-specific language for designing and implementing parallel C++ applications. In *Parallel, Distributed, and Network-Based Processing (PDP), 2016 24th Euromicro International Conference on*, pages 288–295. IEEE, 2016.
34. C. W. Kessler. Symbolic Array Data Flow Analysis and Pattern Recognition in Numerical Codes. In K. M. Decker and R. M. Rehmann, editors, *Programming Environments for Massively Parallel Distributed Systems*, pages 57–68. Birkhauser, April 1994.
35. Herbert Kuchen. A skeleton library. In B. Monien and R. Feldman, editors, *Proc. of 8th Euro-Par 2002 Parallel Processing*, volume 2400 of LNCS, pages 620–629, Paderborn, Germany, August 2002. Springer.
36. Herbert Kuchen. A skeleton library. In *Proceedings of the 8th International Euro-Par Conference on Parallel Processing*, Euro-Par '02, pages 620–629, London, UK, UK, 2002. Springer-Verlag.
37. Joeffrey Legaux, Frédéric Louergue, and Sylvain Jubertie. Osl: An algorithmic skeleton library with exceptions. *Procedia Computer Science*, 18(Complete):260–269, 2013.
38. Mario Leyton and José M. Piquer. Skandium: Multi-core programming with algorithmic skeletons. In *Proceedings of the 2010 18th Euromicro Conference on Parallel, Distributed and Network-based Processing*, PDP '10, pages 289–296, Washington, DC, USA, 2010. IEEE Computer Society.
39. B. L. Massingill, T. G. Mattson, and B. A. Sanders. A Pattern Language for Parallel Application Languages. Technical Report TR 99-022, Univeristy of Florida, CISE, 1999.
40. B. L. Massingill, T. G. Mattson, and B. A. Sanders. A Pattern Language for Parallel Application Programs. In Bode, Ludwig, Karl, and Wismuller, editors, *Euro-Par 2000 Parallel Processing*, LNCS, No. 1900, pages 678–681. Springer Verlag, August 2000.
41. Kiminori Matsuzaki and Kento Emoto. Implementing fusion-equipped parallel skeletons by expression templates. In *Proceedings of the 21st International Conference on Implementation and Application of Functional Languages*, IFL'09, pages 72–89, Berlin, Heidelberg, 2010. Springer-Verlag.
42. Kiminori Matsuzaki, Kazuhiko Takehi, Hideya Iwasaki, Zhenjiang Hu, and Yoshiki Akashi. A Fusion-Embedded Skeleton Library. In *Euro-Par 2004 Parallel Processing, 10th International Euro-Par Conference*, pages 644–653. LNCS Springer, 2004.
43. Timothy Mattson, Beverly Sanders, and Berna Massingill. *Patterns for parallel programming*. Addison-Wesley Professional, first edition, 2004.
44. Timothy Mattson, Beverly Sanders, and Berna Massingill. *Patterns for Parallel Programming*. Addison-Wesley Professional, first edition, 2004.
45. S. McDonald, D. Szafron, J. Schaeffer, and S. Bromling. From Patterns to Frameworks to Parallel Programs. submitted to Journal of Parallel and Distributed Computing, December 2000.
46. Myricom. Myrinet overview, 2011. <http://www.myri.com/myrinet/overview/>.

47. J. Serot, D. Ginhac, and J.P. Derutin. SKiPPER: A Skeleton-Based Parallel Programming Environment for Real-Time Image Processing Applications. In *Proceedings of the 5th International Parallel Computing Technologies Conference (PaCT-99)*, September 1999.
48. SkeTo project home, 2011. <http://www.ipl.t.u-tokyo.ac.jp/sketo/>.
49. Haruto Tanno and Hideya Iwasaki. Parallel skeletons for variable-length lists in sketo skeleton library. In *Euro-Par '09: Proceedings of the 15th International Euro-Par Conference on Parallel Processing*, pages 666–677. Springer-Verlag, 2009.
50. Marco Vanneschi. The programming model of ASSIST, an environment for parallel and distributed portable applications. *Parallel Computing*, 28(12):1709–1732, December 2002.
51. Marco Vanneschi. *High Performance Computing: Parallel Processing Model and Architectures*. Pisa University Press, 2014.